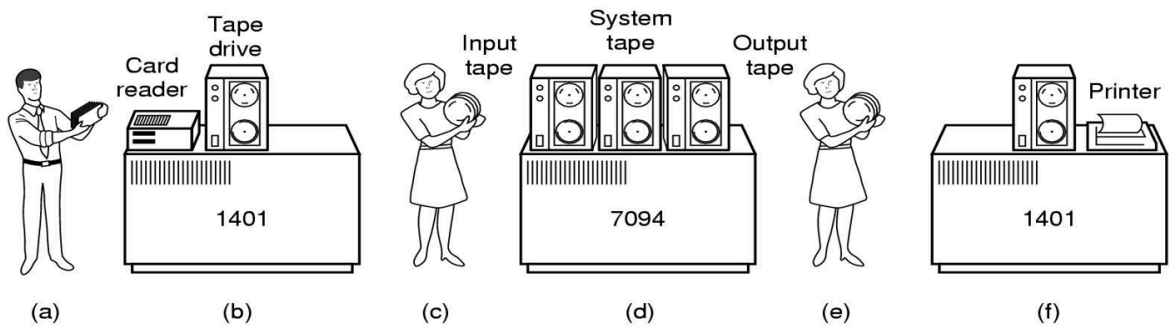


《操作系统》学习笔记

第一章 引言

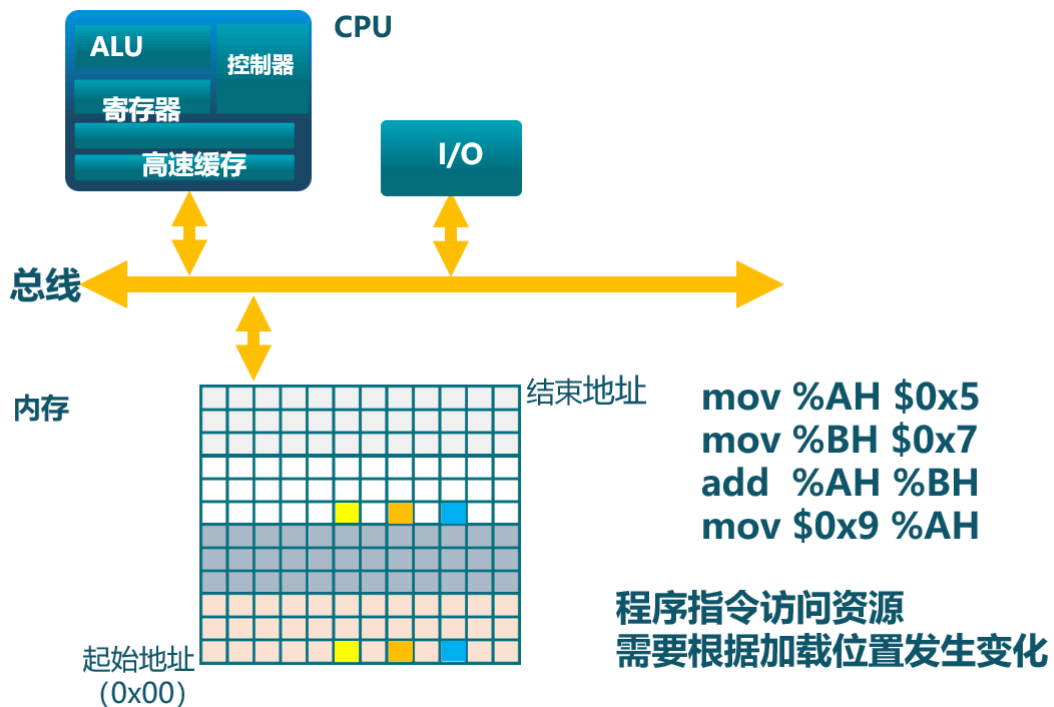
早期的操作系统的样子：



过程：

- bring cards to 1401
- read cards to tape
- put tape on 7094 which does computing
- put tape on 1401 which prints output

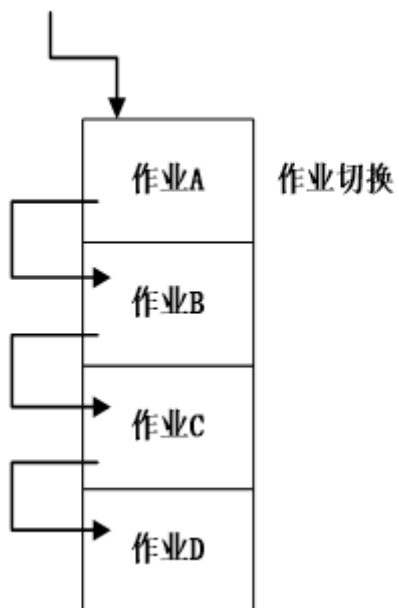
操作系统需要处理的：



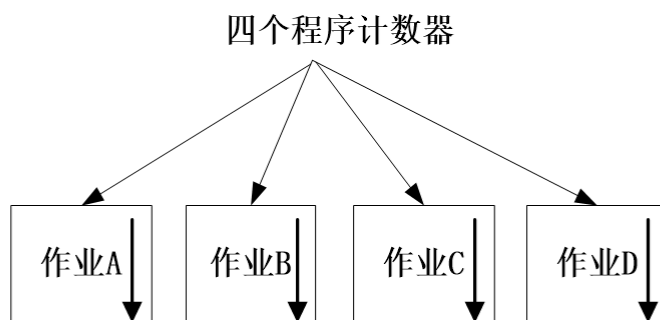
内存扩大的情况下，我们的程序能不能正常运行？

将内存分配给更多的进程来使用，我们只需要将后面的进程的内存位置相对移动即可

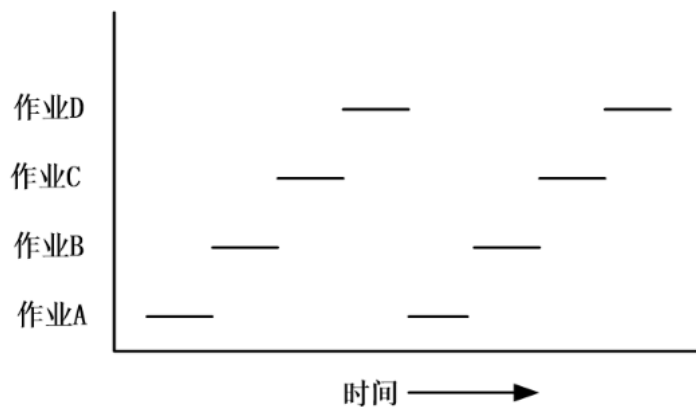
单步执行：每个任务都依次执行，执行时，不需要考虑其他的任务



多步执行：就是多个程序一起运行



分时复用：



每个任务都占用了一个时间片，只要把时间切的足够碎，就可以造成并行执行的假象了。

但是，怎么将一个程序运行一段时间后，再停下来呢？

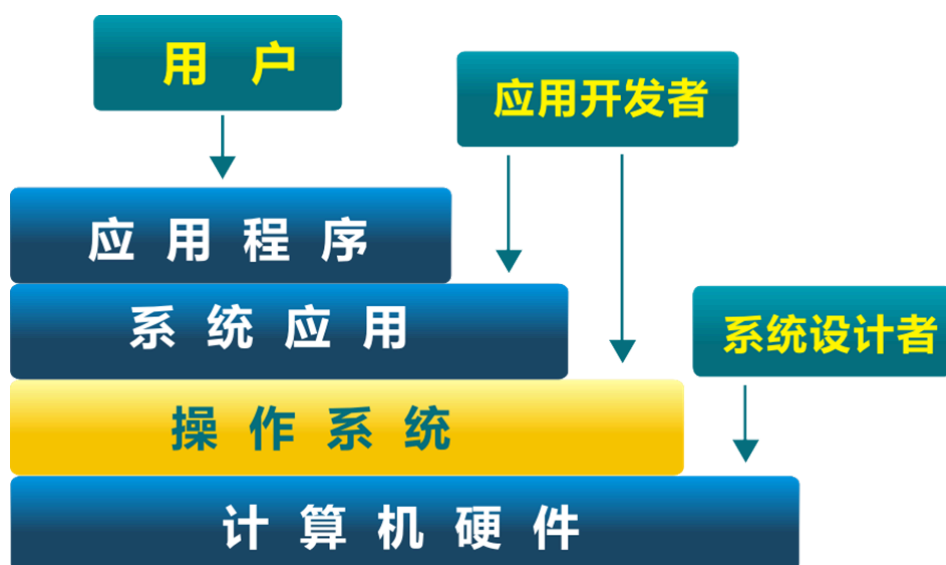
我们需要将原先运行的程序进行存档，然后对接下来需要运行的程序进行读取。——涉及到进程的上下文切换

操作系统是一个**控制程序**

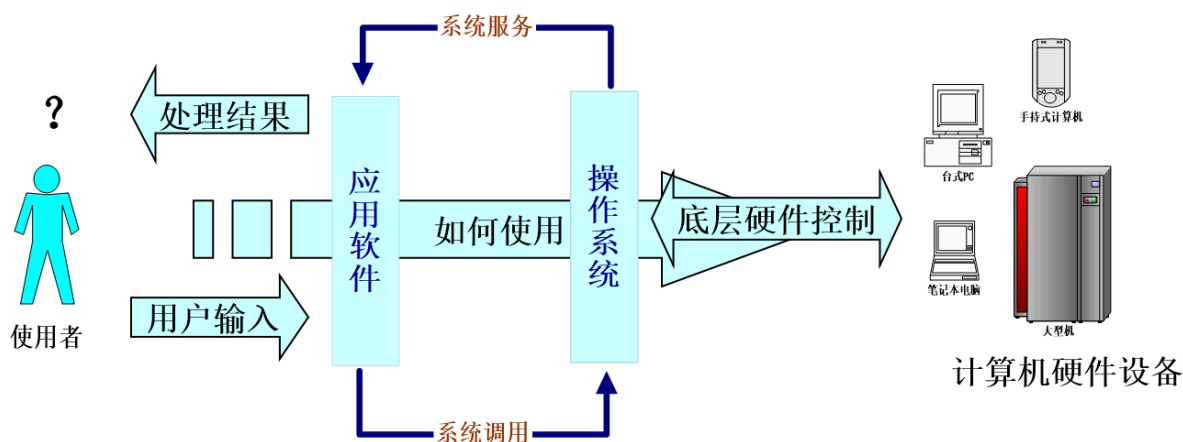
- 一个软件
- 控制程序执行过程，防止错误和计算机的不当使用（阻止应用相互破坏）
- 方便用户使用计算机系统（提供键盘、鼠标、多点触控、图形UI等）

操作系统是一个**资源管理器**

- 应用程序与硬件之间的中间层
- 管理各种计算机软硬件资源，给用户程序提供服务
- 提供访问计算机软硬件资源的高效手段
- 解决资源访问冲突，确保资源公平使用



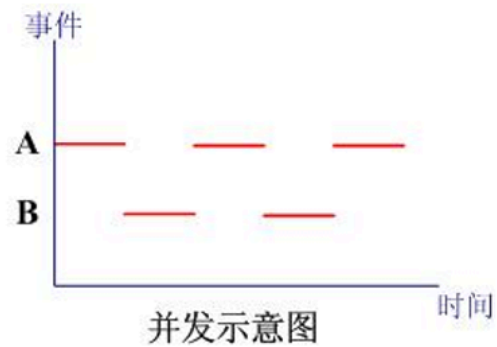
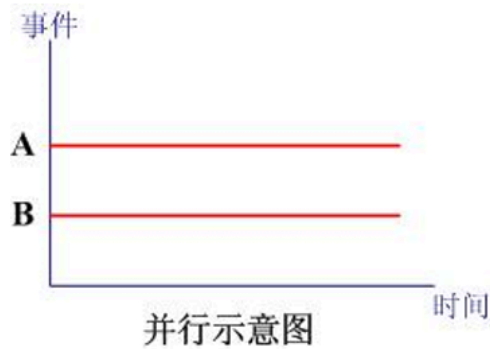
操作系统处于计算机硬件和系统应用之中



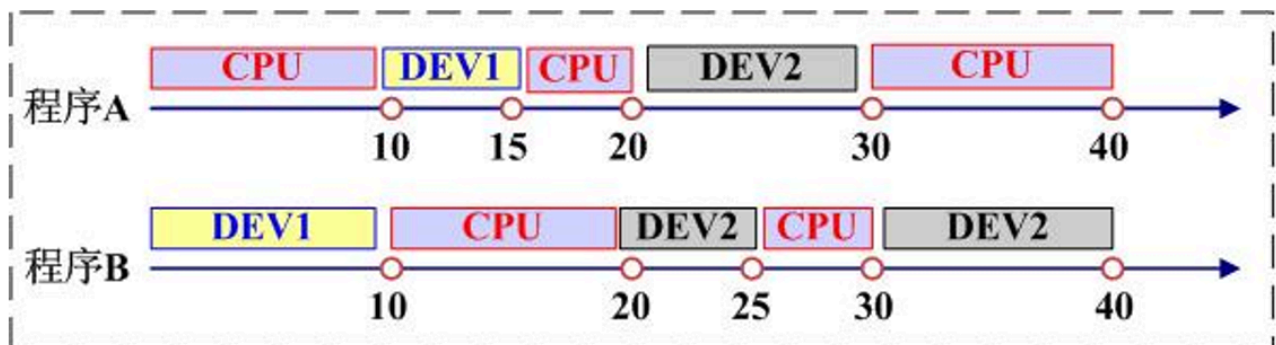
应用软件通过系统调用，来请求操作系统的功能，操作系统给应用软件提供对应的系统服务。

第二章 进程

- 并行：多个任务在同一时间同时运行
- 并发：微观层面，同一时刻只有一个任务在运行；宏观层面，多个任务在差不多相同的时间推进

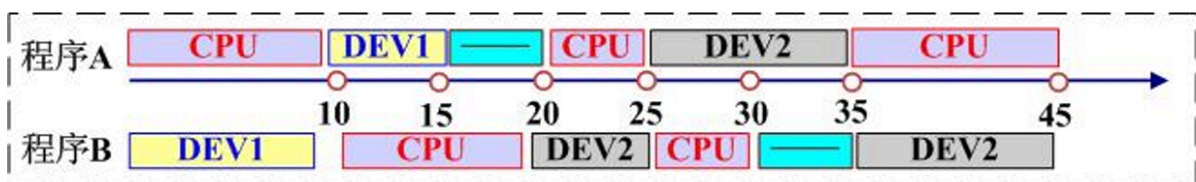


顺序执行：程序总是交替的使用硬件资源



一个程序不会一直使用CPU，而是交替着进行使用。

并发执行：



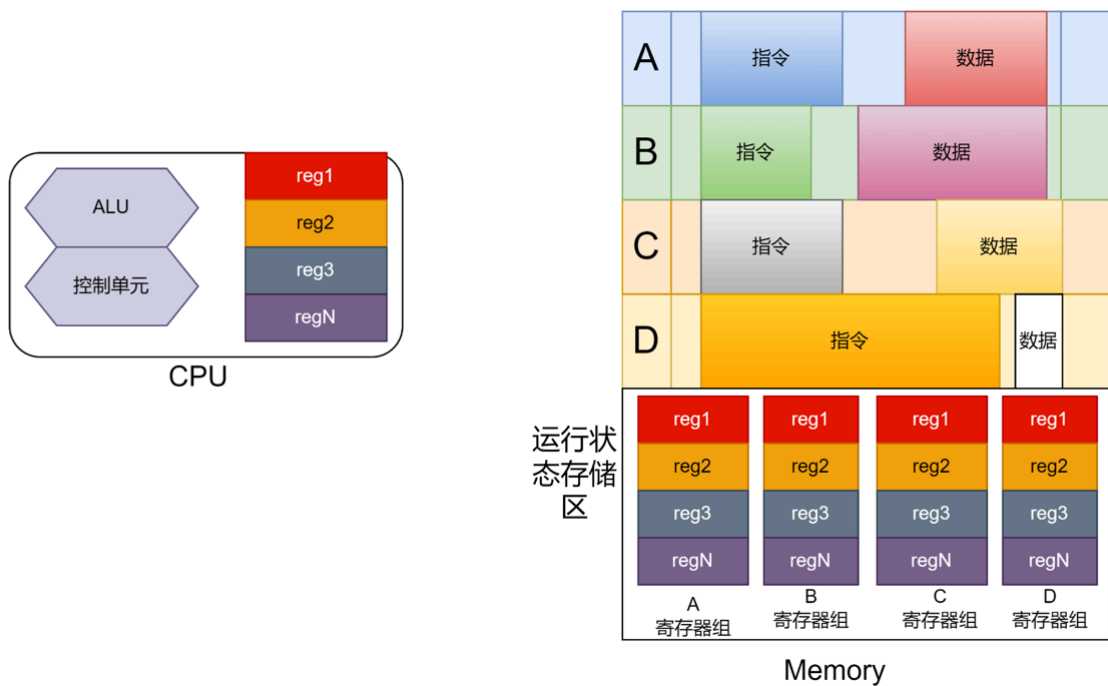
我们发现，并发执行的时候，我们可以节省下一定的时间。上面的蓝色的空挡，说明肯定有人在等，肯定有CPU的部分时间被浪费了。

最关键的问题：

- 怎么保存和恢复一个程序的执行过程？
- 怎么知道这个程序不想用CPU了？

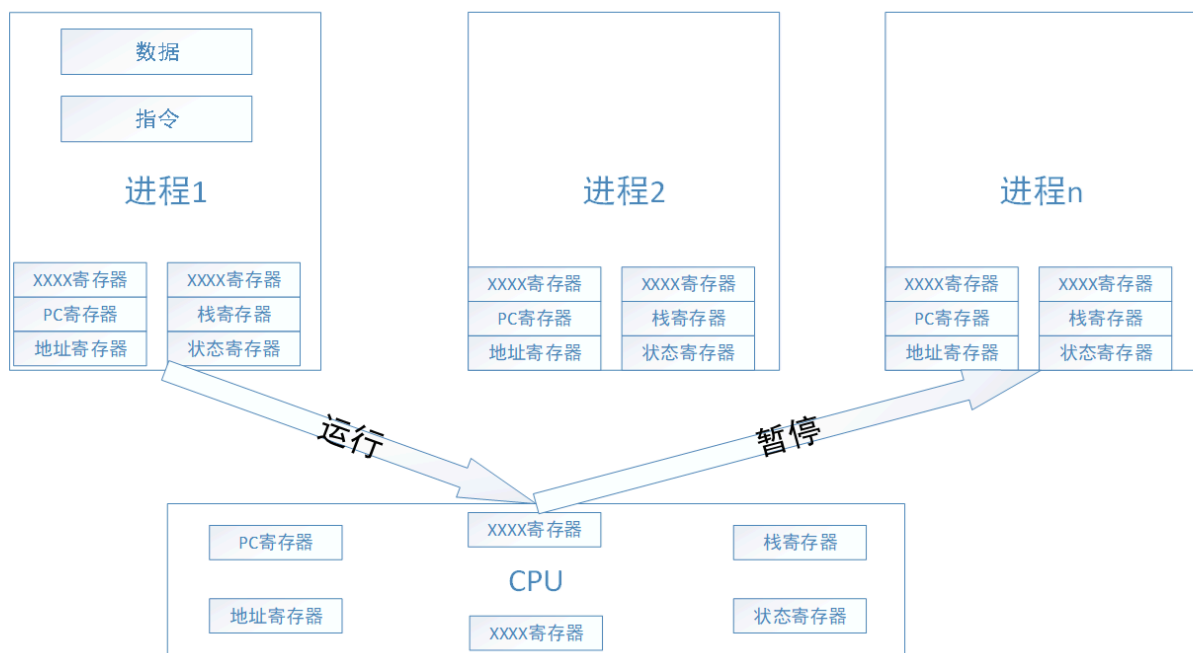
怎么保存和恢复一个程序的执行过程？——我们只需要保存和恢复程序执行过程的寄存器就行

- 写存档：写入所有的寄存器的值
- 读存档：读取所有的寄存器的值



将我们的寄存器状态都存储到**内存**中去，在内存中开辟一块空间，然后进行读取和存储。

一个程序的执行状态对应于CPU上的寄存器值



如果进程1不想运行了，那么就将我们的运行状态存储到CPU中的内存中的寄存器中，然后读取另外一个寄存器中的数据，然后切换到另一个进程。

一个程序不使用CPU的时候，就是在使用一些cin, cout函数等，这样就不调用CPU

也就是说，程序在使用其他的设备

也就是说，当程序调用其他的函数的时候，基本上就是不使用CPU了

每个程序都是间歇的使用CPU，在不使用的瞬间进行存档，然后系统会调用另外一个需要使用CPU的程序进行读档

进程：我们可以理解成把一个程序运行过程中使用到的**数据、指令**等私有信息，与每一个运行时的程序都需要保存的“**寄存器组**”和其他公有信息，以及用于改变“是不是使用CPU”状态的相关函数接口，合并封装成一个“类”并给这个类起名为“进程”，每个系统中的运行着的程序都是一个进程的对象。

进程和程序：

- **进程是动态的，程序是静态的**：程序是有序代码的集合；进程是程序的执行。通常进程不可在计算机之间迁移；而程序通常对应着文件、是静态的并可以复制。
- **进程是暂时的，程序是永久的**：进程是一个状态变化的过程，程序可长久保存。
- **进程与程序的组成不同**：进程的组成包括程序、数据和进程控制块。程序由语句和指令构成。
- **进程与程序有对应关系**：通过多次执行，一个程序可对应多个进程；通过调用关系，一个进程可包括多个程序。

程序想要执行会被加载为“进程”；指令需要进入内存才能被处理器执行；与指令配套的数据也需要加载入内存；与程序运行状态密切相关的寄存器等信息也需要存储在内存中。

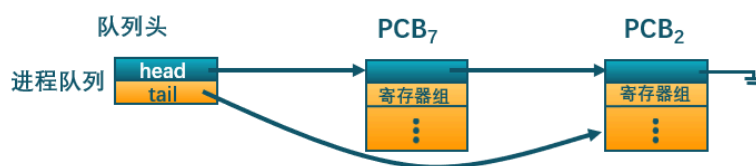
进程的数据结构设计：

```
class Process
{
private:
    //指向保存程序中的指令的内存区的地址
    unsigned char* instructions;
    //指向保存程序中的数据内存区的地址
    unsigned char* data;

public:
    //保存程序状态对应的寄存器
    unsigned long registergroups[];
    //进程的标识
    unsigned int ID;
}
```

PCB：管理进程的数据结构，内核的进程状态记录

内核为每个进程维护了对应的进程控制块（PCB），系统中存在不定数量的进程，因此内核进程的PCB组织为链表



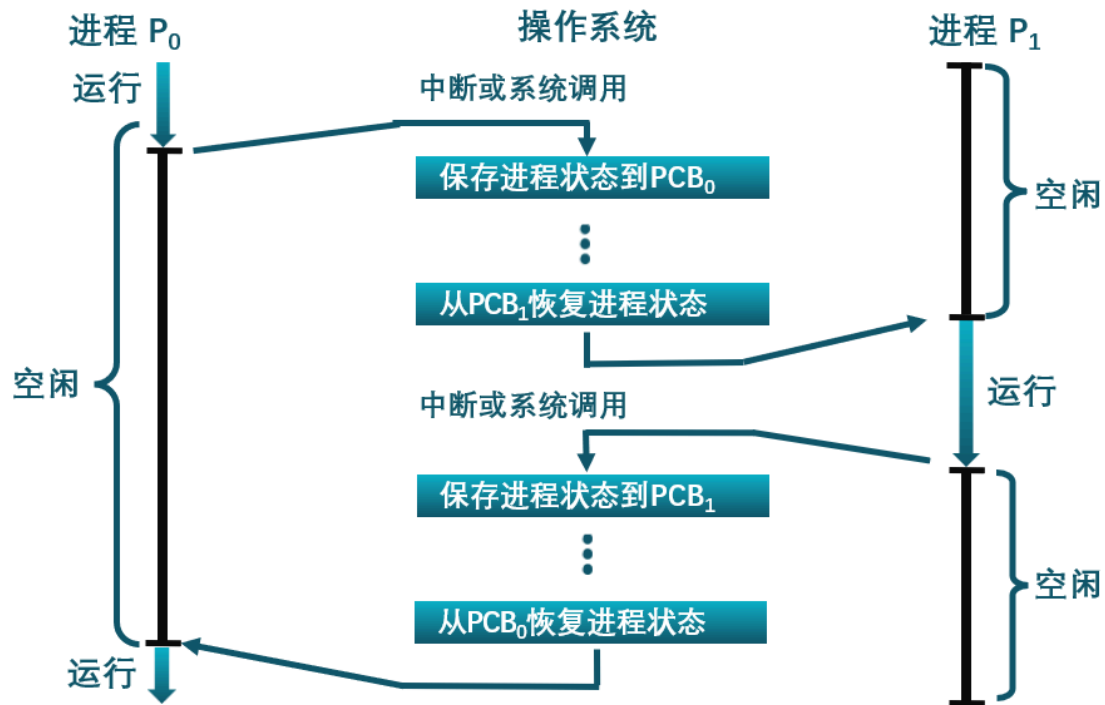
当进程切换的时候，需要暂停当前的进程，然后选择一个其他的进程来进行运行。

进程切换时的存档，被称为上下文（context）

- 切换前，保存进程上下文
- 切换后，恢复进程上下文

上下文中包含了进程运行状态相关的信息，包括**寄存器、CPU的状态、内存地址空间**。

上下文切换的具体流程：



switch_to的实现：

```
/* Save the registers */
sw    sp, 44(a0)
sw    ra, 40(a0)
sw    gp, 36(a0)
sw    s8, 32(a0)
sw    s7, 28(a0)
sw    s6, 24(a0)
sw    s5, 20(a0)
sw    s4, 16(a0)
sw    s3, 12(a0)
sw    s2, 8(a0)
sw    s1, 4(a0)
sw    s0, 0(a0)
nop

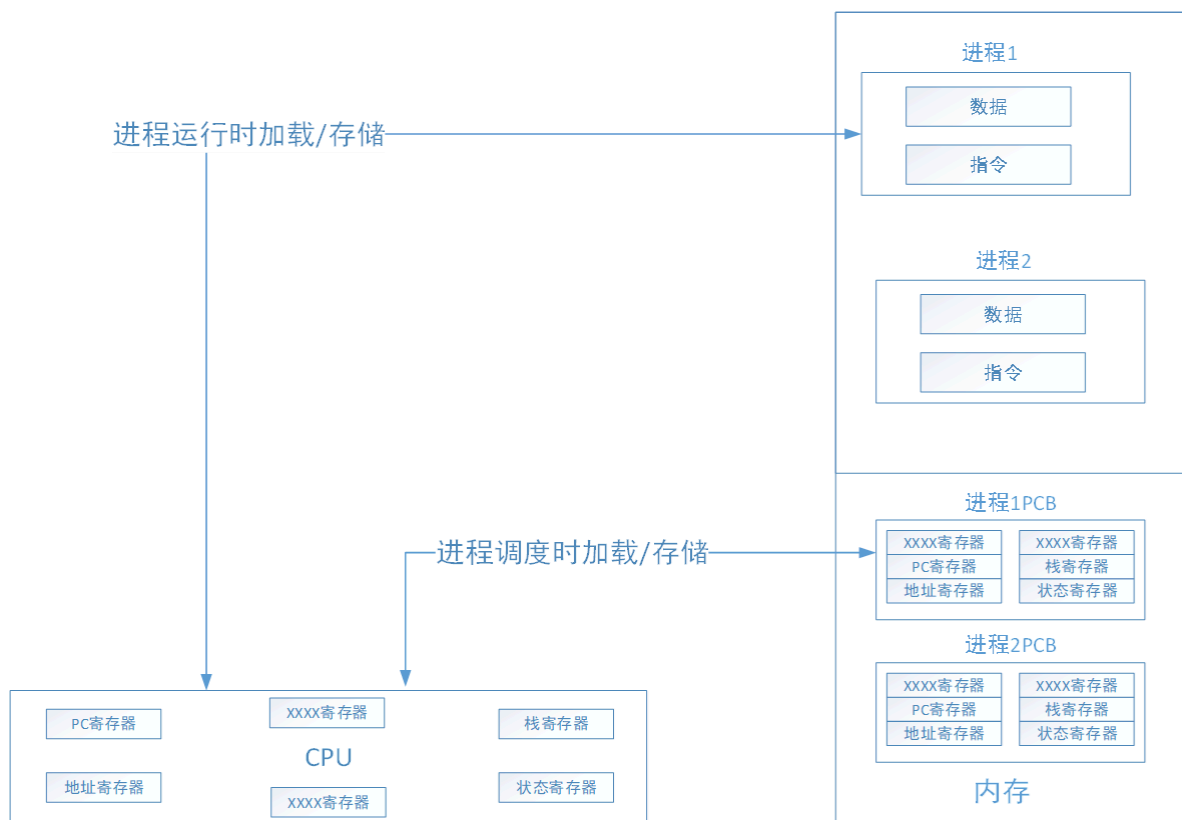
/* Now, restore the registers */
lw    s0, 0(a1)
lw    s1, 4(a1)
lw    s2, 8(a1)
lw    s3, 12(a1)
lw    s4, 16(a1)
lw    s5, 20(a1)
lw    s6, 24(a1)
lw    s7, 28(a1)
lw    s8, 32(a1)
lw    gp, 36(a1)
lw    ra, 40(a1)
lw    sp, 44(a1)
nop      /* delay slot for load */
/* and return. */
j ra
```

`switch_to(from, to)`: `from`就是我们前面需要存储寄存器的值的进程，`to`就是我们后面需要读取寄存器的值的进程

举个例子：`switch_to(from, to)`相当于完成了`from`进程的内容的存储，以及调用`switch_to`函数，加载`ra`寄存器到了`to`的地址上，这样，我们成功跳转到了`to`进程的起始位置。

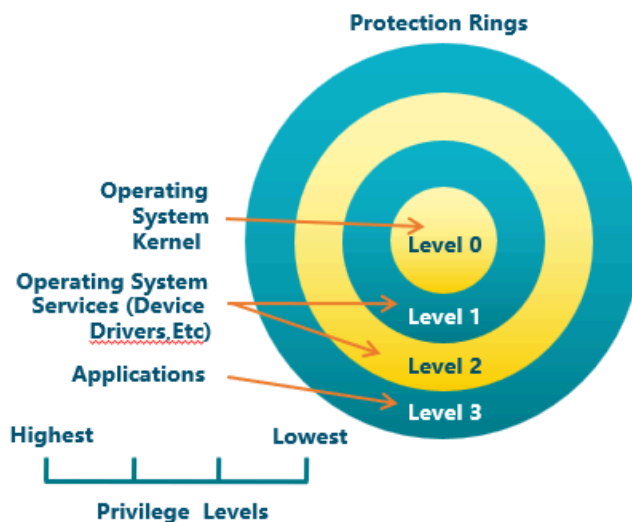
`j ra`就是跳转到我们的`switch_to`函数的返回地址，相当于我们可以找回我们原来被中断的程序地址

进程调度：



进程的管理和调度：我们在实现函数的过程中，为什么要调用`switch_to`函数呢？不写这个函数不就不会被中断了吗？

CPU的设计规避了这一点，采用了权限级别的区分。我们正常的应用程序是不能调用高级别的函数的。那就是说，我们在调用`print`函数时，**我们只能被迫执行`switch_to`函数**，将自己的运行权交给别人



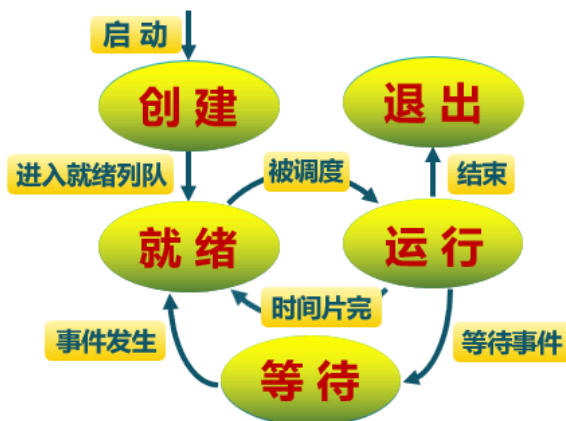
CPU中具有两种形态——用户态和特权态

所以，在应用程序调用print函数的时候，会触发系统调用内核函数write，就会触发调用对应的switch_to，应用程序是无法规避这个系统函数的。

——相当于调用了“外设控制”对应的函数所以被暂停

下一个问题就是，我们在中断了原来的程序之后，下一个执行的程序选择谁去执行呢？

我们需要规定对应的状态模型——**五状态模型**



- 运行状态：进程正在处理机上运行
- 等待状态：进程调用了switch_to函数，处于等待状态
- 就绪状态：进程获得了除处理机之外的所需资源，得到处理机即可运行

总体流程：

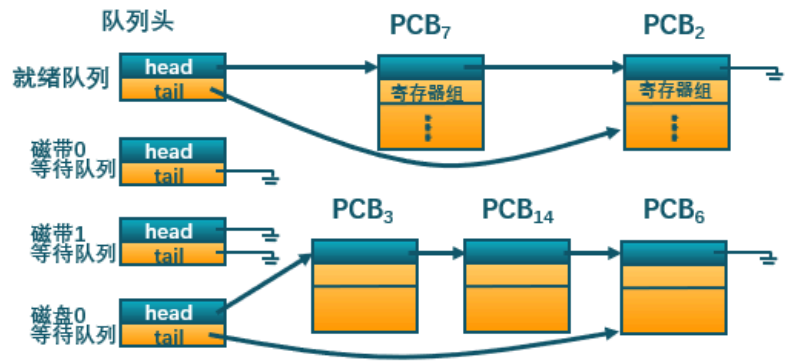
- 首先，创建一个新的进程，用于执行一个程序
- 创建态到就绪态，当进程被创建完毕之后，一切就绪准备运行，就到了就绪态
- 就绪态到运行态，该进程被调度程序选中之后，就分配到处理机中来运行
- 运行到结束，如果完成了功能，就直接退出对应的进程，到了结束的状态
- 运行到就绪，如果调用了中断函数，导致变成了就绪态，等待调度程序的选中
- 运行到等待，就是请求了write之类的函数，然后必须要进行等待的
- 等待到就绪，就是程序收到了某事件，直接从等待态到了就绪态

内核为每个进程维护了对应的进程控制块（PCB），内核将相同状态的进程的PCB放置在同一队列

■ 就绪队列

■ I/O等待队列

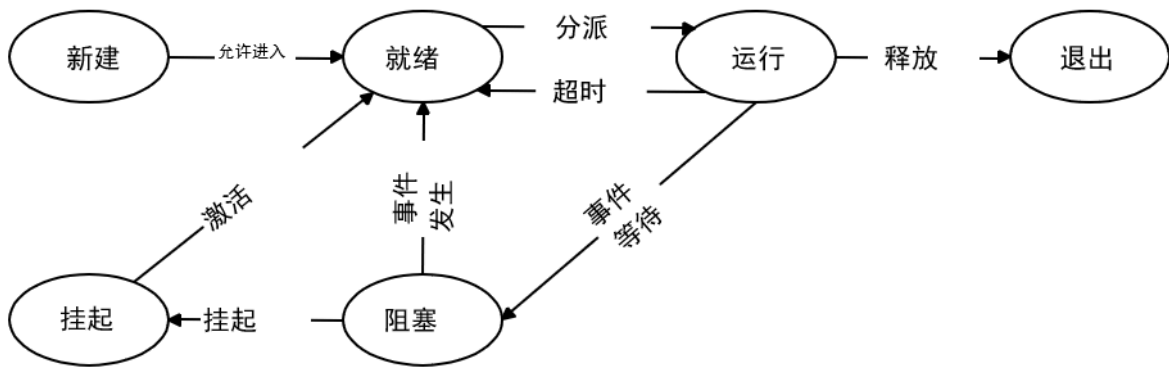
■ 每个设备一个队列



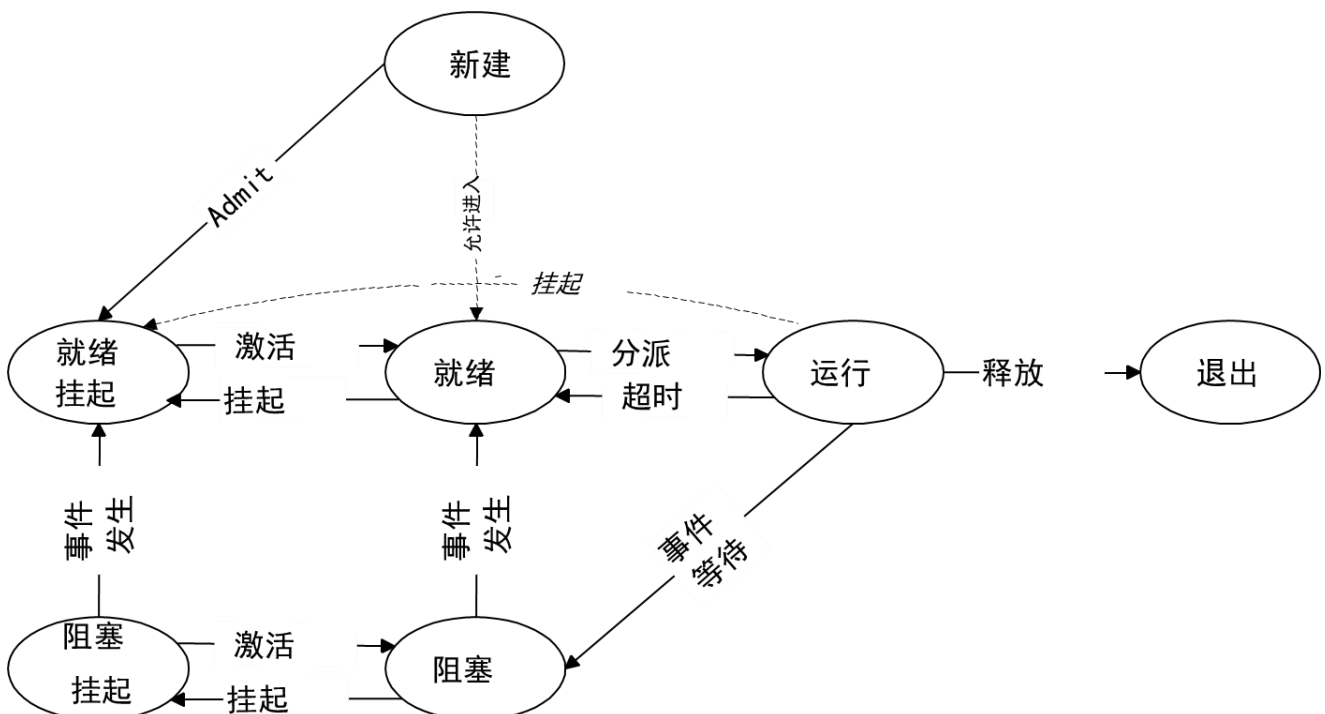
取链表的头部，这样的复杂度是最低的，为 $O(1)$

挂起进程模型：某进程长时间不能运行可以换至外存——就是挂起状态，处于阻塞状态之后，一直没有运行，说明这个进程可以被挂起！

单挂起进程模型：



双挂起进程模型：



- 就绪状态(Ready)：进程在内存且可立即进入运行状态；

- 阻塞状态(Blocked)：进程在内存并等待某事件的出现；
- 阻塞挂起状态 (Blocked, suspend)：进程在外存并等待某事件的出现；还在阻塞当中
- 就绪挂起状态 (Ready, suspend)：进程在外存，但只要进入内存，即可运行，就绪了，进入内存就可以运行了。

挂起与激活：

- 挂起：把一个进程从内存拿到外存
- 激活：把一个进程从外存拿到内存

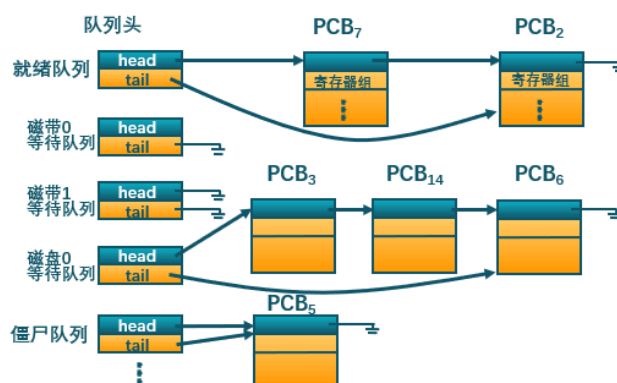
内核将挂起状态的PCB移出（压缩）以节省资源，也就是说挂起状态的进程就没有PCB在队列中了

就绪队列

I/O等待队列

- 每个设备一个队列

僵尸队列



外部存储（例如硬盘）

挂起队列



进程调度的代价：

Table 2.2 Example Time Scale of System Latencies

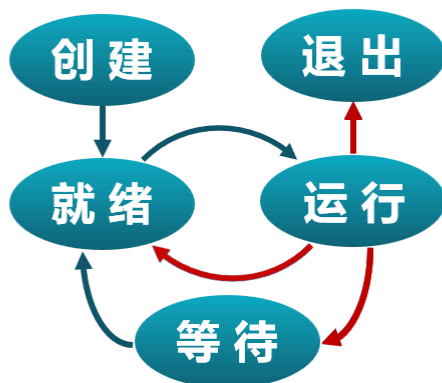
Event	Latency	Scaled
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access (DRAM, from CPU)	120 ns	6 min
Solid-state disk I/O (flash memory)	50–150 μ s	2–6 days
Rotational disk I/O	1–10 ms	1–12 months
Internet: San Francisco to New York	40 ms	4 years
Internet: San Francisco to United Kingdom	81 ms	8 years
Internet: San Francisco to Australia	183 ms	19 years
TCP packet retransmit	1–3 s	105–317 years
OS virtualization system reboot	4 s	423 years
SCSI command time-out	30 s	3 millennia
Hardware (HW) virtualization system reboot	40 s	4 millennia
Physical system reboot	5 m	32 millennia

进程的调度所消耗的时间十分漫长——对于CPU来说

进程的调度算法：需要考虑速度和效率

需要考虑调度策略和调度时机

调度时机：什么时候需要去调度我们的进程？



当进程从运行状态切换到等待状态的时候，进程被终结了

- 非抢占系统：当前的进程主动放弃CPU
- 可抢占系统：当前的进程被抢占了

那些处于就绪状态的进程，是怎么排队的呢？——如何进行进程的管理？**下面就是一些进程的调度算法！！重要**

1.先来先服务算法(FCFS)

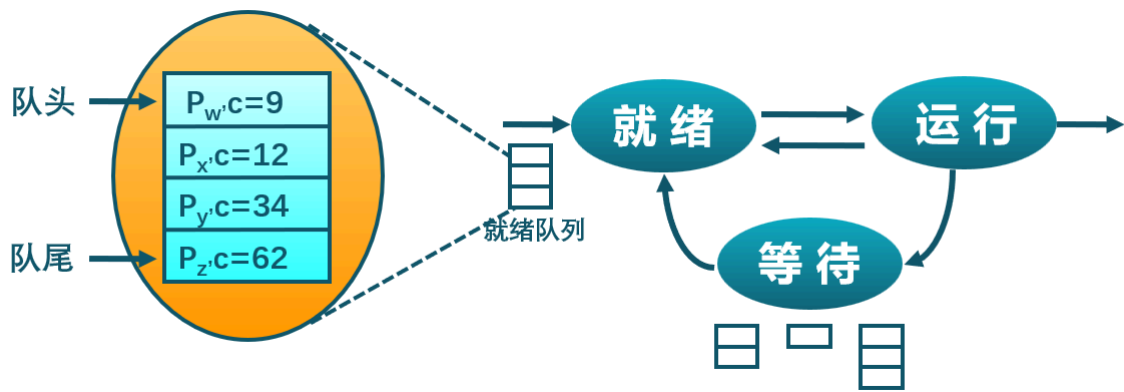
依据进程进入就绪状态的先后顺序排列，进程进入等待或结束状态时，就绪队列中的下一个进程占用CPU



- 优点：简单
- 缺点：平均等待的时间波动很大，短的进程可能排在长的进程后面，CPU资源的利用率较低

2.短进程优先算法(SPN)

选择就绪队列中执行时间最短进程占用CPU进入运行状态，就绪队列按预期的执行时间来排序



短进程优先算法具有**最优平均周转时间**

缺点：连续的短进程流会使长进程无法获得CPU资源，而且我们不能预知进程的使用时间，需要进行预测

3.最高响应比优先算法(HRRN)

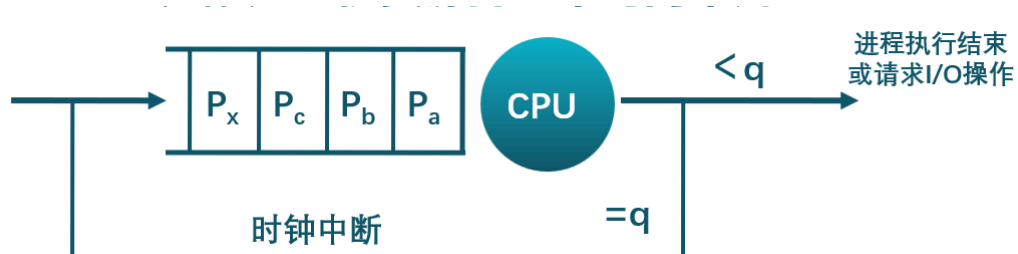
选择就绪队列中**响应比R值**最高的进程

- w: 等待时间(waiting time)
- s: 执行时间(service time)

$$R = (w + s) / s$$

4.时间片轮转算法(RR, Round-Robin)

时间片：分配处理机资源的基本时间单元



时间片结束时，按FCFS算法切换到下一个就绪进程；每隔(n-1)个时间片进程执行一个时间片q

例子：时间片为20

示例: 4个进程的执行时间如下

P1 53
P2 8
P3 68
P4 24

甘特图如下:



也就是说，如果一个时间片走完了，还没有运行完的话，就先让出资源；如果时间片还没到的话，已经运行完了，就直接让出资源，进入下一个时间片即可。

该方法的额外开销：就是额外的上下文切换——时间片的频繁切换会导致程序切换过多

- 当时间片过大的时候，等待的时间过长；
- 当时间片较小的时候，反应迅速，但是产生了过多的上下文切换，会影响系统的吞吐量

5.多级队列调度算法(MQ)——先分类，再使用策略

就绪队列被划分成多个独立的子队列，每个队列拥有自己的调度策略

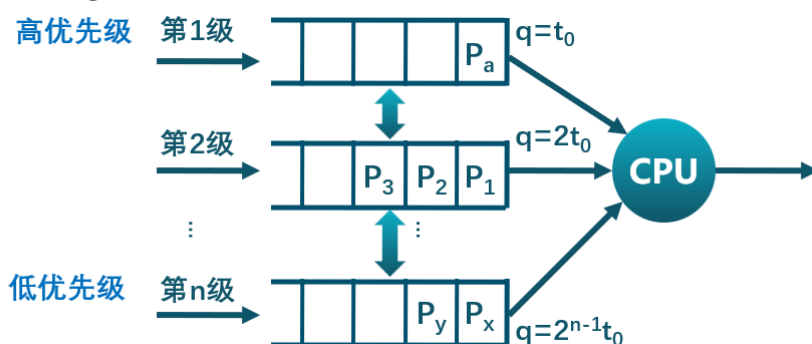
队列间的调度：

- 固定优先级：先处理前台，然后处理后台；可能导致饥饿
- 时间片轮转：每个队列都得到一个确定的能够调度其进程的CPU总时间，比如80%CPU时间用于前台，20%CPU时间用于后台

6.多级反馈队列算法(MLFQ)

进程可在不同队列间移动的多级队列算法

- 时间片大小随优先级级别增加而增加
- CPU密集型进程的优先级下降很快
- I/O密集型进程停留在高优先级
- 如进程在当前的时间片没有完成，则降到下一个优先级

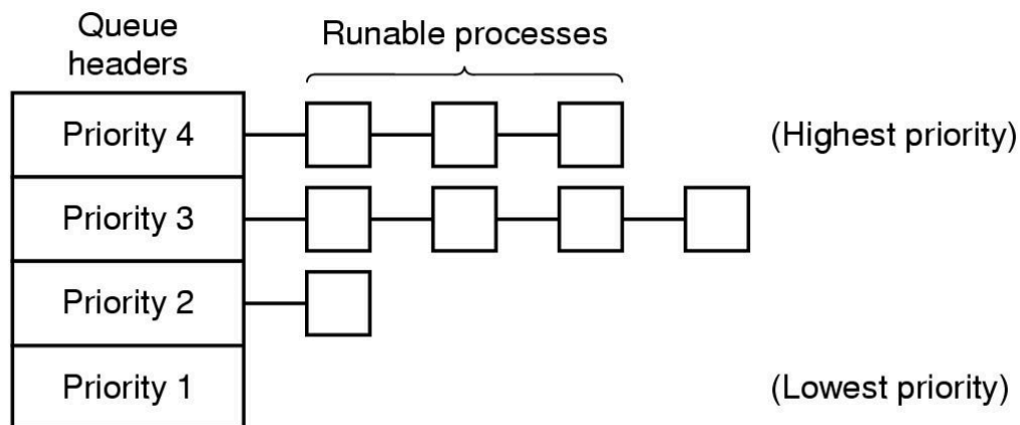


7.优先级算法(Priority Scheduling)

是多级队列算法的改进，平衡各进程对响应时间的要求。可适用作业和进程调度，可分成抢先式和非抢先式。

- 静态优先级：优先级在创建进程时就确定，直到进程终止前都不改变。通常是一个整数
 - 进程类型（系统进程优先级较高）
 - 对资源的需求（对CPU和内存需求较少的进程，优先级较高）
 - 用户要求（紧迫程度和付费多少）

▪



- 动态优先级：优先级在创建进程时就赋予，**在进程运行中被改变**
 - 在就绪队列中，等待时间延长则优先级提高，从而使优先级较低的进程在等待足够的时间后，其优先级提高到可被调度执行；——等的时间越长，就给他提高优先级
 - 进程每执行一个时间片，就降低其优先级，从而使一个进程持续执行时，其优先级降低直到可出让CPU。——如果一直给一个进程运行，那么就降低他的优先级
- 线性优先级调度算法（SRR, Selfish Round Robin）

调度算法（策略）设计的目标：一般来说，对于不同的人，对应的策略是不一样的

调度算法的比较——定义指标

- CPU使用率：CPU处于忙状态的时间百分比
- 吞吐量：单位时间内完成的进程数量
- 周转时间：进程从初始化到结束(包括等待)的总时间
- 等待时间：进程在就绪队列中的总时间
- 响应时间：从提交请求到产生响应所花费的总时间

以响应时间为目标：

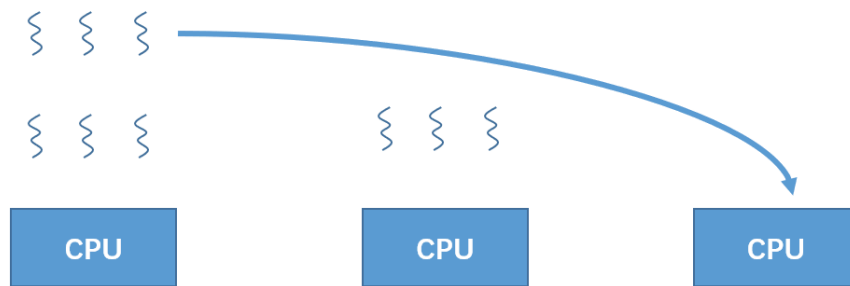
- 减少响应时间
- 减少平均响应时间的波动

以吞吐量为目标：

- 增加吞吐量
- 减少等待时间

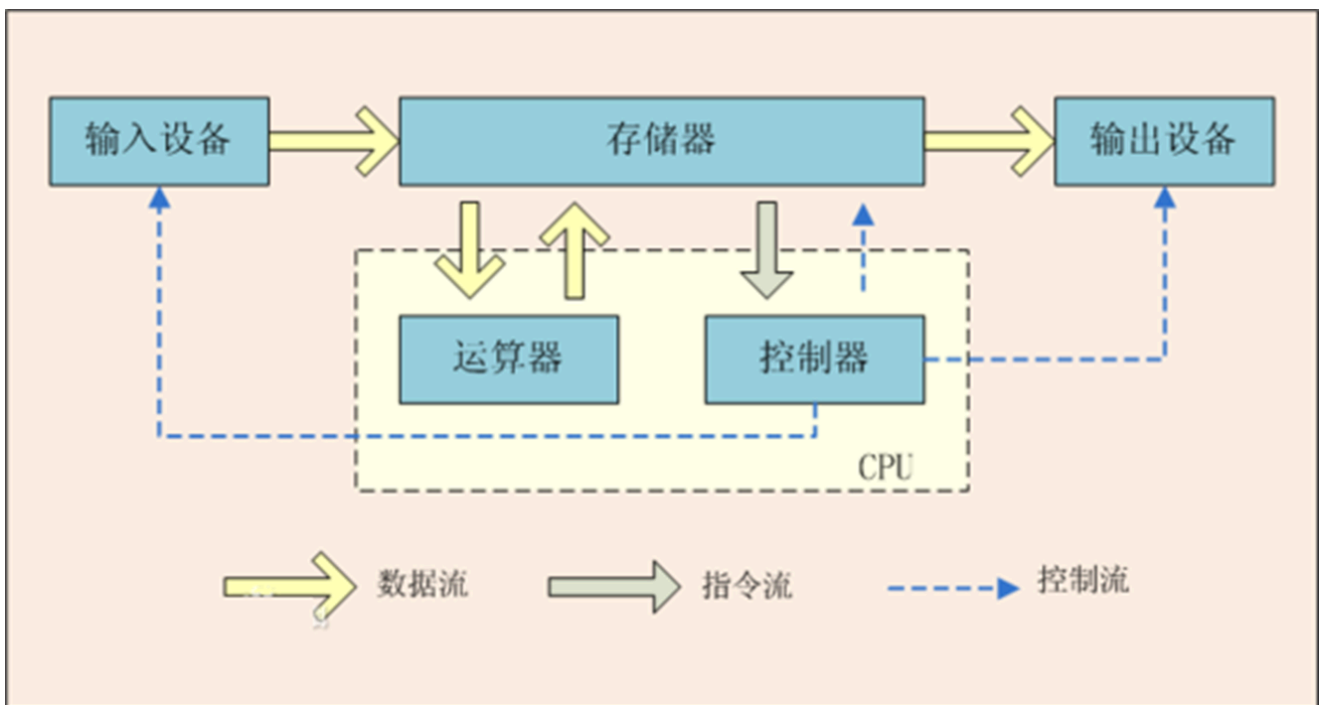
CPU多核的进程设计：将任务从高负载的CPU移动到低负载的CPU上

追踪每个CPU上的负载
把任务从高负载的CPU移动到低负载的CPU



这样可以将CPU的负载进行一个平均，不会出现有一个CPU很过载，但是其他的很空闲这样的情况！

第三章 内存管理



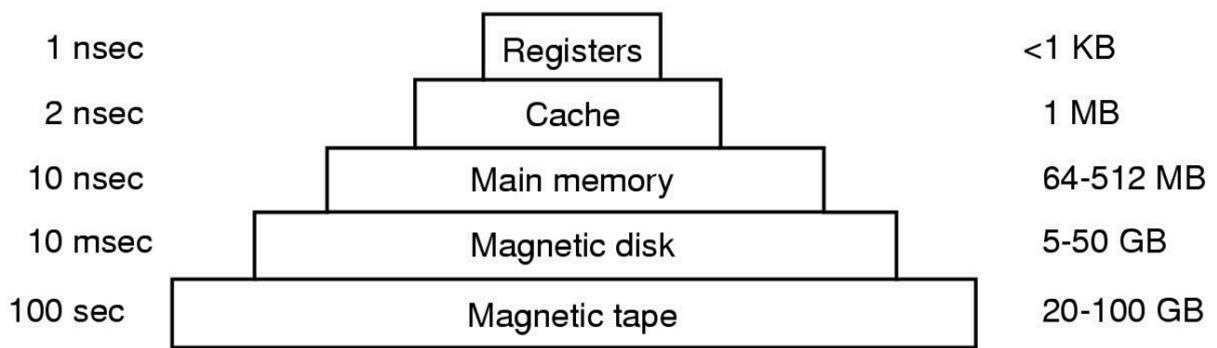
冯诺依曼体系结构：**存储器**就是所有部件的连接核心

系统的存储结构：

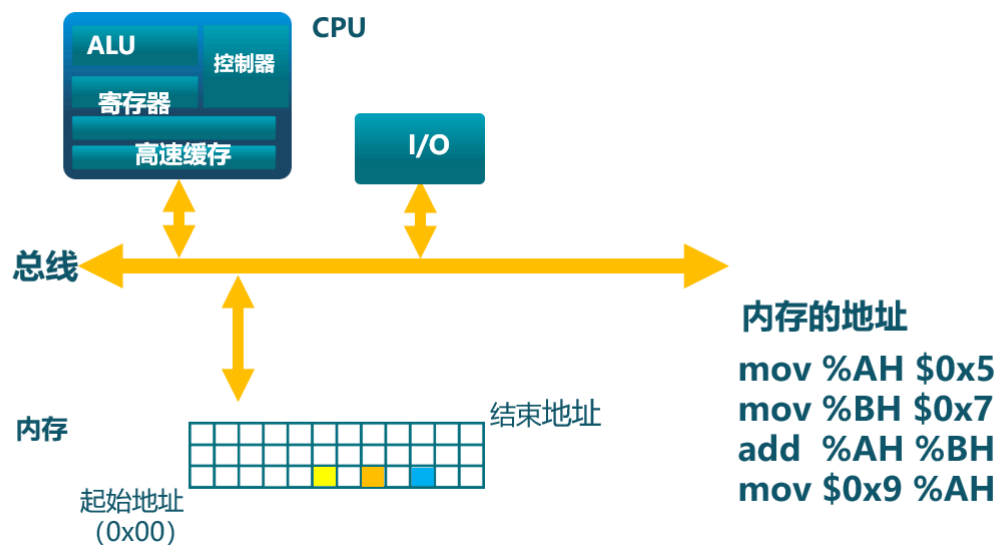
- 少量快速、昂贵存储器——cache
- 一些中级速度, 价格适中的主存储器
- 大量低速的, 便宜的, 仅供批量访问的磁盘存储器

Typical access time

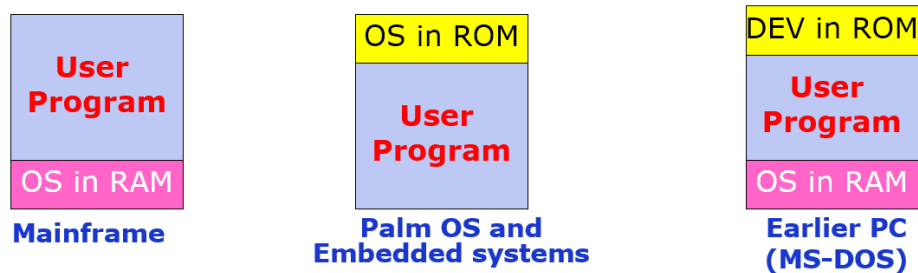
Typical capacity



内存地址:

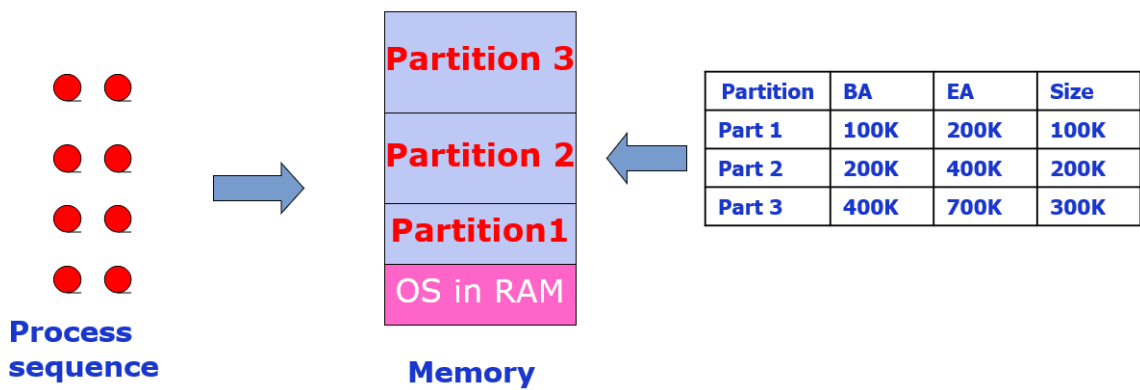


单道程序: 操作系统与应用程序使用同一个地址空间

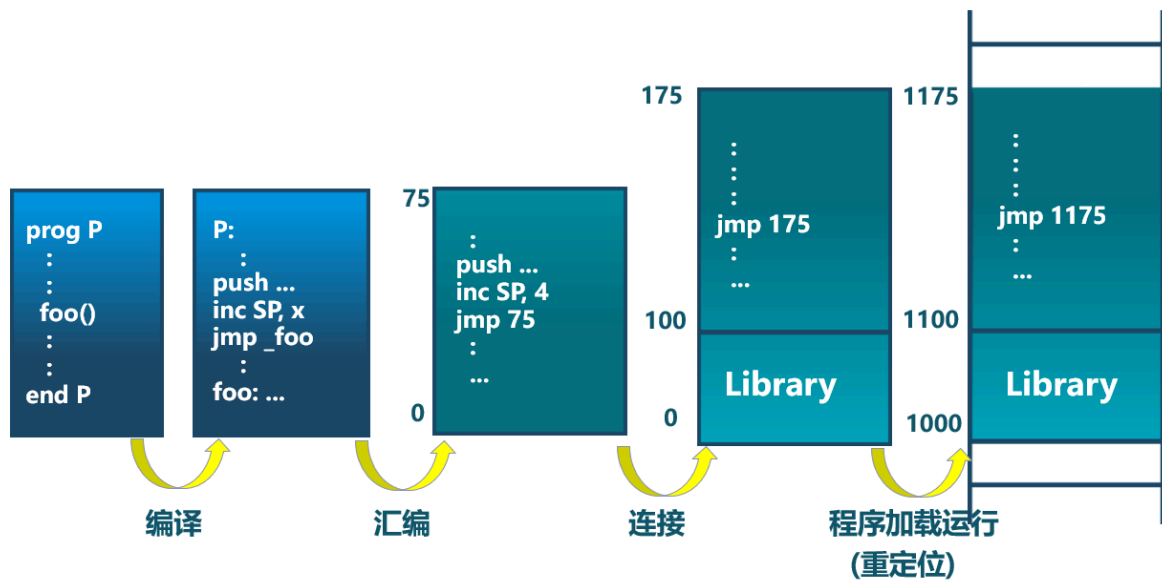


- 优点: 易于管理, 软硬件支持需要少
- 缺点: 内存浪费大; 不必要的内存占用; 程序及数据共享性差; **系统安全性差**——用户说不定就可以访问系统内核的数据

多道程序: 使用分区固定的方法——将内存分成固定大小的块, 并加载适合大小的应用程序

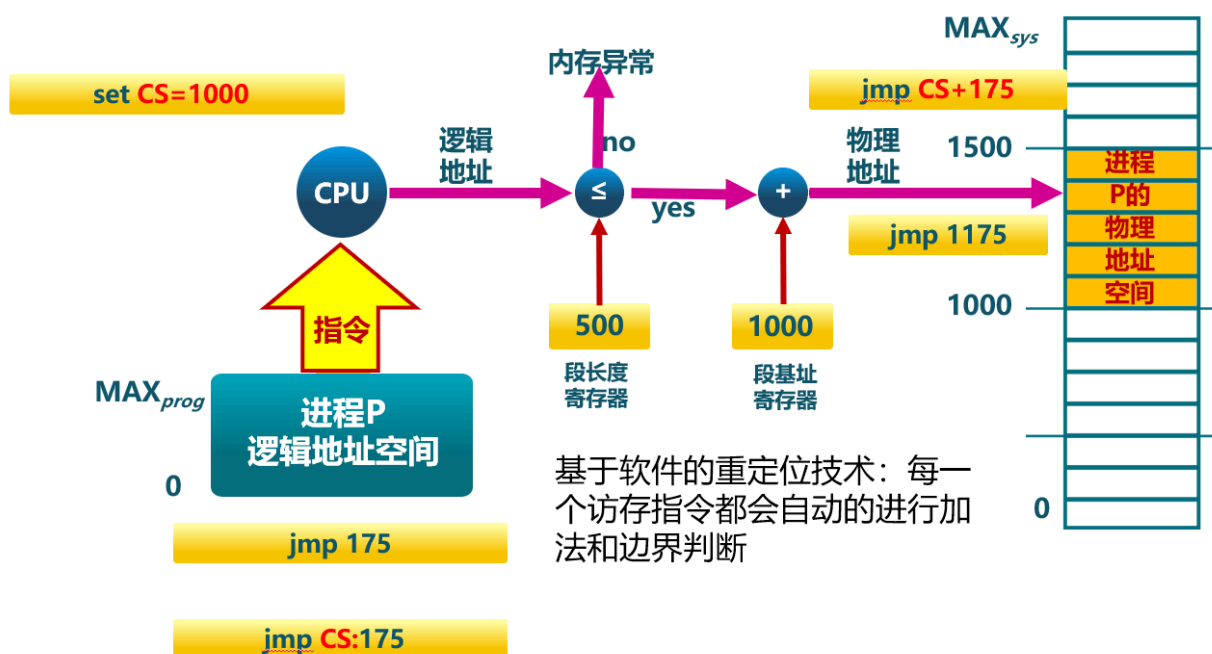


在多道程序中，我们需要解决内存地址的问题。



首先，我们将源程序编译，然后再汇编。得到了jmp 75的指令。然后进行库的连接，变成了175。然后程序进行加载运行，自动重定位到1175。

我们使用硬件进行辅助，将程序自动加载到对应的地址

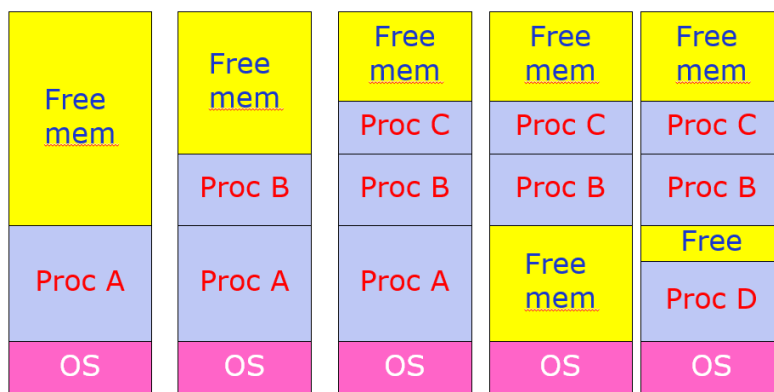


我们需要完成动态内存的分配工作

当程序被加载执行时，分配一个进程指定大小可变的分区(块、内存块)

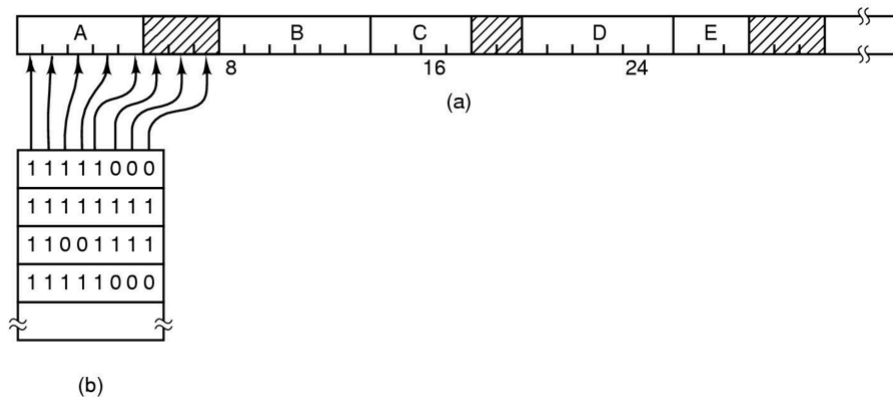


程序在分配完内存空间之后，可以进行内存的释放，如下所示



我们有很多管理内存的方式：——内存的管理方法和内存块的匹配方式，重要！

1.使用位图的存储管理



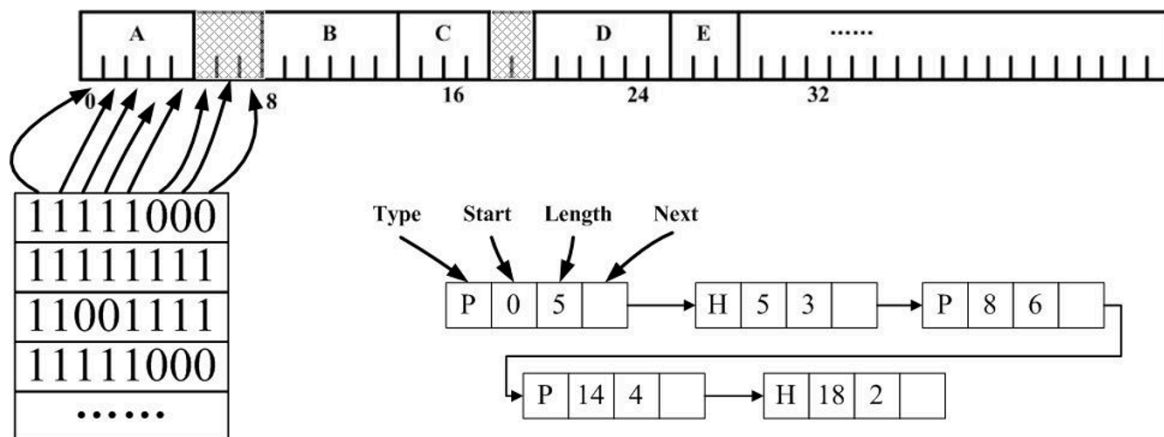
如图所示，比如说0-7的位置，共8个位置，我们将已经存储的位置写入1，将空闲的位置写入0

这样的话，1越多，说明内存占的比较多；如果0较多的话，说明内存比较空

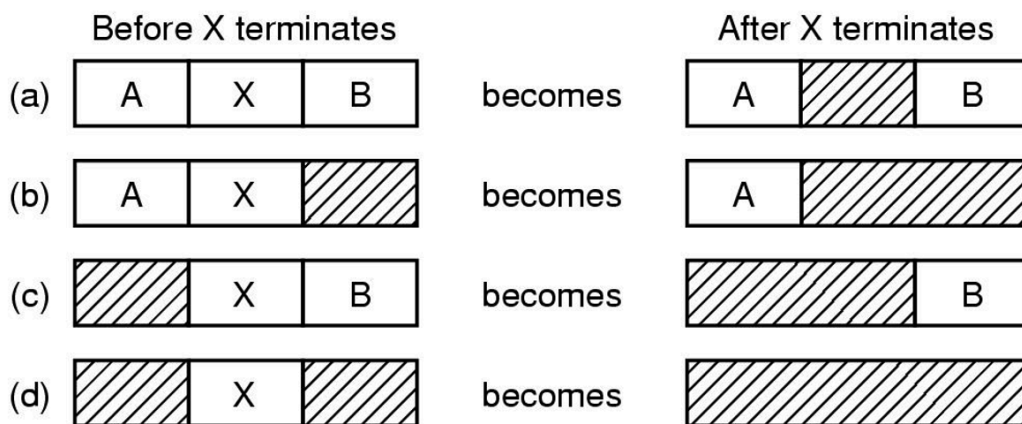
2.使用链表的存储管理

开头的H或者P表示该内存的剩余情况，P代表已占用，H表示未占用

start表示起始地址，Length表示长度，next表示下个链针



链表中的空闲内存的管理模式：

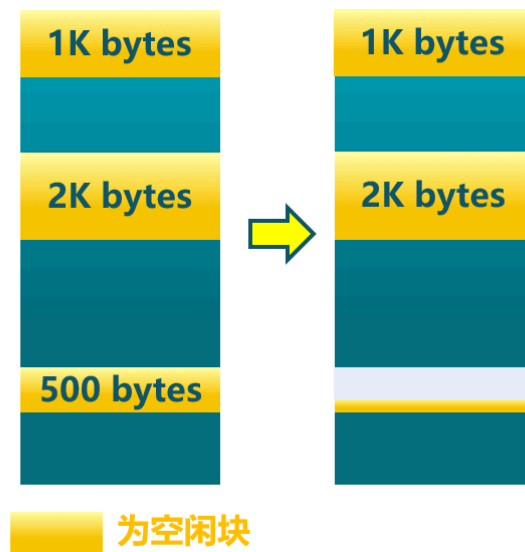


- 第一种就是将中间的内存释放
- 第二种就是后面跟着的内存释放
- 第三种就是将前面的内存释放
- 第四种就是将中间的内存释放，前后都是空闲的

内存管理匹配模式：——重要!!!

1.最佳匹配(best-fit)

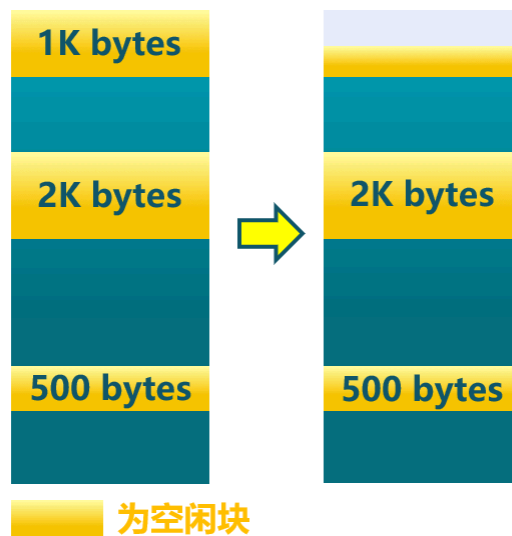
就是选取最接近所需要的内存块大小的内存块



比如这个情况，我们需要请求400bytes的内存块，我们就直接选取500bytes的内存块——哪一个内存大小离的近就选哪一个

2.最先匹配(first-fit)

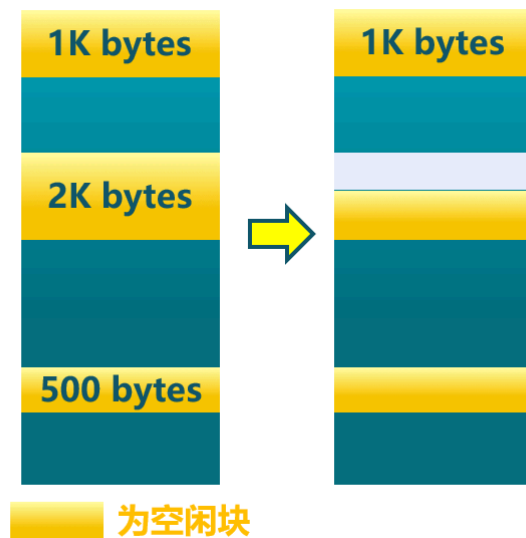
就是选取最前面的一个空闲的内存块



比如这个情况，我们就直接选取最前面的1000bytes的内存块——选择在系统内存中最靠前的

3.最差匹配(worst-fit)

就是选取差距最大的内存块，选取最大的



比如这个情况，我们就直接选取最大的哪个块，也就是2000bytes的内存块——选择内存大小差距最大的——这样可以减少内存中碎片的数量

4.下次匹配(next-fit)

按分区的先后次序，从上次分配的分区起查找（到最后分区时再回到开头），找到符合要求的第一个分区就分配——类似于first-fit，只不过我们开始搜索的位置变成了上一次进行分配的位置

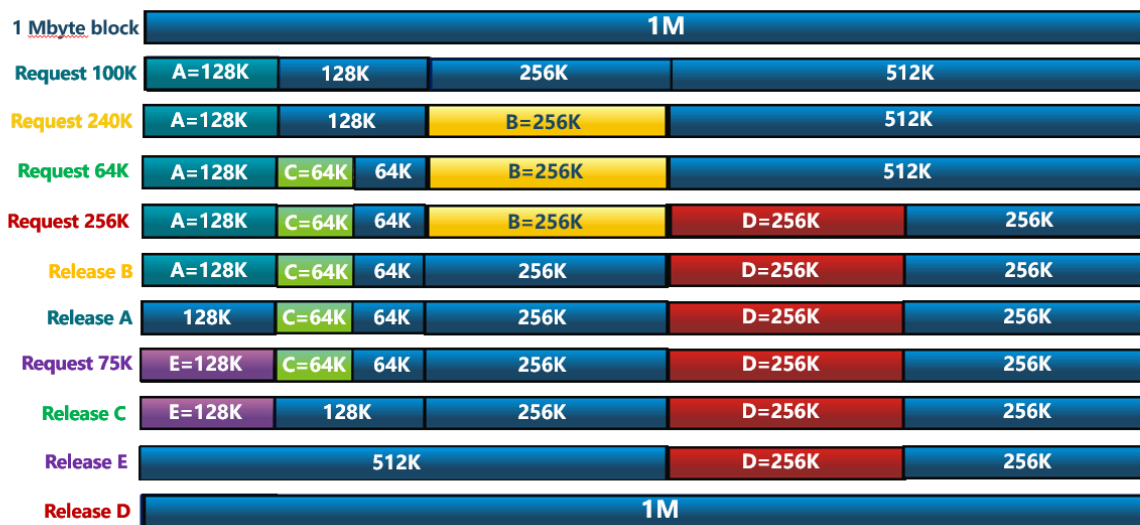
这个方法：使空闲分区分布得更均匀

伙伴系统**buddy system**

- 整个可分配的分区大小 2^U
- 需要的分区大小为 $2^{U-1} < s \leq 2^U$ 时，把整个块分配给该进程；
- 如 $s \leq 2^{i-1}$ ，将大小为 2^i 的当前空闲分区划分成两个大小为 2^{i-1} 的空闲分区
- 重复划分过程，直到 $2^{i-1} < s \leq 2^i$ ，并把一个空闲分区分配给该进程

刚开始的时候，只有一个大小为 2^U 的空闲块；由小到大在空闲块数组中找最小的可用空闲块；如空闲块过大，对可用空闲块进行二等分，直到得到合适的可用空闲块

举例：



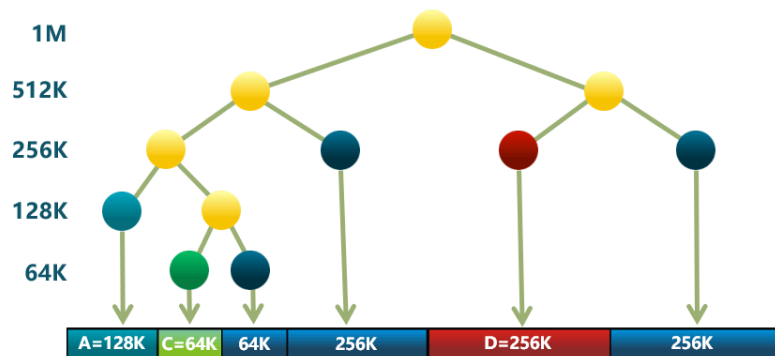
- 首先，初始化为1M的块
- 然后分配一个100K的内存块，我们把1M的切开来——一直切，切到128KB，然后就使用这一块
- 然后接下来请求240K的内存块，取走256K的内存块
- 以此类推，最后释放的时候如上所示——如果释放完之后，存在相邻的大小相等的块，我们就直接合并，迭代的进行合并

■ 释放过程

- ▣ 把释放的块放入空闲块数组
- ▣ 合并满足合并条件的空闲块

■ 合并条件

- ▣ 大小相同 2^i
- ▣ 地址相邻
- ▣ 低地址空闲块起始地址为 2^{i+1} 的倍数

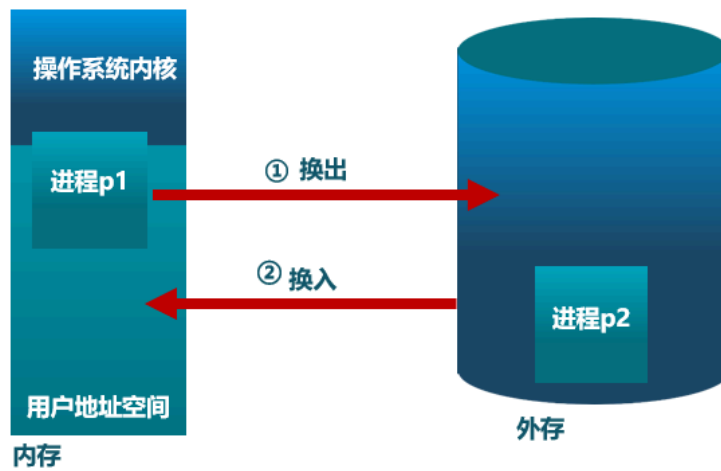


我们发现，首先释放64KB的块，然后与旁边的64KB进行合并，然后再迭代的进行合并即可。

内存不够用了——就需要对内存剩余的块进行内存合并的操作

我们可以通过碎片整理和碎片紧凑的方法来将一些小的内存块聚合成大的内存块

但是如果内存还是不够用的话，怎么办？——因为我们实际上的内存还是不足以提供给所有的进程进行使用



我们可以将那些**不使用的进程存储到外存**去，这样就可以省下对应的内存的位置去存储对应我们所需要的进程。

实际操作：我们可以通过抢占处于等待进程状态的分区，来增大我们的内存空间

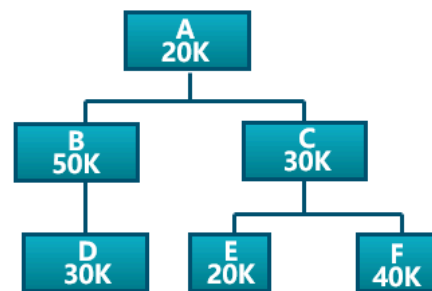
但是实际上我们的进程所需要的空间，进程不一定会完全使用

比如一些逻辑分支函数，使用了if分支部分的内存，就一定不会使用else部分分支的内存

```

A;//20K
if( m >0){
    B;//50K
    .....
    D;//30K
}else{
    C;//30K
    if( n <0){
        E;//20K
    }else{
        F;//40K
    }
}
    
```

程序调用结构：190K



我们可以使用内存覆盖的方法来节约我们的内存使用

内存覆盖技术：

- 划分对应的模块，确定对应的覆盖关系，在执行前进行预先的加载和交换
- 但是我们实际上难以完成程序模块的划分，难以预先知道程序执行的路径，所以这个方法不是很适用

虚拟存储——重要!!!

- 一个或多个进程的程序段、数据段、堆栈段总和可以大于物理存储空间
- 进程中的各段不必完全装入内存，就可启动进程运行
- 操作系统定时将暂时不用的信息换出内存
- 需要时操作系统再将交换区信息换入内存

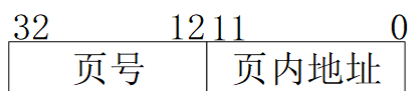
我们需要将我们的内存使用页来进行管理

分页管理

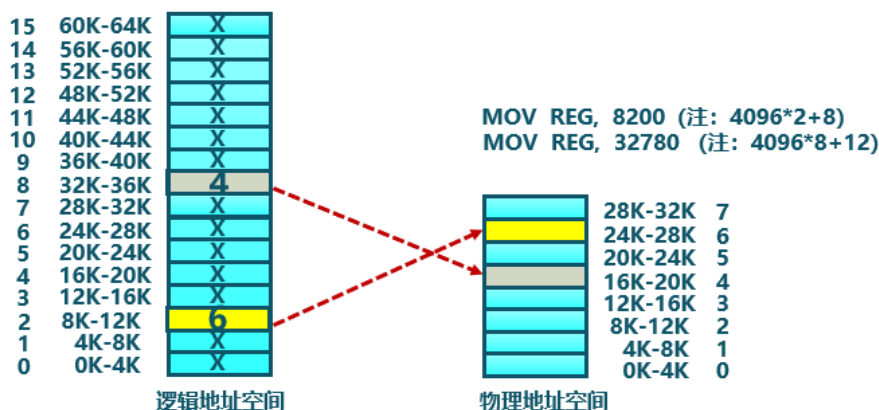
将程序的逻辑地址空间划分成**固定大小的页**(page)

分页后进程的逻辑地址由两部分构成：

- **页号**： $\text{address} / \text{page_size}$ ，因为一个页的大小是4096KB，所以我们的页内地址占到了12位，页号占到了剩下的20位
- **页内地址**： $\text{address} \% \text{page_size}$ ，取一个余数



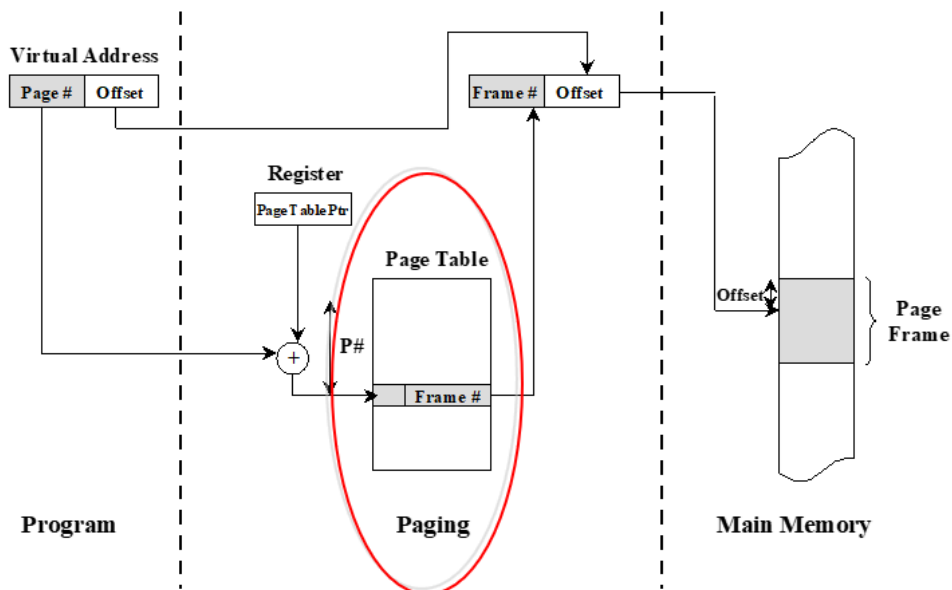
举个例子：



操作系统在开始的时候，不会去分配对应的内存地址。当读取当一条指令后，才会去为指令进行页的分配。

第一条指令读取到了8200地址，我们需要使用页表2的内容，所以操作系统将页号为2的页表分配给物理地址空间的6号页框

同理，32780也是一样，将8号页表分配到了4号页框

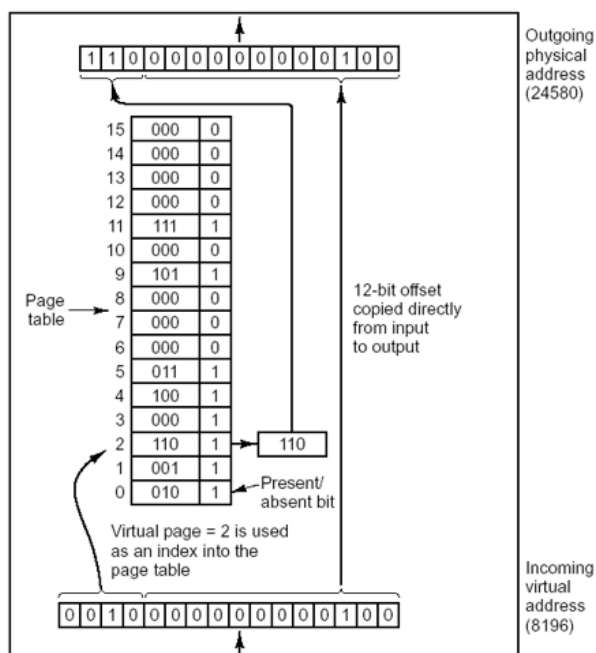


我们有一个**页表**来记录页号和页帧的对应关系

举个例子：

将虚址8196转换成实址24580的过程：

- 虚址8196在虚页2中；
- 该虚页被映射到实页面6上；
- 虚页号（前4位）作为页表的索引；
- “在不在”位决定是否产生缺页异常；
- 后12位作为页内偏移量。

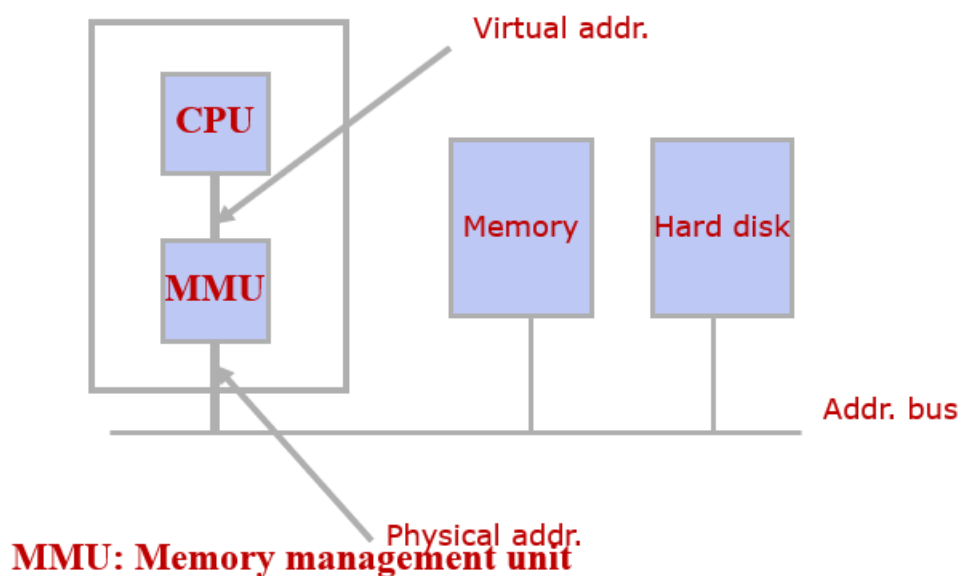


我们其实只需要对地址的高位进行改变就可以了，后面的页内偏移是一样的

我们找到page table中的2号页表，然后发现对应到了110，也就是页面6

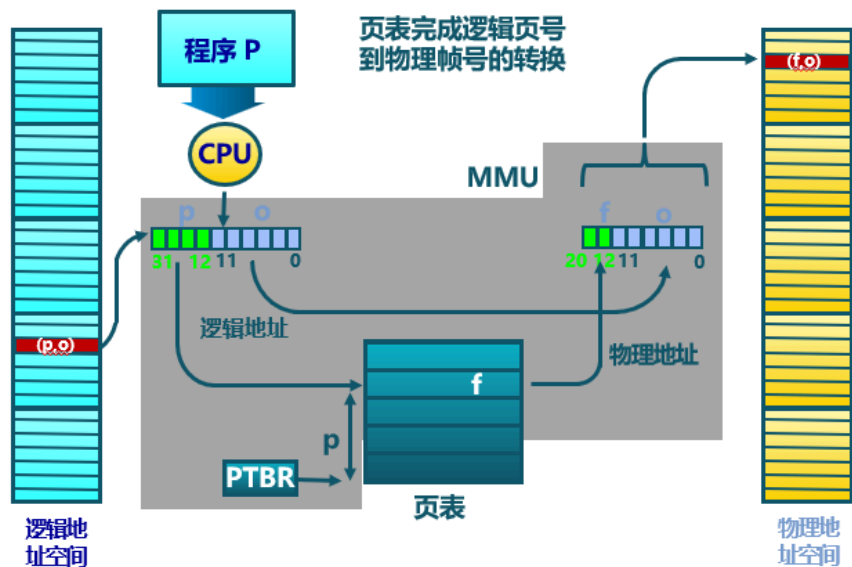
然后将110写入物理地址的前三位即可，这样就完成了对应的转换过程

我们可以使用硬件来完成这个过程——MMU



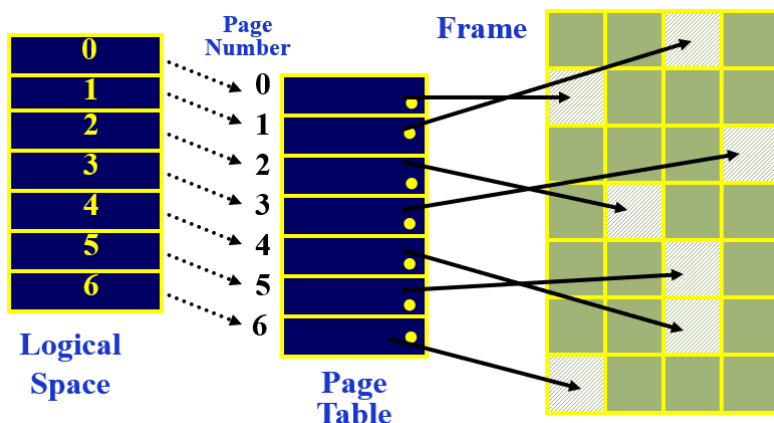
- MMU出现在CPU与内存之间
- MMU会检测每一个CPU到内存的访问，并发现程序在运行过程中使用了哪个块

地址转换的具体实现：



首先，在逻辑地址空间取出对应的页号，也就是前面的位数，将其在页表中进行查表，看看有没有对应的页帧号。如果有的话，相当于将其写入到前面的页帧号中，形成了物理地址，这样就得到了物理地址空间。

页表将不连续的物理地址变成了连续的逻辑地址



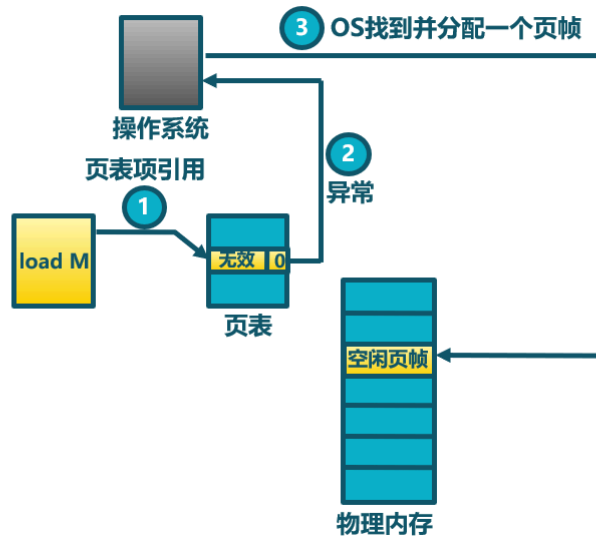
如果发生缺页异常怎么办？——就是我们读取到的页表项不是有效值——具体的处理过程：

1. CPU发出访存指令到MMU，MMU根据地址高位部分得到页号，以页号查页表

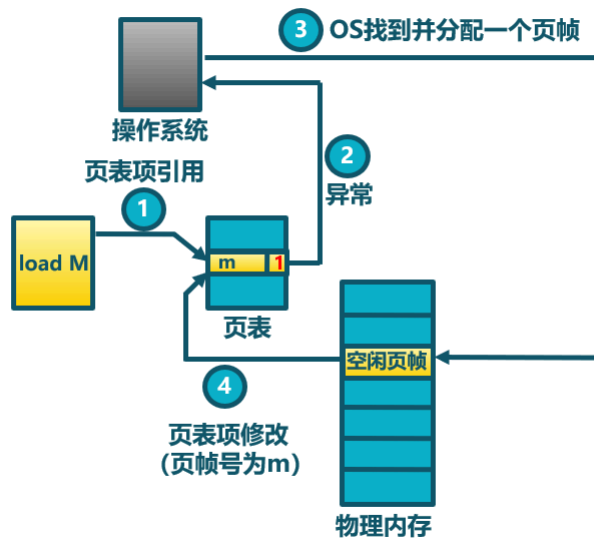


2. 查表发现该页表项未填入有效值，因此产生缺页异常

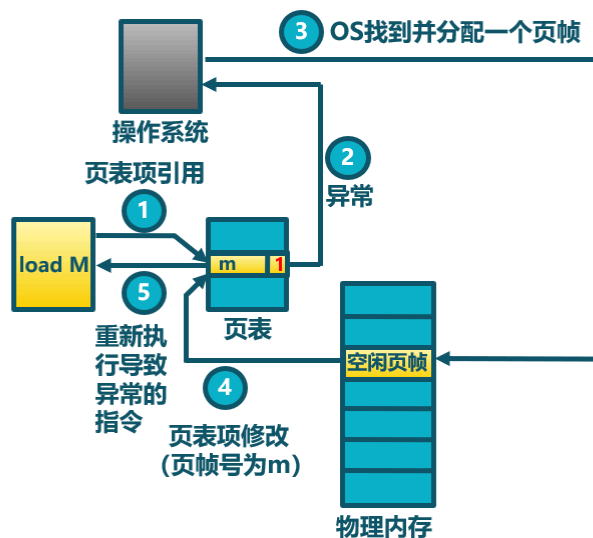
3. 缺页异常由OS响应，OS需要分配对应的页帧，并找到该页的内容在磁盘中的位置



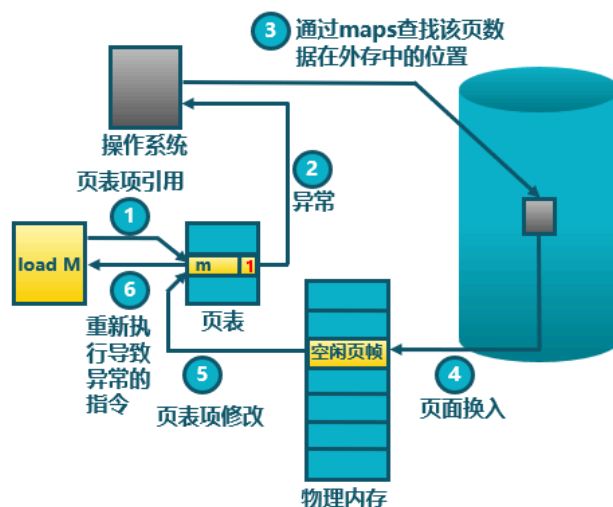
4.找到系统中的空闲页帧，并取得页帧号，将页帧号填入页表中，并修改页表的有效位（将页帧号m填入，将有效位的0改为1）



5.异常返回，重新执行产生缺页的指令，也就是说该指令如果缺页的话，就会重新执行一遍



tips: 第三步中, 实际上通过maps查找该页数据在外存中的位置



综上所述, 缺页异常的全过程:

- 编译阶段把虚拟地址空间划分成了若干区域
- 链接阶段将这些区域信息和相应的数据和指令存在文件中
- 加载阶段OS读取这些区域信息和存储的对应关系——在MMU中进行查找（并不加载真正的数据和指令）
- 程序访问某个区域, 触发缺页异常, 在异常的响应函数中, 完成对数据和指令的加载, 并出现在物理内存和页表中——修改对应的页帧号和有效位, 把当前分配的有效页面写入
- 缺页异常的响应函数返回至触发异常的指令, 该指令得以再次执行, 并且不再触发异常——如果出现了缺页异常, 必须再执行一遍! 也就是保证MMU的记录没有问题

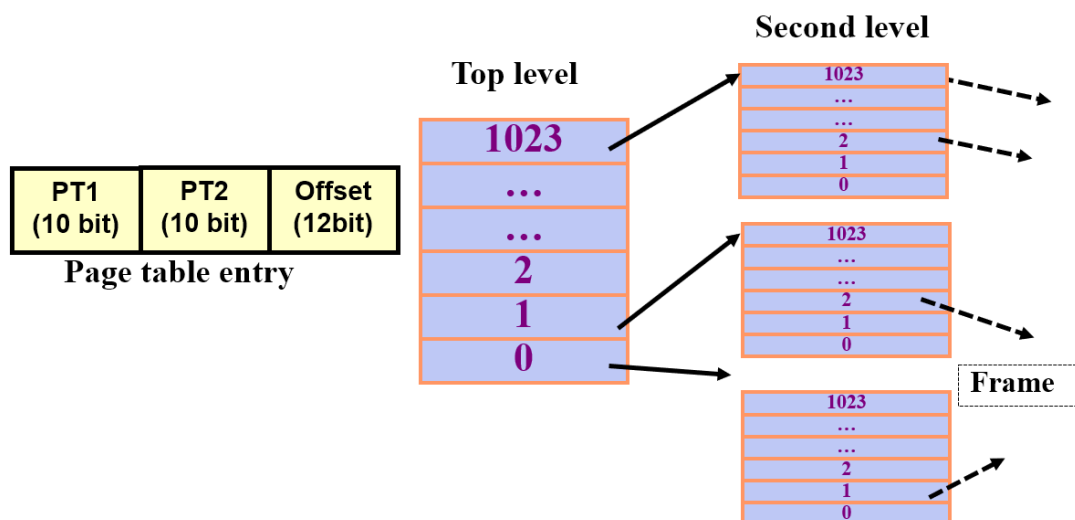
页表的设计——重要!!!

page应该设计成多大? ——应该设计成2的整数次幂

选择4KB, 是一个很好的选择

一个32位的地址空间, 我们需要100万项页表, 这样的话我们存储页表的空间就过于大了 (因为我们的页内偏移为12位, 所以需要 2^{20} 项页表)

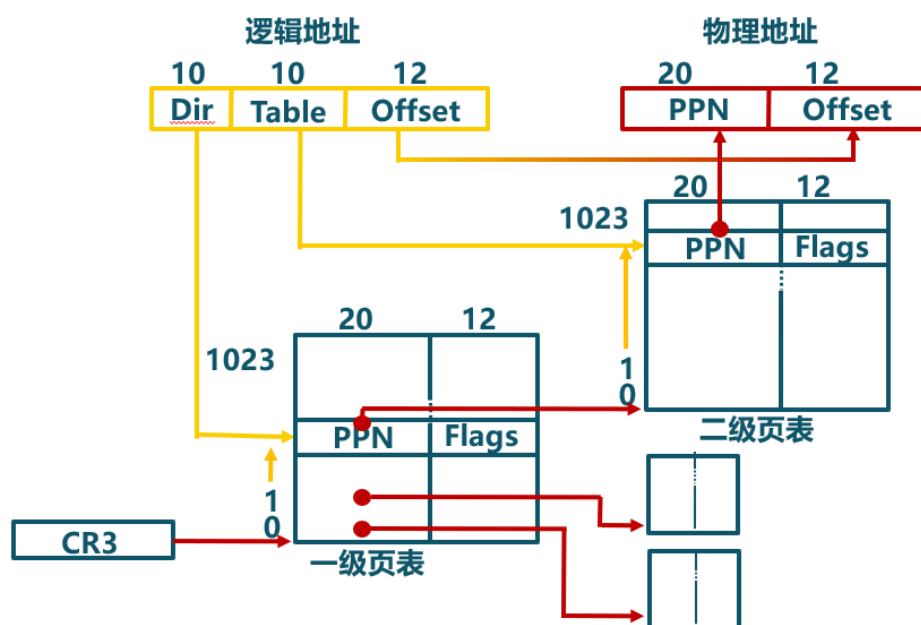
我们采用多级页表的方式进行存储, 两级页表的话, 相当于我只需要完成两个1024的页表存储即可



可以看到，PT1也就是第一级页表，PT2也就是第二级页表，后面的offset也就是页内的偏移

在多级页表进行寻址的时候，我们首先去查第一级的页表项，找到对应的二级页表的空间，然后去找对应的第二级页表的页表项，以此类推

x86的页表结构：逻辑地址分为了两级页表，和12位的页内偏移；物理地址还是20位的页表项和12位的页内偏移



页表大小的计算方法：

$$PageSize / PageTableEntrySize$$

可以得出一页（一个二级页表）中可以容纳的页表项数，从而得出所需的地址位数

所以，假设：

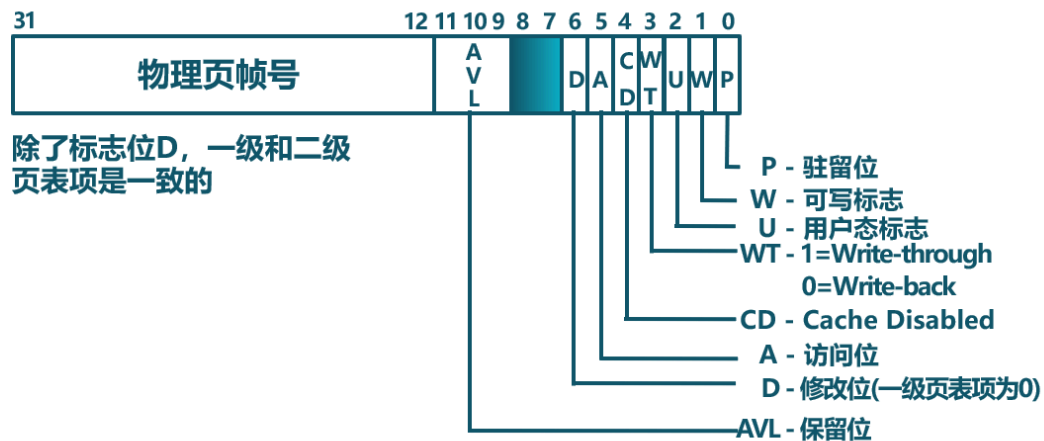
$$PageSize = 2^x \text{ bytes}$$

$$PageTableEntrySize = 2^2 \text{ bytes}$$

$$X(\text{页内地址位数}) + (X - 2)(\text{一级页表消耗的地址线}) + (X - 2)(\text{二级页表消耗的地址线}) = 32(32\text{位系统})$$

也就是说, X 的值为12, 刚好是4096KB的设计! (此处, 减去2是因为, 一个页表所占的空间是4096B, 也就是说占到了4KB, 所以我们要除上4, 在二进制的位数上减2)

x86对应的页内偏移的分配情况:

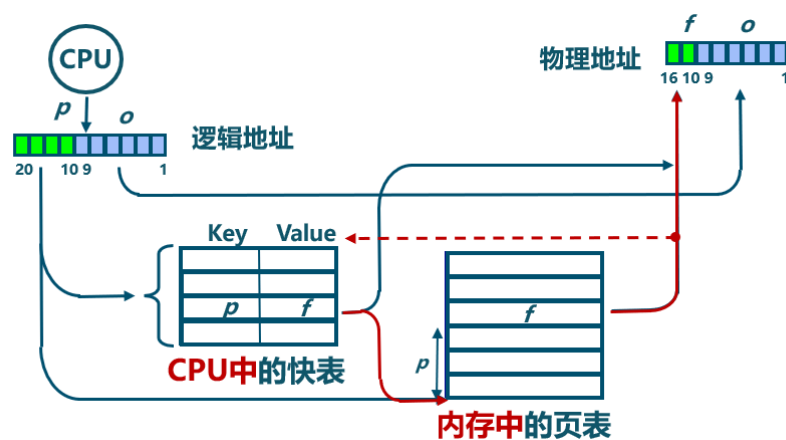


但是实际上，我们通过三级页表的访问，减慢了我们访问内存的深度，原本只需要访问一次的内存，现在我们需要访问三次的内存，相当于访问时间变长了

TLB快表:

缓存近期访问的页表项

- 如果TLB命中，物理页号可以很快被获取
- 如果TLB未命中，对应的表项被更新到TLB中——TLB快表的更新



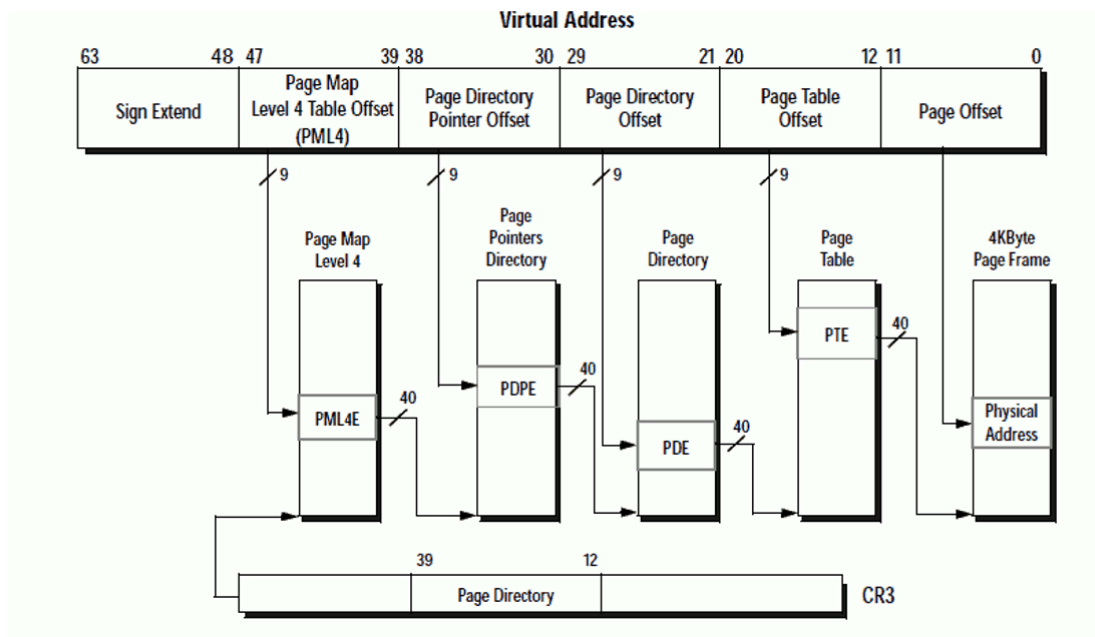
一个64位的地址空间，需要多少项页表？假设一个页表项8字节，页表有多大？

64位的地址空间，页的大小还是不变的，为4KB，也就是说，我们有 2^{12} 的页内偏移，就是说，我们需要512项

$$2^{12}/(64/8) = 512$$

反置页表：基于Hash映射值查找对应页表项中的帧号

x64的页表:

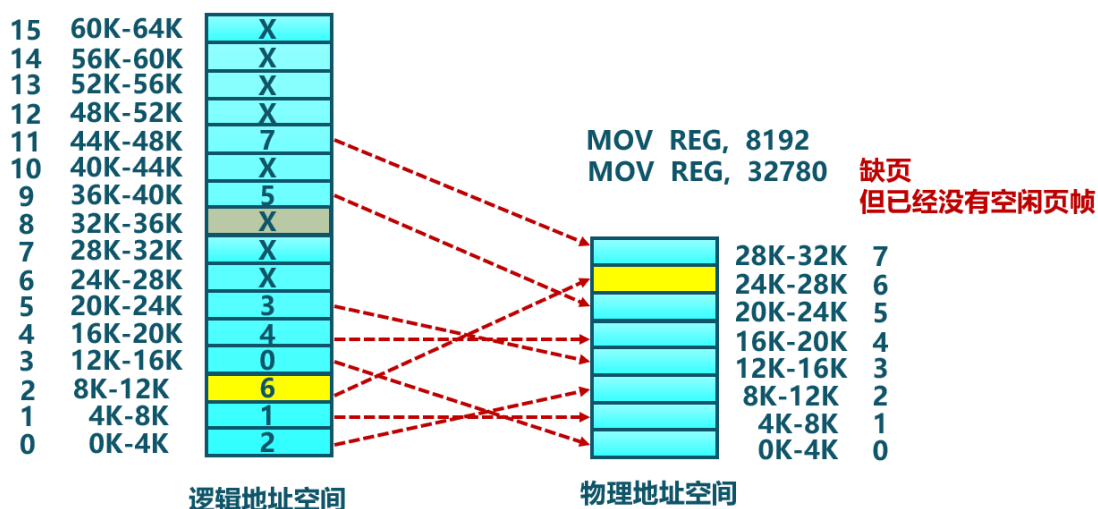


- 每个页表里面有512项
 - $4k/8\text{字节}(64bit) = 512$
 - 因此用于页表项的寻址部分仅有9位
- 类似的，页目录项也只有512项
 - 页目录项也只有9位
- 页表扩展为4级，每级都是9位地址
 - 寻址空间最大为 $256T$
 - 最高的16位目前没有使用
- 实际使用的内存远没有这么多

所以，我们使用了4级页表来进行索引，页内偏移为12位，然后是4个9位的页表项，还剩16位就当做扩展位

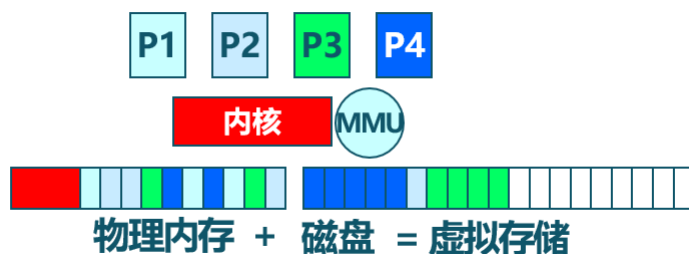
换页

如果我们将所有的物理地址空间都完成了分配，怎么办？



找到一个近期不会使用的页，将页帧写入硬盘中——把谁丢出去？对应的换页算法

虚拟存储——物理内存+磁盘



- 正在使用的数据放在物理内存中
- 从未使用的数据和临时不用的数据，放在硬盘中

借助MMU隔绝了程序对物理内存的直接访问

- 程序中使用的地址都位于虚拟地址空间中
- 经由MMU翻译后得到物理地址空间发往内存器件

有效存储访问时间 (effective memory access time EAT)

$$EAT = 10(1 - p) + 5000000p$$

所以如果需要降低访存时间的话，就需要尽可能的降低缺页率

页面置换算法

当出现缺页异常，需调入新页面而内存已满时，置换算法选择被置换的物理页面——尽可能减少页面的调入调出次数

1.最优页面置换算法(OPT, optimal)

置换在未来最长时间不访问的页面

缺页时，计算内存中每个逻辑页面的下一次访问时间，选择未来最长时间不访问的页面

时间	0	1	2	3	4	5	6	7	8	9	10
访问请求		c	a	d	b	e	b	a	b	c	d
物理帧号	0	a	a	a	a	a	a	a	a	a	a
	1	b	b	b	b	b	b	b	b	b	b
	2	c	c	c	c	c	c	c	c	c	c
	3	d	d	d	d	e	e	e	e	e	e
缺页状态						●					●
每页的下次访问时间						a=7 b=6 c=9 d=10				a=? b=? c=? d=?	

比如说上图，首先我们前四个请求只需要正常访问即可，第五个请求e，发现没有位置了，所以我们需要去找到后面的页面请求的最远的一个，也就是d，把d换出去，把e换进来，以此类推即可

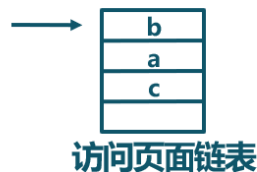
但是这个算法，我们无法预知每个页面在下次访问前的等待时间

所以我们只能，在模拟器上运行某个程序，并记录每一次的页面访问情况，然后第二遍运行时使用最优算法

2.先进先出算法 (First-In First-Out, FIFO)

在4个页帧中执行：

■ 假定初始a- > b- > c- > d顺序



时间	0	1	2	3	4	5	6	7	8	9	10
访问请求		c	a	d	b	e	b	a	b	c	d
物理帧号	0	a	a	a	a	e	e	e	e	e	d
	1	b	b	b	b	b	b	a	a	a	a
	2	c	c	c	c	e	c	c	b	b	b
	3	d	d	d	d	d	d	d	d	c	c
缺页状态						●		●	●	●	●

先进来的先被换出去

比如上图，前四个都没有问题，把abcd都存进去，然后e存进来，把最先进来的a换出去，然后是b，正常访问，然后是a，将最先进来的b换出去，然后是访问b，把c换出去，访问c，把d换出去，然后访问d，就轮到换出e了

我们需要维护一个访问页面的链表

- 该算法选择在内存驻留时间最长的页面进行置换
- 链表元素按驻留内存的时间排序，链首最长，链尾最短
- 出现缺页时，选择链首页面进行置换，新页面加到链尾

这个算法实现比较简单，但是性能一般来说很差，调出的页面可能是经常需要访问的，而且会产生Belady现象——进程分配物理页面数增加时，缺页并不一定减少

FIFO算法的异常现象——Belady现象

当页面大小为3的时候：

访问顺序：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

物理页面数: 3 缺页次数: 9

FIFO	1	2	3	4	1	2	5	1	2	3	4	5
页	1	2	3	4	1	2	5	5	5	3	4	4
头		1	2	3	4	1	2	2	2	5	3	3
头			1	2	3	4	1	1	1	2	5	5
缺页状态	●	●	●	●	●	●	●			●	●	

当页面大小为4的时候：

访问顺序：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

物理页面数: 4 缺页次数: 10

FIFO	1	2	3	4	1	2	5	1	2	3	4	5
页	1	2	3	4	4	4	5	1	2	3	4	5
头		1	2	3	3	3	4	5	1	2	3	4
头			1	2	2	2	3	4	5	1	2	3
头				1	1	1	2	3	4	5	1	2
缺页状态	●	●	●	●			●	●	●	●	●	●

我们发现，我们增加了物理页面的数量，但是并没有减少缺页的次数，反而增加了缺页的次数！

原因：

- FIFO算法的置换特征与进程访问内存的动态特征无法匹配
- 被它置换出去的页面并不一定是进程近期不会访问的

3.最近最久未使用算法 (Least Recently Used, LRU)

选择最长时间没有被引用的页面进行置换

- 缺页时，计算内存中每个逻辑页面上一次访问时间
- 选择上一次使用到当前时间最长的页面

使用一个栈就可以实现对应的功能

保持一个最近使用页面的“栈”

时间	0	1	2	3	4	5	6	7	8	9	10
访问请求		c	a	d	b	e	b	a	b	c	d
物理帧号	0	a	a	a	a	a	a	a	a	a	a
	1	b	b	b	b	b	b	b	b	b	b
	2	c	c	c	c	e	e	e	e	e	b
	3	d	d	d	d	d	d	d	d	c	c
缺页状态											

访问页面栈	c	a	d	b	e	e	b	a	c	d
被替换页面					c				d	e

如上图所示，我们维护一个访问页面的栈，刚开始，将cadb分别压入我们的栈中，然后遇到了c的访问请求。我们将c压入栈，所以在栈底的c就出去了，被替换掉了。同样的道理，如果我们遇到了在栈中的物理帧号，那么我们不作置换，但是我们会把我们访问的那个帧号提取到栈的最上面。

也就是：

- 访问内存时，找到相应页面，并把它移到链表之首
- 缺页时，置换链表尾节点的页面

tips：对于一次访存而言，LRU不会出现belady是因为，增加页框数量后原本不引起缺页事件的访存必然不会引起缺页

- 因为留在页框中的内容是按最近访问的顺序排列的，增加页框，得到的留在内存中的页面，一定是增加之前页面集合的超集(也就是说，如果我们原来有3个页，那么3个页中存储的一定是最近访问的三个页；那么我们将页数增加到4的时候，我们4个页中存储的一定是最近访问的四个页，这四个页一定包含了前面的三个页，所以不会出现belady现象)
- 新增的页框最差情况下就是不发挥作用，但是不会引出新的缺页事件，因此不会增加

4.最不常用算法 (Least Frequently Used, LFU)

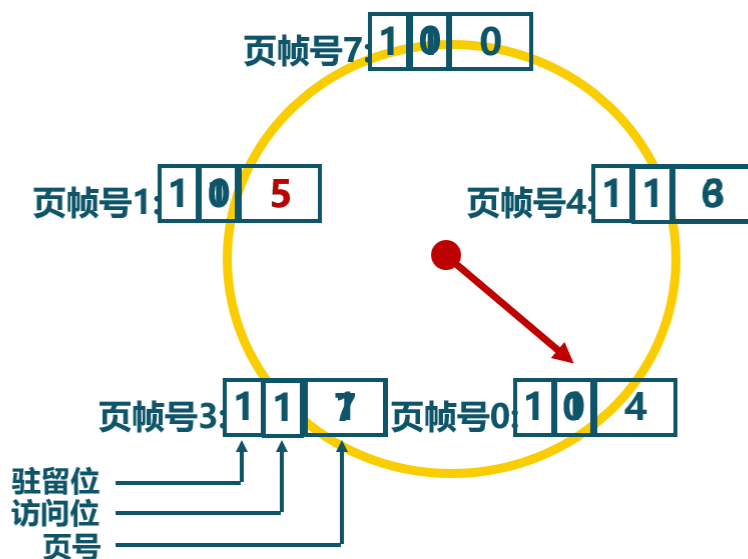
记录过去一段时间中，物理页面被访问的次数，根据访问的次数大小来进行页面的替换

将CPU中的MMU中进行访问次数的记录

时间	0	1	2	3	4	5	6	7	8	9	10
访问请求		c ⁷	a ¹	d ¹⁴	b ⁵	e ¹⁸	b ¹	a ¹⁹	b ²⁰	c ²⁰	d ¹⁷
物理帧号	0	a ⁸	a ⁹	a ⁹	a ⁹	e ¹⁸	e ¹⁸	e ¹⁸	e ¹⁸	e ¹⁸	d ¹⁷
	1	b ⁵	b ⁵	b ⁵	b ¹⁰	b ¹⁰	b ¹¹	a ¹⁹	a ¹⁹	a ¹⁹	a ¹⁹
	2	c ⁶	c ¹³	c ¹³	c ¹³	c ¹³	c ¹³	c ¹³	b ²⁰	b ²⁰	b ²⁰
	3	d ²	d ²	d ¹⁶	d ¹⁶	d ¹⁶	d ¹⁶	d ¹⁶	d ¹⁶	c ²⁰	c ²⁰
缺页状态											

比如说，这个例子，刚开始abcd被访问的次数分别为8,5,6,2。然后，访问7次c，就把c的访问次数加上7。最后需要进行置换的时候，直接将当前访问次数最少的进行置换掉就可以了

5. 时钟置换算法 (Clock) —— 只有一个bit的原始算法



原理很简单，增加对应的访问位，也就是中间的那一位。该页面形成一个环形的链表，页面初始化时，**将这些页面的访问位都设置为0**

当该页面被访问的时候，就将这个页面的访问位设置为1

当发生缺页异常的时候，我们有一个指针去循环遍历这个链表，**每当遇到一个1，就将其清零**；遇到的第一个0，**我们就将其进行置换**。置换完之后，将这个位置的**访问位写为1**，对应的页号也要更新

举例：

时间	0	1	2	3	4	5	6	7	8	9	10
访问请求		c	a	d	b	e	b	a	b	c	d
物理帧号	0	a	a	a	a	e	e	e	e	e	d
1	b	b	b	b	b	b	b	b	b	b	b
2	c	c	c	c	c	c	c	a	a	a	a
3	d	d	d	d	d	d	d	d	d	c	c
缺页状态						●		●		●	●
驻留页面的页表项	0 a 0 b 0 c 0 d	0 a 0 b 1 c 0 d	1 a 0 b 1 c 0 d	1 a 0 b 1 c 1 d	1 a 1 b 1 c 1 d	1 e 0 b 0 c 0 d	1 e 1 b 0 c 0 d	1 e 0 b 1 a 0 d	1 e 1 b 1 a 0 d	1 e 1 b 1 a 1 c	1 d 0 b 0 a 0 c

上面这个例子，访问abcd，然后把abcd的访问位都从0写为1。然后访问e，我们扫一圈，发现都是1，全部清零，然后到第一个a，将其置换即可。置换完之后，将e的位置写入1，表示置换完之后，就去访问它

如果所有的页表都是1的话，发生缺页异常，我们就直接扫一圈，把所有的1都扫成0，然后回到了最初的位置，将最初的那个页表进行置换即可。

clock这个方法是LRU和FIFO方法的折中方法——基本上是工业界的最优解

6. 改进的Clock算法(硬件增加一个bit之后的优化)

增加一个修改位，现在变成了**修改位和访问位**——通过两个bit来表示我们的页表的状态

对应的位结构：



换出的优先级：

1. 既没有被修改，又没有被访问的页（访问位0，修改位0）
2. 没有访问，但是被修改了（访问位0，修改位1）
3. 访问了，但是没有修改（访问位1，修改位0）
4. 访问了，也修改了（访问位1，修改位1）

时间	0	1	2	3	4	5	6	7	8	9	10
访问请求		c	a ^w	d	b ^w	e	b	a ^w	b	c	d
物理帧号	0	a	a	a	a	a	a	a	a	a	a
	1	b	b	b	b	b	b	b	b	b	d
	2	c	c	c	c	e	e	e	e	e	e
	3	d	d	d	d	d	d	d	d	c	c
缺页状态											

驻留页面的 页表项	00	a	00	a	11	a	11	a	11	a	00	a	00	a	11	a	11	a	11	a	00	a [*]
	00	b	00	b	00	b	00	b	11	b	00	b	10	b	10	b	10	b	10	b	10	d
	00	c	10	c	10	c	10	c	10	c	10	e	10	e	10	e	10	e	10	e	10	e
	00	d	00	d	00	d	10	d	10	d	00	d	00	d	00	d	00	d	10	c	00	c

如上图所示，首先访问c，但是没有写，所以我们将c的使用位修改为1；然后写a，所以我们将a变成11，后面是一样的，看第一次缺页的情况，请求e，首先扫描一圈，a变成01，b变成01，c变成00，d变成00，再扫一圈，a和b都变成00，遇到00的c，直接替换掉，变成10，现在指针到了d的位置...以此类推！（在替换完之后，指针还会往后走一位，因为替换的那一位下一次不可能选中它了）

总结：

- 只访问不修改的：将访问位置为1，修改位不变
- 访问+修改的：将访问位和修改位都置为1
- 缺页时：
 - 遇到11——变成01
 - 遇到10——变成00
 - 遇到01——变成00
 - 遇到00——直接替换

7.工作集置换算法

一个进程当前正在使用的逻辑页面集合，可表示为二元函数 $W(t, \Delta)$

- t 是当前的执行时刻
- Δ 称为工作集窗口 (*working-set window*)，即一个定长的页面访问时间窗口
- $W(t, \Delta)$ 是指在当前时刻 t 前的 Δ 时间窗口中的所有访问页面所组成的集合
- $|W(t, \Delta)|$ 指工作集的大小，即页面数目

也就是说，比如我们的工作集窗口是5，我们的时间为10秒，那么就是我们在10秒内，访问长度为5的所访问到的页面的总数

当内存访问的局部性区域的位置大致稳定时，工作集大小也大致稳定

常驻集：就是我们在当前时刻，进程实际驻留在内存当中的页面集合(也就是进程被分配的页框数量)

- 常驻集 $\supseteq$ 工作集时，缺页较少
- 工作集发生剧烈变动（过渡）时，缺页较多
- 进程常驻集大小达到一定数目后，缺页率也不会明显下降

算法思路——就是每加载一次页面，就选择那些超出工作集的页面进行释放

工作集的窗口大小就是我们规定的，如果窗口的大小为 n 的话，就是说我们的前 n 个页面的集合就是我们的工作集

$\tau = 4$

时间	0	1	2	3	4	5	6	7	8	9	10
访问页面		c	c	d	b	c	e	c	e	a	d
逻辑页面a	● $t=0$	●	●	●						●	●
逻辑页面b					●	●	●	●			
逻辑页面c		●	●	●	●	●	●	●	●	●	●
逻辑页面d					●	●					●
逻辑页面e							●	●	●	●	●
缺页状态		●			●		●			●	●

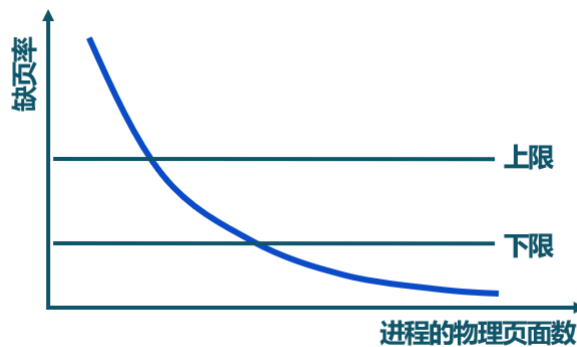
主要思路：不断查看内存，找到过去的 τ 个时间单位中没被访问的页面，OS主动换出这些页面

也就是说，我们每一次访问页面时，都需要将那些四个时间内没有访问到的页面直接在目前的状态页面中进行删除

8.缺页率置换算法（PFF, Page-Fault-Frequency）

我们根据系统每次缺页的时候，相隔的距离，来判断是不是需要释放一些页面

缺页率=缺页次数 / 内存访问次数



通过调节常驻集大小，使每个进程的缺页率保持在一个合理的范围内

- 若进程缺页率过高，则**增加常驻集**以分配更多的物理页面
- 若进程缺页率过低，则**减少常驻集**以减少它的物理页面数

算法实现：

- 访存时，设置引用位标志
- **缺页时，计算从上次缺页时间 t_{last} 到现在 $t_{current}$ 的时间间隔**
 - 如果 $t_{current} - t_{last} > T$ ，则置换所有在 $[t_{last}, t_{current}]$ 时间内没有被引用的页
 - 如果 $t_{current} - t_{last} \leq T$ ，则增加缺失页到工作集中

■ 假定窗口大小为 2

时间	0	1	2	3	4	5	6	7	8	9	10
访问页面		c	c	d	b	c	e	c	e	a	d
逻辑页面状态											
页面a	●	●	●							●	●
页面b					●	●	●	●	●		
页面c		●	●	●	●	●	●	●	●	●	●
页面d	●	●	●	●	●	●	●	●	●		●
页面e	●	●	●	●			●	●	●	●	●
缺页状态		●			●		●			●	●
$t_{cur} - t_{last}$		1			3		2			3	1

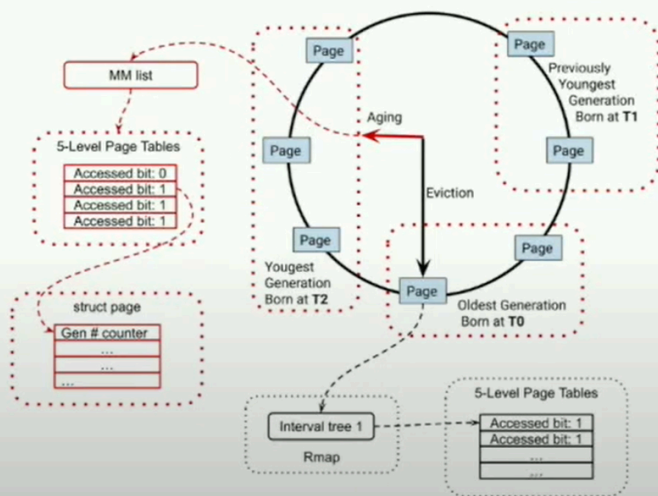
1. 仅在缺页事件发生时处理系统内存
2. 缺页过于频繁（间隔小于等于2）时增加内存
3. 缺页如果不频繁（间隔大于2），则回收不用的内存

如上图所示，首先我们第一个缺页的时候，加上了一个页面c，然后到第四轮访问的时候，隔了3个周期才缺页，所以我们减少我们的工作集，以此类推

补充：linux内核中的换页机制

MGLRU internals

- Generations
- Page table walks
- Feedback loops



我们把page分为很多种类，比如youngest和oldest的page，我们根据每一轮的时钟读取，如果一轮中没有读到这个页面，我们就把这个页面的等级做一个改变。每当出现了缺页的情况时，我们就把那堆oldest的页表存储到硬盘中即可

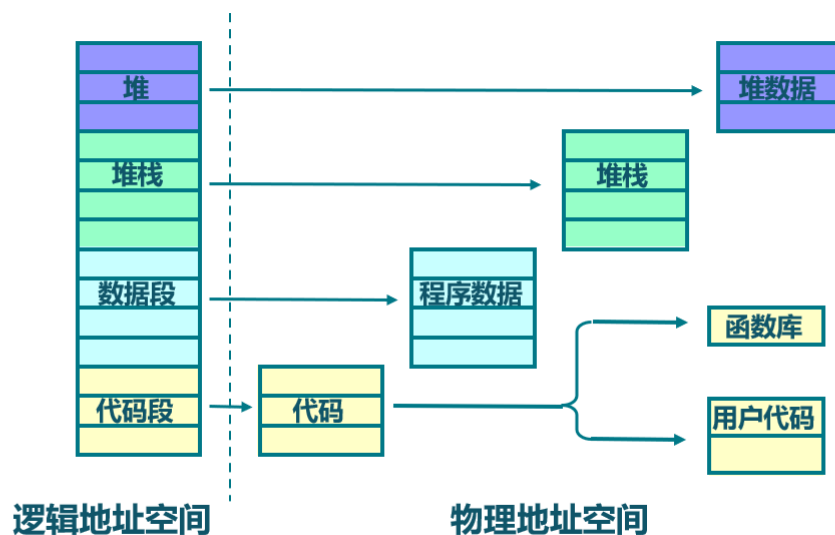
每次都是将最老的页进行换出，表示最久没有被使用

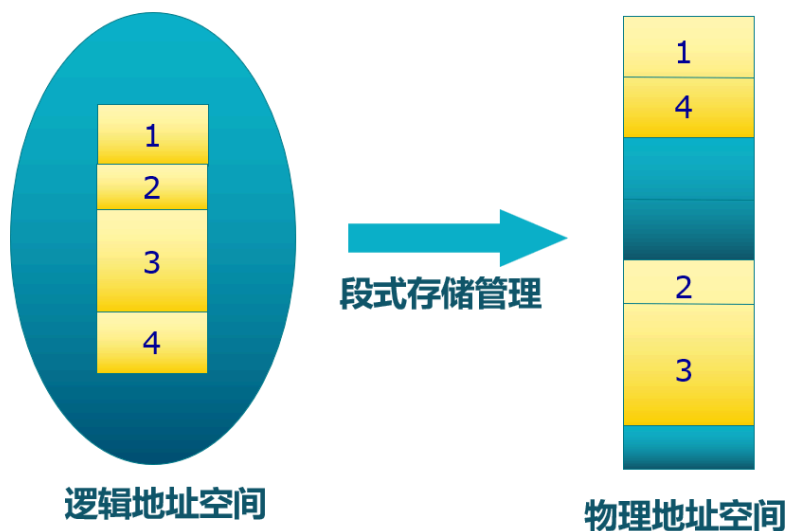
段机制应该是不考

虚拟页式存储中的外存管理

虚拟页式存储中的优先换出的内容——代码段和常量段：可执行二进制文件；动态加载的共享库程序段：动态调用的库文件；其它段：交换空间

段式地址空间的不连续二维结构





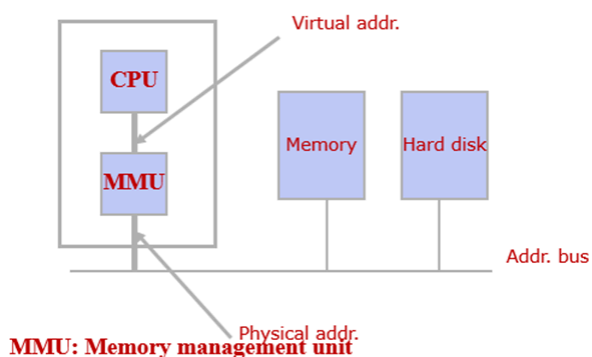
段和页的比较：

Consideration	Paging	Segmentation
Need the programmer be aware that this technique is being used?	No	Yes
How many linear address spaces are there?	1	Many
Can procedure and data be distinguished and separately protected	No	Yes
Can tables whose size fluctuates be accommodated easily	No	Yes
Is sharing of procedures between user facilitated	No	Yes

内存管理的补充内容：

CPU运行的地址空间是虚拟还是物理的？——**虚拟地址**

通过MMU进行传输，物理地址直接转化为了物理地址，所以我们运行的地址实际上是虚拟地址



- 虚拟空间的转换由MMU查表自动完成
- OS不用页表也知道虚拟地址与物理地址的对应关系——利用MMU查看

RISCV64中的页表组织

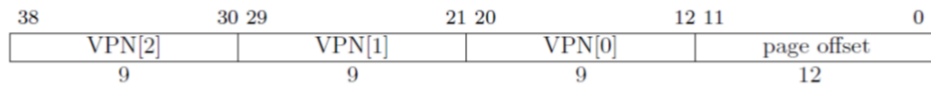


Figure 4.19: Sv39 virtual address.

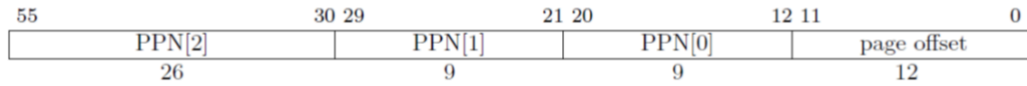
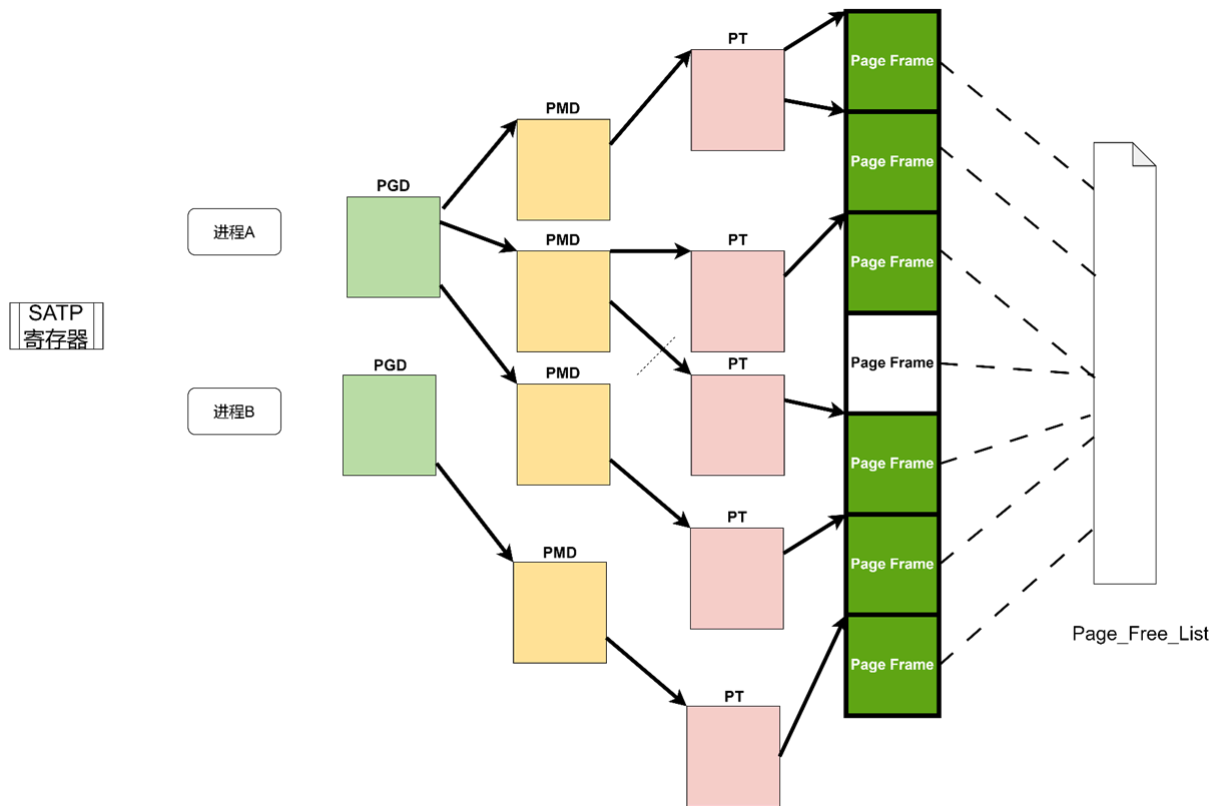


Figure 4.20: Sv39 physical address.



我们存储页表，将空闲的页表帧都存储到我们的Page_Free_List中

fork和exec的调用：

```

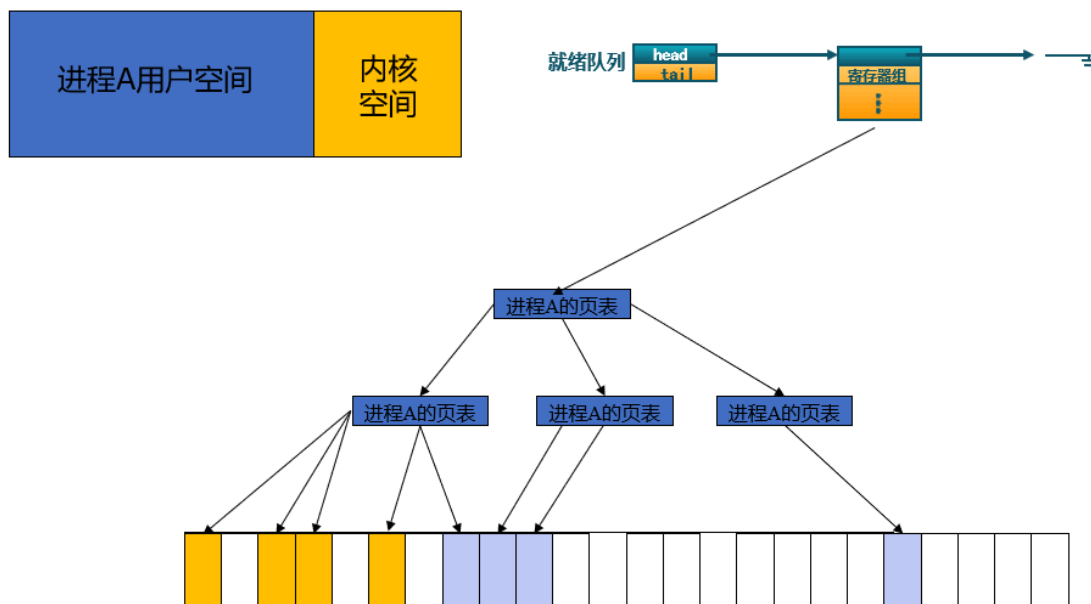
2#include <stdio.h>
3#include <stdlib.h>
4
5char command[256];
6void main()
7{
8    int rtn; /*子进程的返回数值*/
9    while(1) {
10        /* 从终端读取要执行的命令 */
11        printf( ">" );
12        fgets( command, 256, stdin );
13        command[strlen(command)-1] = 0;
14        if ( fork() == 0 ) { /* 子进程执行此命令 */
15            execlp( command, NULL );
16            /* 如果exec函数返回，表明没有正常执行命令，打印错误信息*/
17            perror( command );
18            exit( errno );
19        }
20        else { /* 父进程， 等待子进程结束，并打印子进程的返回值 */
21            wait ( &rtn );
22            printf( " child process return %d\n", rtn );
23        }
24    }
25}

```

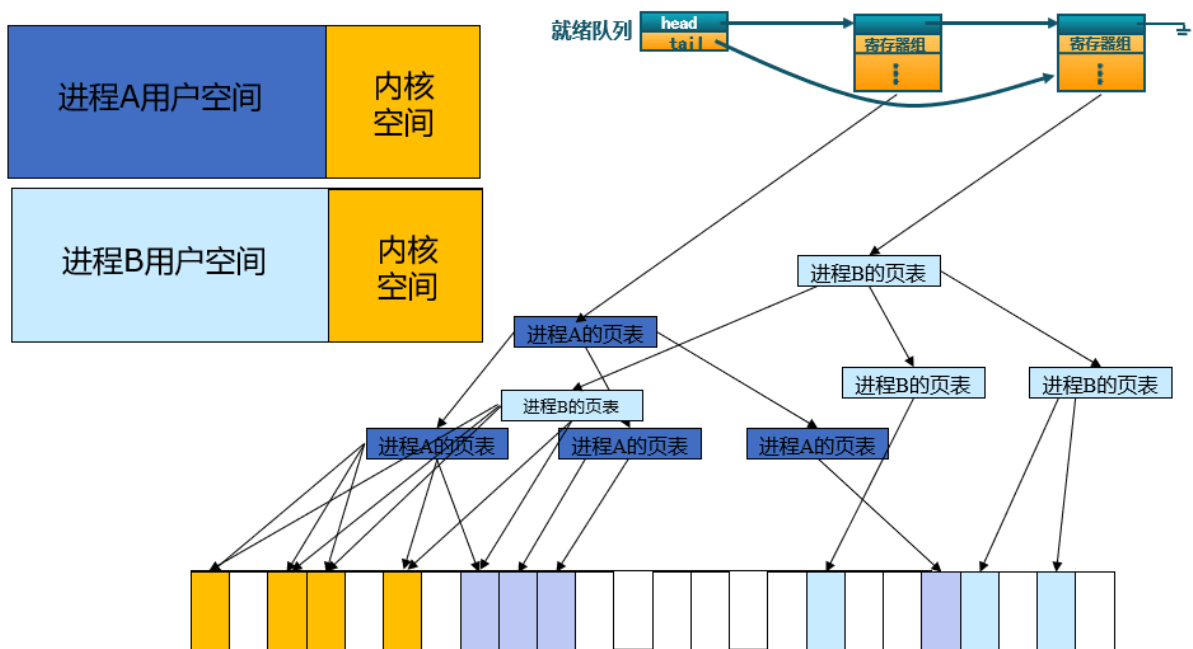
对应这个程序，我们

Fork由父进程调用，创建一个新的进程为子进程

fork调用前：



fork调用后：

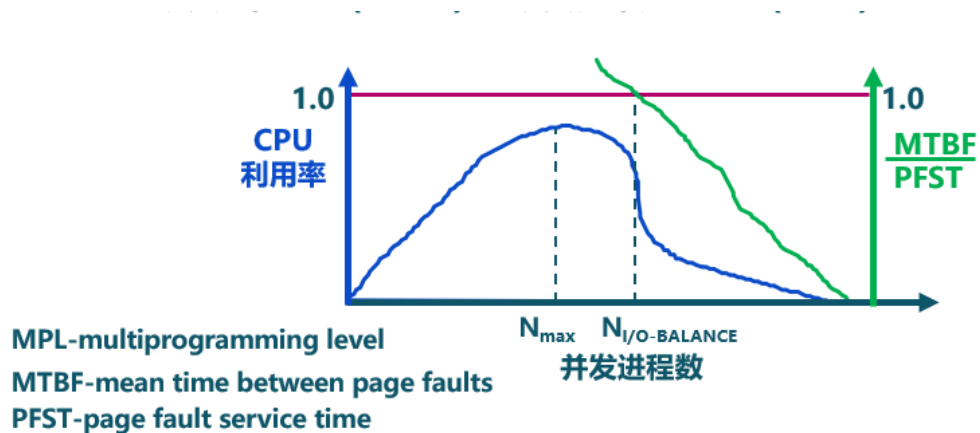


进程需要内存的时候：

- 进程发生缺页，OS需要找到可用页框
- 查找系统的free_list，试图找到空闲页框
- 如果没有找到
 - 按照策略，选取某些进程，令其释放内存
 - 针对被选中的进程使用页面替换算法（访问其页表）
 - 得到的空闲内存交还给全局页表
- 如果还没有找到
 - 按照策略，选取某些进程，直接杀死
 - 进程销毁，回收其所有资源
- 如果还没有找到
 - OOM panic (Out Of Memory)系统宕机

回收进程占据的资源？——主动回收：free, fclose

抖动问题：



就是当我们分配的进程数太多了，所以我们的页表就不够用了，导致了很多的缺页，导致CPU利用率急剧下降

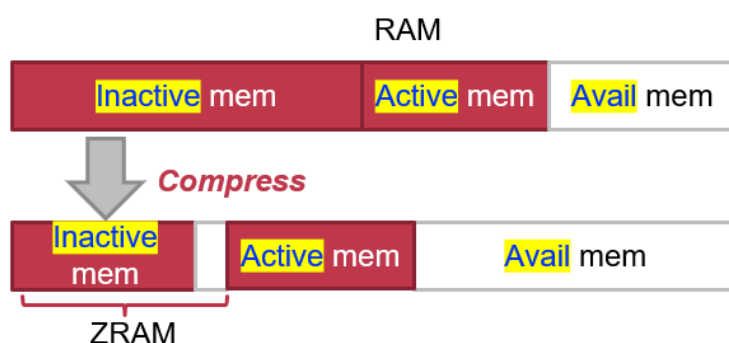
抖动的原因：随着驻留内存的进程数目增加，分配给每个进程的物理页面数不断减小，缺页率不断上升

- 进程物理页面太少，不能包含工作集
- 造成大量缺页，频繁置换
- 进程运行速度变慢

补充：安卓中的内存管理

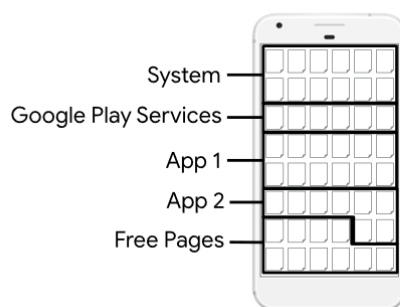
• ZRAM: Memory Compression

- RAM-based block devices named /dev/zram<id> (<id> = 0, 1, ...)
- Pages written to these disks are compressed and stored in memory itself
- Fast I/O and compression provides good amounts of **memory savings**

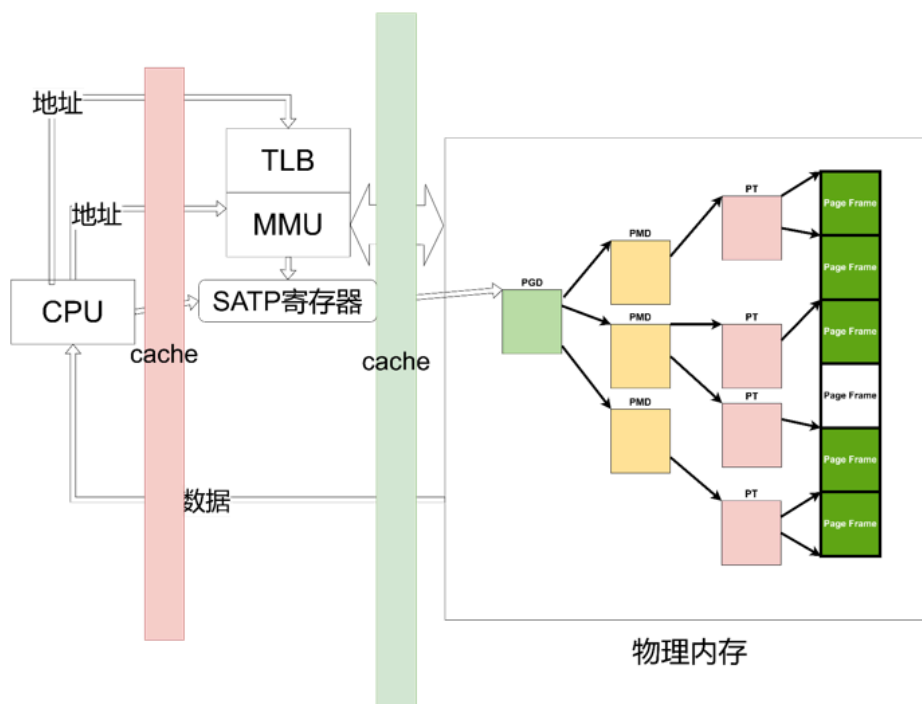


通过把内存中的部分进行压缩，来进行存储

我们在安卓中回收进程，会优先选择那些可以被换出的进程进行杀死，一些高权限的System也不能随便杀死



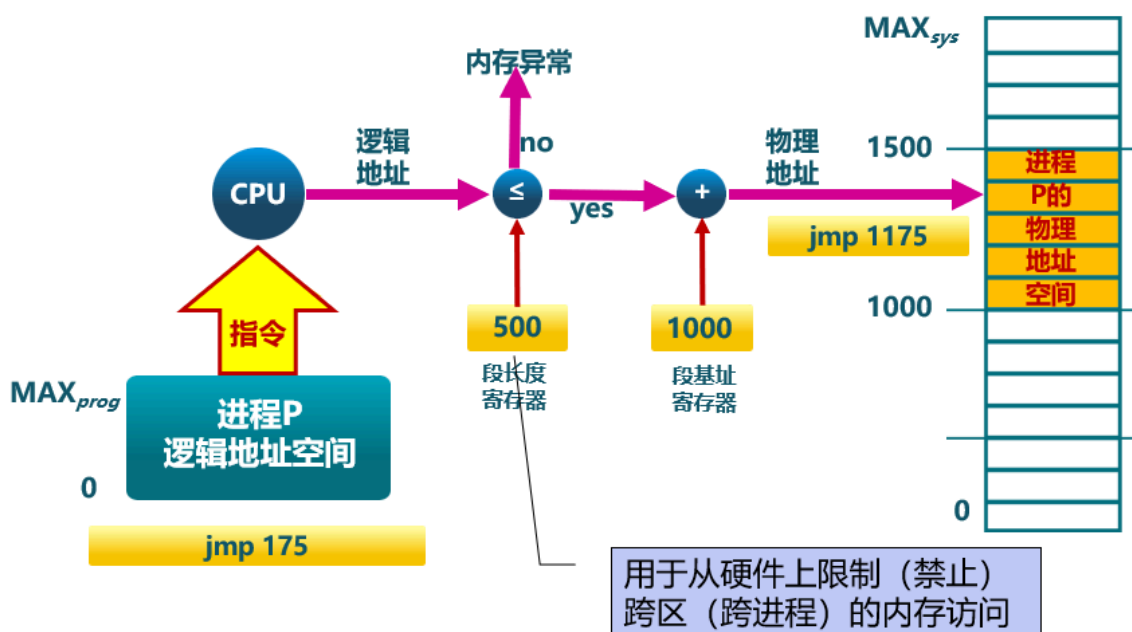
CPU的cache：



我们将cache放在虚拟地址的位置，还是放在已经经过了MMU的转换的物理地址？——放到红色的cache位置

第四章 进程间通信与并发控制

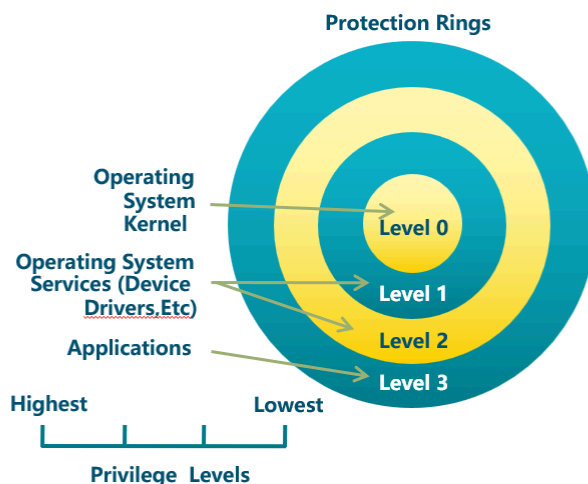
多进程共存的情况，各进程之间不能互相影响其他的进程——从硬件上完成了跨区的访问限制



所以，在虚拟内存中，各进程所占的空间都是隔离开的

进程间隔离可以借助页表实现（分配虚拟内存空间的时候直接隔开就可），进程内区域间隔离需要硬件支持——对于内核中的一些数据，我们不能让用户空间来进行访问，会出问题

操作系统所存储的数据位置，我们不能让用户随意访问，所以我们按照权限等级来对操作系统和应用程序做了一个区分



我们给操作系统内核设置了ring0的级别，给应用程序设置了ring3的级别。

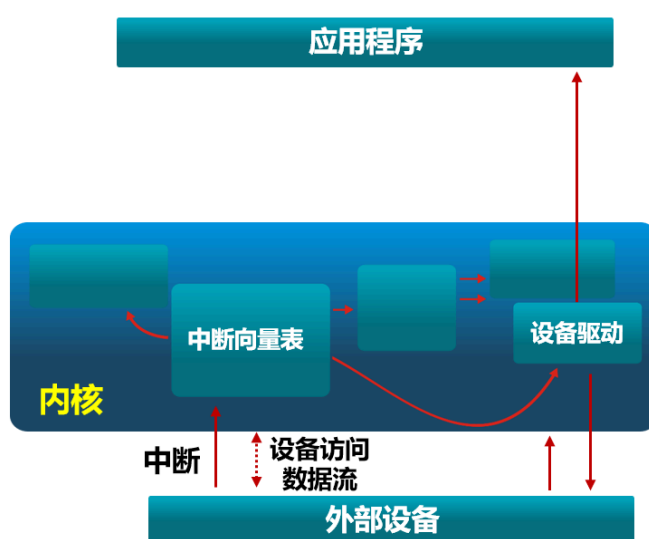
- 所以，我们用户程序的**中断**时，会**提升对应的权限等级**，也就是从ring3提升到了ring0，这是在中断的过程中就完成的
- 然后在中断返回的时候，将**权限等级降下来**即可——有些OS设计的时候，在回收时忘记恢复权限等级，会出问题！

OS中设计了三种可以在运行时“呼叫”操作系统，并提升权限的方式

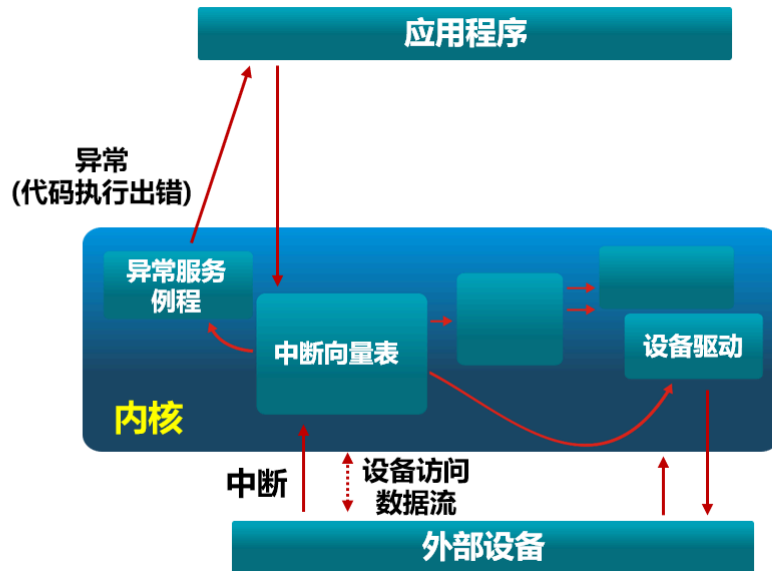
- 中断 (Interrupt)
- 异常 (Exception)
- 系统调用 (System Call)

这些机制能够提升当前CPU的权限（改变当前的CPU状态到ring0），同时，跳转PC到指定的函数入口处（提升权限后只能执行OS**预定的函数**，例如中断向量表中的函数，不能去访问其他的内核的函数）

中断：



异常：



中断和异常是由硬件实现的一种跳转机制，在某些特定事件发生时如设备按下（中断），或除数为0导致指令无法执行（异常），需要打断现有的指令流，改变当前权限状态，并调用预设的处理函数（即PC设置到该函数的入口地址）

系统调用syscall:

也就是应用程序需要执行某个高权限的操作的时候，**主动**向操作系统请求高权限服务

在系统调用的过程中:

```
int
sys_read(int64_t fd, void *base, size_t len) {
    return syscall(SYS_read, fd, base, len);
}
```

首先，调用了sys_read函数，返回一个syscall函数的值

```
Terminal 终端
Terminal 终端 - xiaoli@xiaoli-VirtualBox: /m
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
/* syscall number */
#define SYS_exit      1
#define SYS_fork      2
#define SYS_wait      3
#define SYS_exec      4
#define SYS_clone     5
#define SYS_yield     10
#define SYS_sleep     11
#define SYS_kill      12
#define SYS_gettime   17
#define SYS_getpid    18
#define SYS_mmap      20
#define SYS_munmap    21
#define SYS_shmem     22
#define SYS_putc      30
#define SYS_pgdir     31
#define SYS_open     100
#define SYS_close    101
#define SYS_read     102
#define SYS_write    103
#define SYS_seek     104
```

对应的，每一个函数都有对应的一个标号，比如read函数是102号

```
syscall(int num, ...) {
    ...
}
```

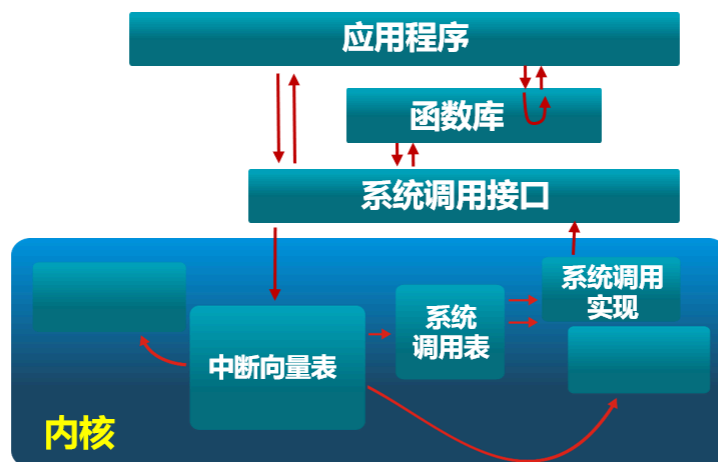
```
asm volatile (
    "lw a0, %1\n"
    "lw a1, %2\n"
    "lw a2, %3\n"
    "lw a3, %4\n"
    "lw a4, %5\n"
    "lw a5, %6\n"
    "ecall\n"//这一句会产生一个“中断”
    "sw a0, %0"
    : "=m" (ret)
    : "m" (num), //这一句的作用是把102放进了a0
      "m" (a[0]),
      "m" (a[1]),
      "m" (a[2]),
      "m" (a[3]),
      "m" (a[4])
    : "memory"
);    return ret;
```

这个syscall函数，将num的值放入了a0寄存器中

所以，系统调用事件有一个固定中断号，同时借助一个寄存器，传递了系统调用中具体哪个调用（系统调用号）

最后，执行完之后，还需要返回到原先的权限等级，不然就会出问题。

对应的实现流程：



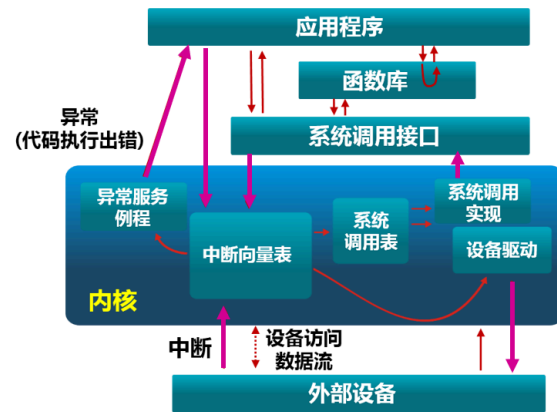
程序访问通常是通过高层次的API接口而不是直接进行系统调用

三种API:

- Windows:Win32 API
- UNIX, LINUX, Mac OS X:POSIX API
- JVM:Java API

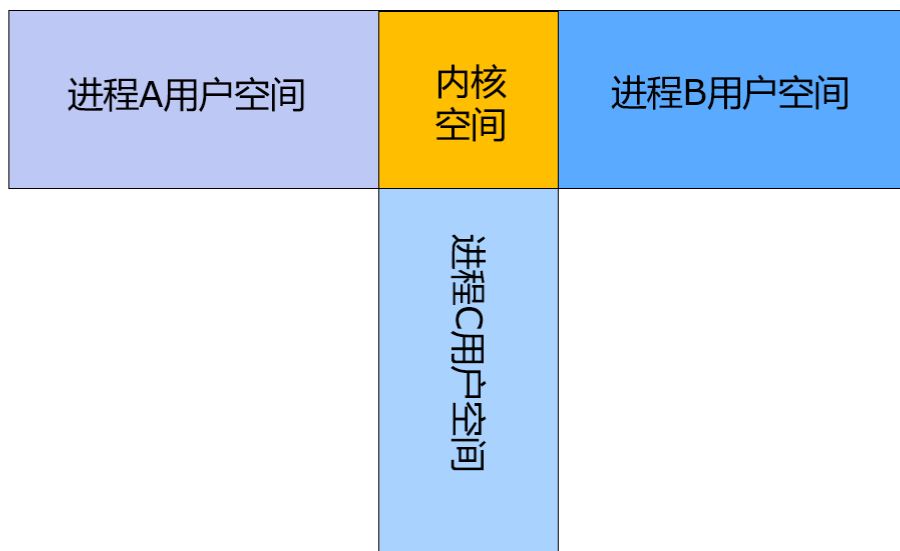
三个调用方式的对比:

- 源头
 - ▣ 中断：外设
 - ▣ 异常：应用程序意想不到的行为
 - ▣ 系统调用：应用程序请求操作提供服务
- 响应方式
 - ▣ 中断：异步
 - ▣ 异常：同步
 - ▣ 系统调用：异步或同步
- 处理机制
 - ▣ 中断：持续，对用户应用程序是透明的
 - ▣ 异常：杀死或者重新执行意想不到的应用程序指令
 - ▣ 系统调用：等待和持续

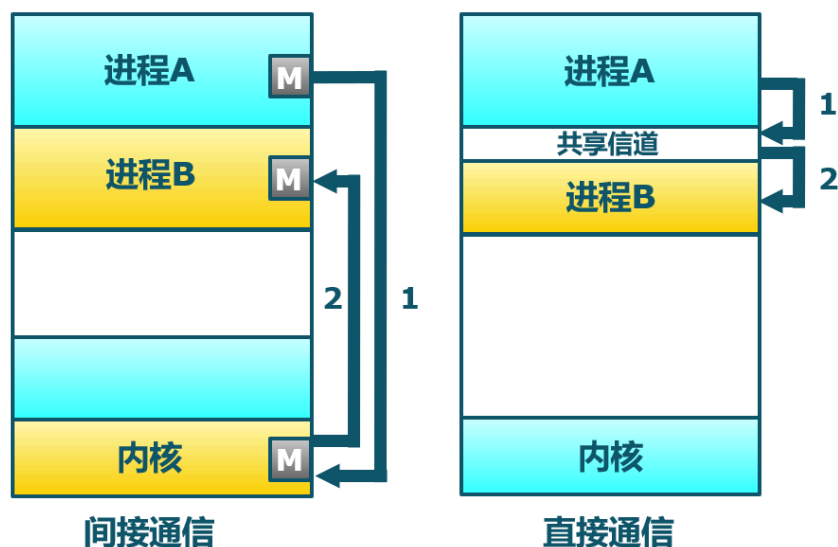


进程间怎么通信？

——为了便于管理，内核没有一人一份，而是所有进程共享的



有两种通信方式——直接通信和间接通信



- **间接通信**：就是进程A先将数据存储在某个位置，然后进程B再去对应的位置去取走对应的数据——每个消息队列都有一个唯一的标识，只有共享了相同消息队列的进程，才能够通信
- **直接通信**：就是进程A直接将数据发送给进程B

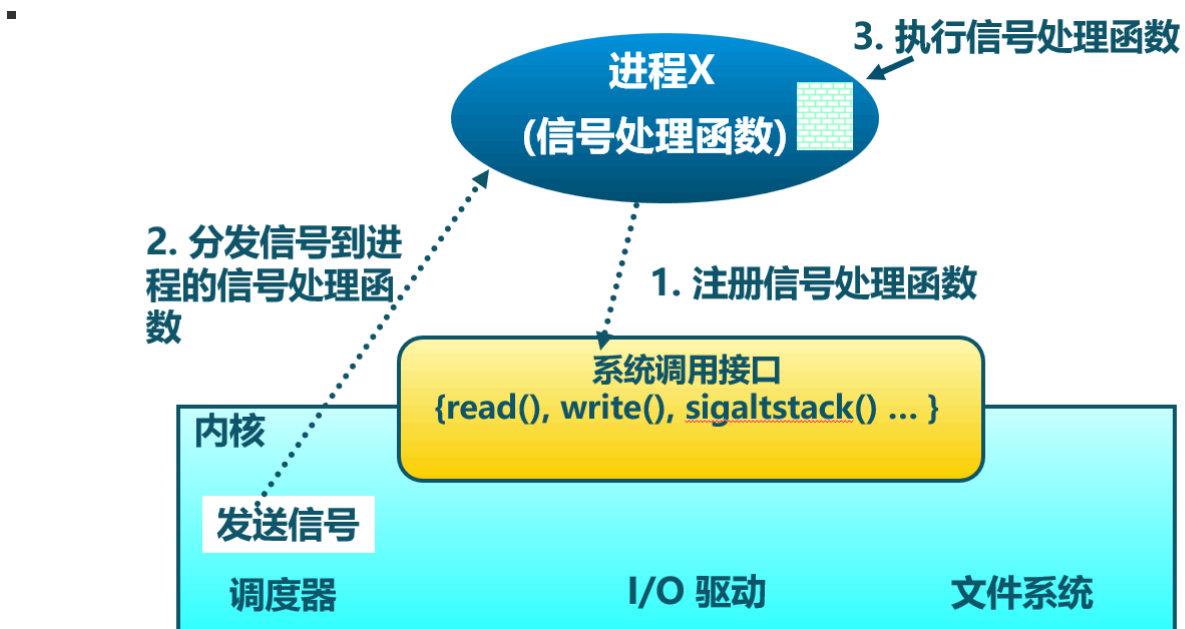
进程通信分为阻塞和非阻塞通信——阻塞通信就是同步的，相当于你发送一个进程的时候，不能干别的事；非阻塞通信就是异步的，就是你在发送一个进程的时候，可以干其他的事

发送消息在链路上的三种缓冲方式：

- **0容量**：发送方必须等待接收方
- **有限容量**：通信链路缓冲队列满时，发送方必须等待
- **无限容量**：发送方不需要等待

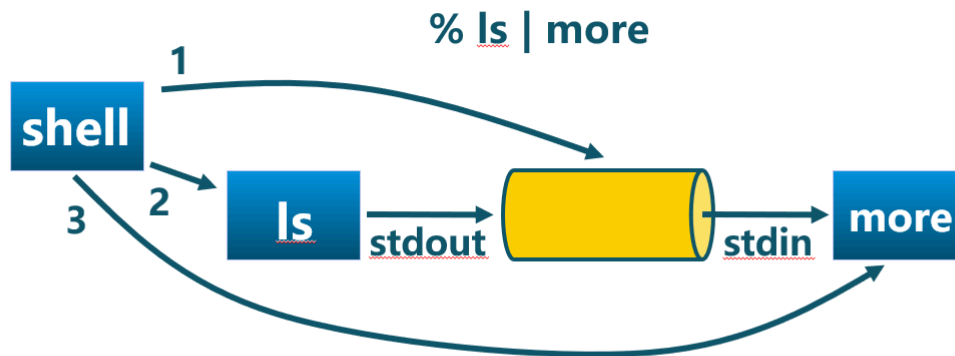
常见的四种进程管理的方法

- **信号**：进程间的软件中断通知和处理机制——SIGKILL, SIGSTOP, SIGCONT



- **管道**：分为读管道和写管道
 - 读管道：read(fd, buffer, nbytes)
 - 写管道：write(fd, buffer, nbytes)
 - 创建管道：pipe(rgfd)
 - rgfd是2个文件描述符组成的数组
 - rgfd[0]是读文件描述符
 - rgfd[1]是写文件描述符

▪



shell

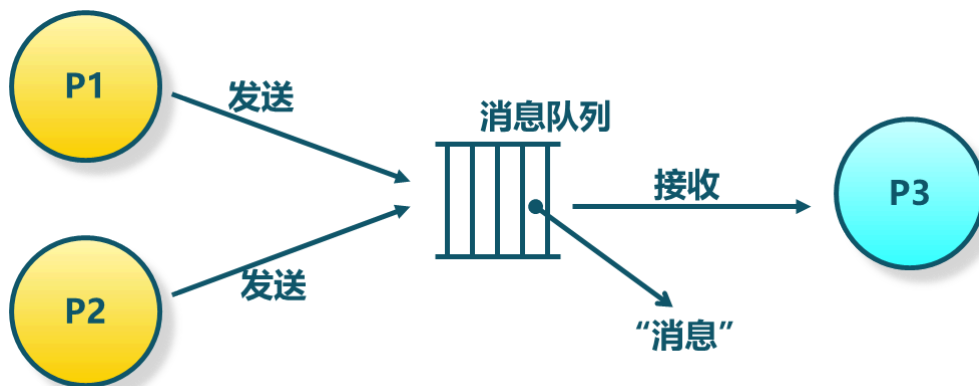
创建管道

为ls创建一个进程, 设置 `stdout` 为 管道写端

为more 创建一个进程, 设置 `stdin` 为管道读端

- **消息队列**: 每个消息(Message)是一个字节序列, 相同标识的消息组成按先进先出顺序组成一个消息队列 (Message Queues)

▪



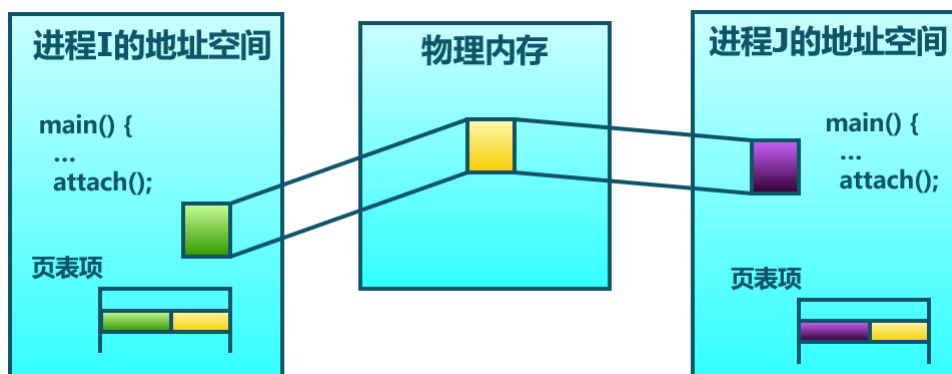
- 对于消息队列的系统调用:

- `msgget ( key, flags )` —— 获取消息队列标识
- `msgsnd ( QID, buf, size, flags )` —— 发送消息
- `msgrcv ( QID, buf, size, type, flags )` —— 接收消息
- `msgctl( ... )` —— 消息队列控制

- **共享内存**: 共享内存是把同一个物理内存区域同时映射到多个进程的内存地址空间的通信机制

- 因为如果我们每次想要共享数据的时候, 都去访问对应的内核空间的话, 这样太浪费时间了
- 但是我们又不能直接去读取其他进程的数据, 因为每个进程的空间都是私有的
- 为了解决这个问题, 我们给每个进程的内存地址空间明确设置共享内存段

▪



- 一些函数:

- `shmget( key, size, flags)` ——创建共享段
- `shmat( shmid, *shmaddr, flags)` ——把共享段映射到进程地址空间
- `shmdt( *shmaddr)` ——取消共享段到进程地址空间的映射
- `shmctl( ...)` ——共享段控制

举个例子：

```
3 int main(int argc, char **argv)
4 {
5     int running = 1;
6     void *shared_memory = (void *)0;
7     struct shared_use_st *shared_stuff;
8     int shmid;
9
10    srand((unsigned int) getpid());
11
12    shmid = shmget((key_t)1234, sizeof(struct shared_use_st), 0666|IPC_CREAT);
13    if(shmid == -1){
14        fprintf(stderr, "shmget failed\n");
15        exit(-1);
16    }
17
18    shared_memory = shmat(shmid, (void *)0, 0);
19    if(shared_memory == (void *)-1){
20        fprintf(stderr, "shmat failed\n");
21        exit(-1);
22    }
23
24    shared_stuff = (struct shared_use_st *)shared_memory;
25    shared_stuff->written_by_you = 0;
26    while(running){
27        if(shared_stuff->written_by_you){
28            printf("You wrote: %s", shared_stuff->some_text);
29            sleep(rand()%4);
30            shared_stuff->written_by_you = 0;
31            if(strncmp(shared_stuff->some_text, "end", 3) == 0){
32                running = 0;
33            }
34        }
35    }
36}
```

这个例子中，我们的两个进程就通过key为1234来进行通信，但是有个缺点，就是容易被监听对应的通信内容，只要key被别人知道了的话

线程

进程可以很好的支持多段程序的快速交叉执行，但是不能实现数据高效共享——出现了线程

线程是进程内一个相对独立的、具有可调度特性的执行单元。

线程举例：

```

1  #include <stdio.h>
2  #include <pthread.h>
3  void* thread( void *arg )
4  {
5      printf( "This is a thread and arg = %d.\n", *(int*)arg);
6      *(int*)arg = 0;
7      return arg;
8  }
9  int main( int argc, char *argv[] )
10 {
11     pthread_t th;
12     int ret;
13     int arg = 10;
14     int *thread_ret = NULL;
15     ret = pthread_create( &th, NULL, thread, &arg );
16     if( ret != 0 ){
17         printf( "Create thread error!\n");
18         return -1;
19     }
20     printf( "This is the main process.\n" );
21     pthread_join( th, (void**)&thread_ret );
22     printf( "thread_ret = %d.\n", *thread_ret );
23     return 0;
24 }

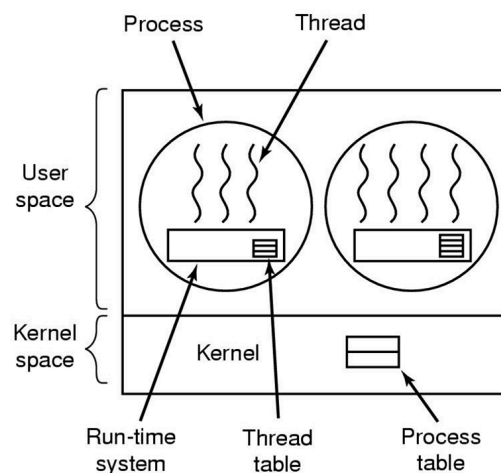
```

我们在main函数中，出现了一个ret等于pthread_create(&th,NULL,thread,&arg)，去调用了前面的thread函数，也就是说新开了一个线程

实现进程机制的多种方案：

- **纯用户级线程（ULT）**：线程的管理全部由用户程序完成，核心部分只对进程管理，但增加“线程库”概念。

▪

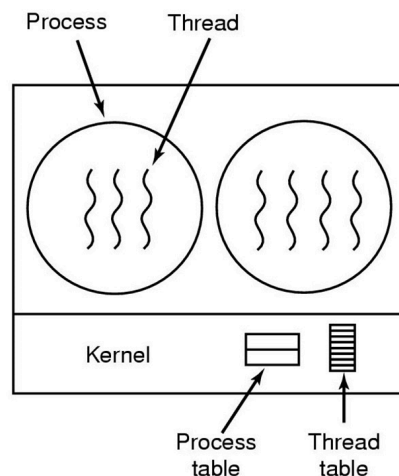


- 对应的进程表在内核区，而线程表由用户控制，在用户区

- 用户可以调用线程相关的函数

- pthread_create——创建新线程
- pthread_exit——结束调用的线程
- pthread_join——等待一个线程退出（同步）
- pthread_yield——释放CPU让其运行另一线程
- pthread_attr_init——初始化一个线程属性结构

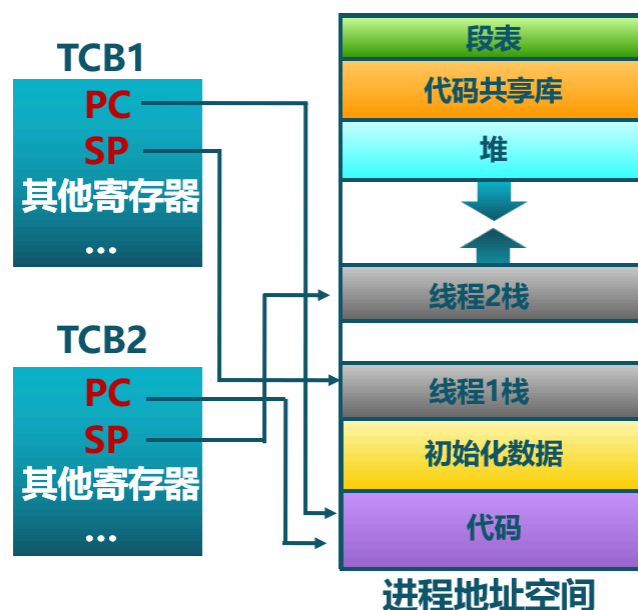
- pthread_attr_destroy——删除一个线程属性结构
- 纯用户级线程的优势
 - **线程切换不需要内核模式特权**
 - 线程调用可以是应用程序级的,根据需要可改变调度算法,但不会影响底层的操作系统调度程序.
 - ULT管理模式**可以在任何操作系统中运行**,不需要修改系统内核,线程库是提供应用的实用程序。
- 纯用户级线程的劣势
 - 系统调用会引起进程阻塞
 - 这种线程不利于使用多处理器并行
- **核心级线程 (KLT)：** 线程由OS内核进行管理，内核给应用程序级提供系统调用，实现对线程的使用。
-



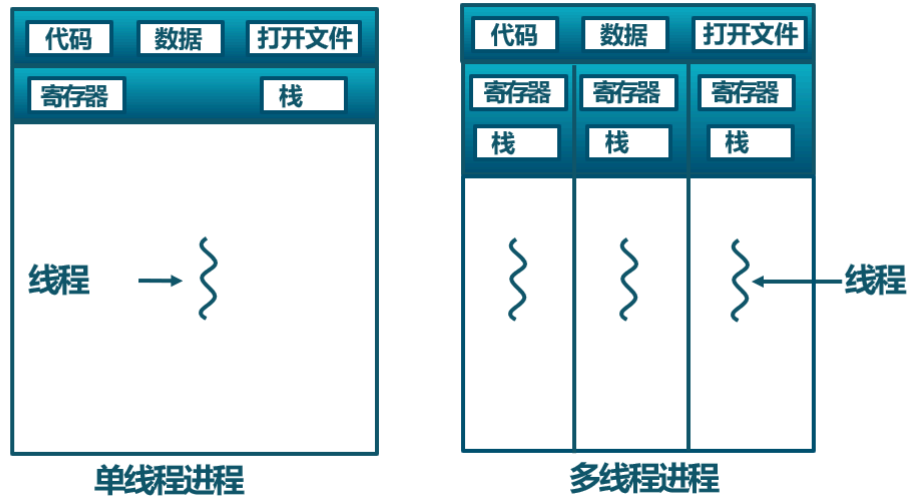
- **线程和进程都在用户空间完成；进程表和线程表都放在核心区**

线程的概念：它是进程中的指令执行流的最小单元，是CPU调度的基本单位。

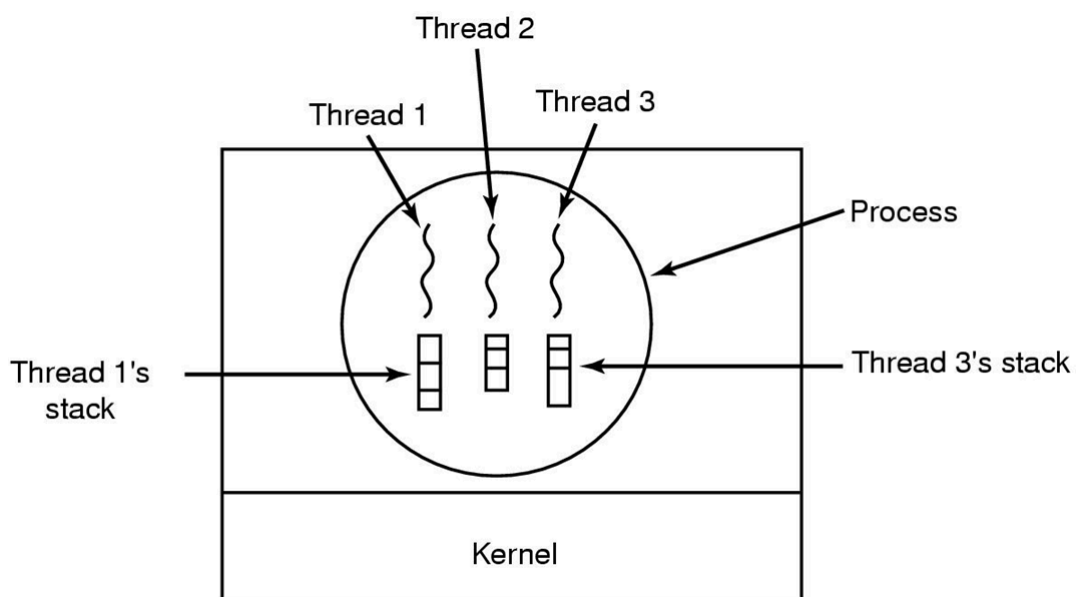
- 进程的资源分配角色：进程由一组相关资源构成，包括地址空间（代码段、数据段）、打开的文件等各种资源
- 线程的处理机调度角色：线程描述在进程资源环境中的指令流执行状态



在一个进程里，可以只有一个线程，也可以有多个线程！



每一个线程都要有自己的独立的栈区，因为我们的线程执行是独立的，需要独立的堆栈。



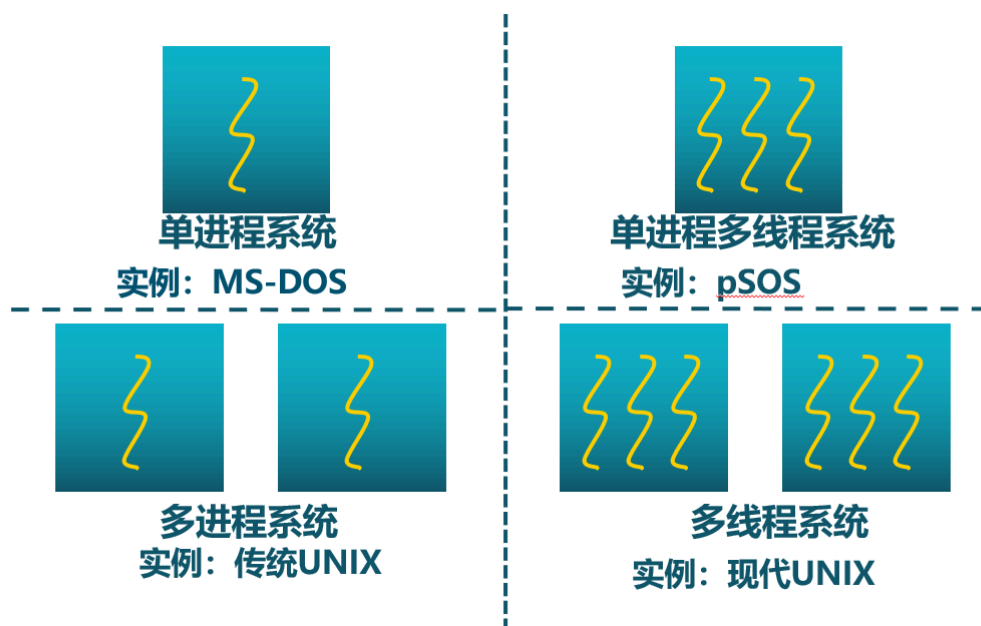
OS记录一个进程的执行状态所需的数据：

- 上下文（寄存器组）保存当前状态
- 栈（用于保存函数调用的序列和局部变量）

线程的共享项和私有项：

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

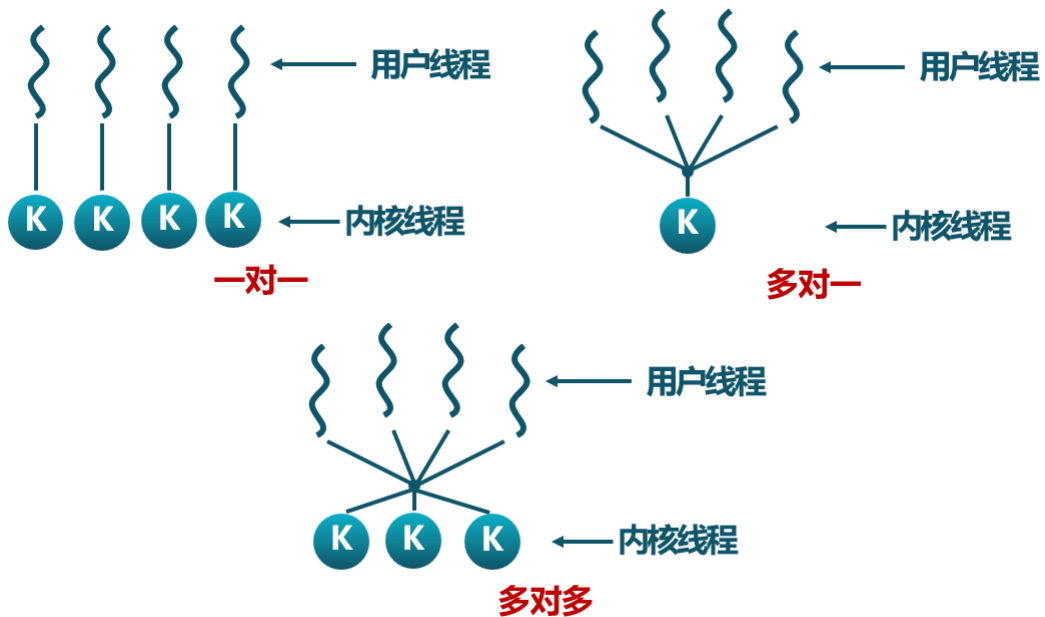
各类的线程进程系统：



线程VS进程：

- 进程是资源分配单位，线程是CPU调度单位
- 进程拥有一个完整的资源平台，而线程只独享指令流执行的必要资源，如寄存器和栈
- 线程具有**就绪、等待和运行**三种基本状态和状态间的转换关系
- 线程能减少并发执行的时间和空间开销
 - 线程的创建时间比进程短
 - 线程的终止时间比进程短
 - 同一进程内的线程切换时间比进程短
 - 由于**同一进程的各线程间共享内存和文件资源，可不通过内核进行直接通信**

线程和进程的三种关系：



并发控制

举一个例子：

Spooler Dir

.....
4: File1
5: File2
6: File3
7: Null
8: Null
.....

Out: 4

In: 7

```
Proc A: N_f_s = In; //In == 7
        InsertFileIntoSpooler(N_f_s);
        In=N_f_s++; //In == 8
```

↓ CPU switch (Correct)

```
Proc B: N_f_s = In; //In == 8
        InsertFileIntoSpooler(N_f_s);
        In=N_f_s++; //In == 9
```

我们现在在我们的管道内有456已经存放了对应的文件，7和8是空的

然后我们有两个进程，分别去往里面存文件

如果没有在中间被中断的话，我们的存储是正确的

但是如果被打断的话，就会出现这样的情况：

Spooler Dir

.....
4: File1
5: File2
6: File3
7: Null
8: Null
.....

Out: 4

In: 7

Proc A: N_f_s = In; //In == 7

↓ CPU switch

Proc B: N_f_s = In; //In == 7
InsertFileIntoSpooler(N_f_s);
In=N_f_s++; //In == 8

↓ CPU switch, Proc B lost data

Proc A:
InsertFileIntoSpooler(N_f_s);
In=N_f_s++; //In == 8

我们的进程A在执行完赋值操作之后，没有做加一运算，就被打断了，所以导致后面那个进程使用的编号还是7，这样就会出问题！

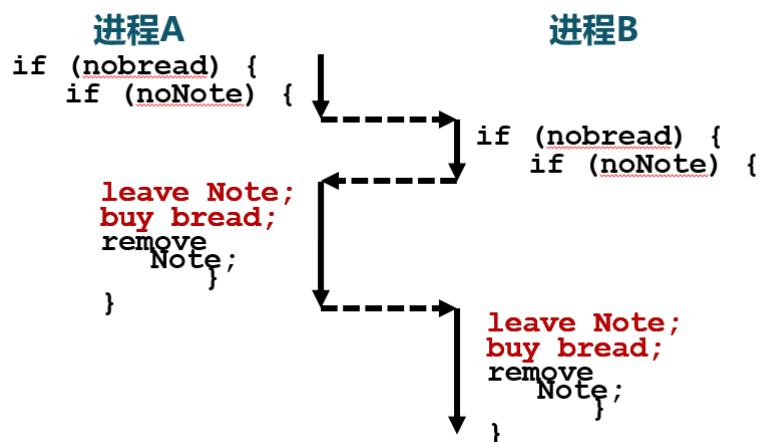
案例分析：寝室买面包问题

方案1:

```
if (nobread) {  
    if (noNote) {  
        leave Note;  
        buy bread;  
        remove Note;  
    }  
}
```

我们使用一张便签来标记自己要去买面包了，这样下一个人看到标签之后，就不会去买面包了

但是这样不太可行，如下所示：



我们在进程A发现冰箱上没有便签的时候，正准备要往上贴标签，这时候发生了中断，这样的话进程B去判断有没有标签，发现没有标签，然后再中断，这样，两个人都会以为没有人去买面包，这样两个人都会去买面包

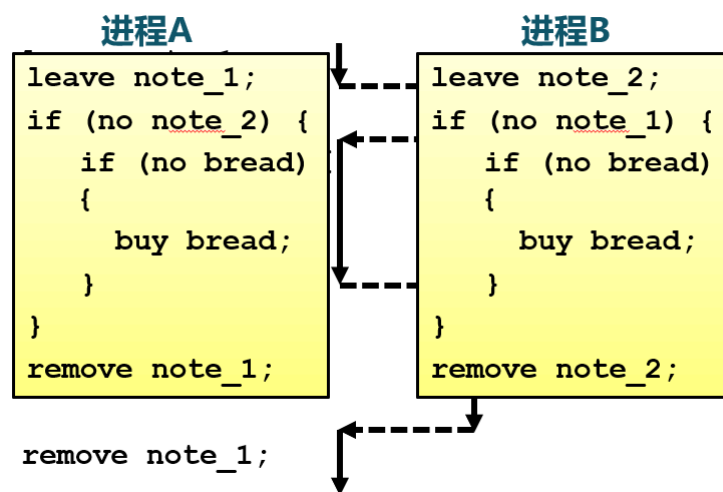
方案2:

```
leave Note;
if (nobread) {
    if (noNote) {
        buy bread;
    }
}
remove note;
```

我们首先去留一个标签，然后看到没有面包+没有标签，再去买

这样是肯定错误的，因为可以看到自己留的标签，所以肯定都不会去买面包

方案3:



就是给便签加上标记，标记成自己的序号

但是这样也不行，两个人都会以为对方去买面包了，就都不买了

方案4:



这样是有效的，但是造成了两个进程的代码不相同 难以维护

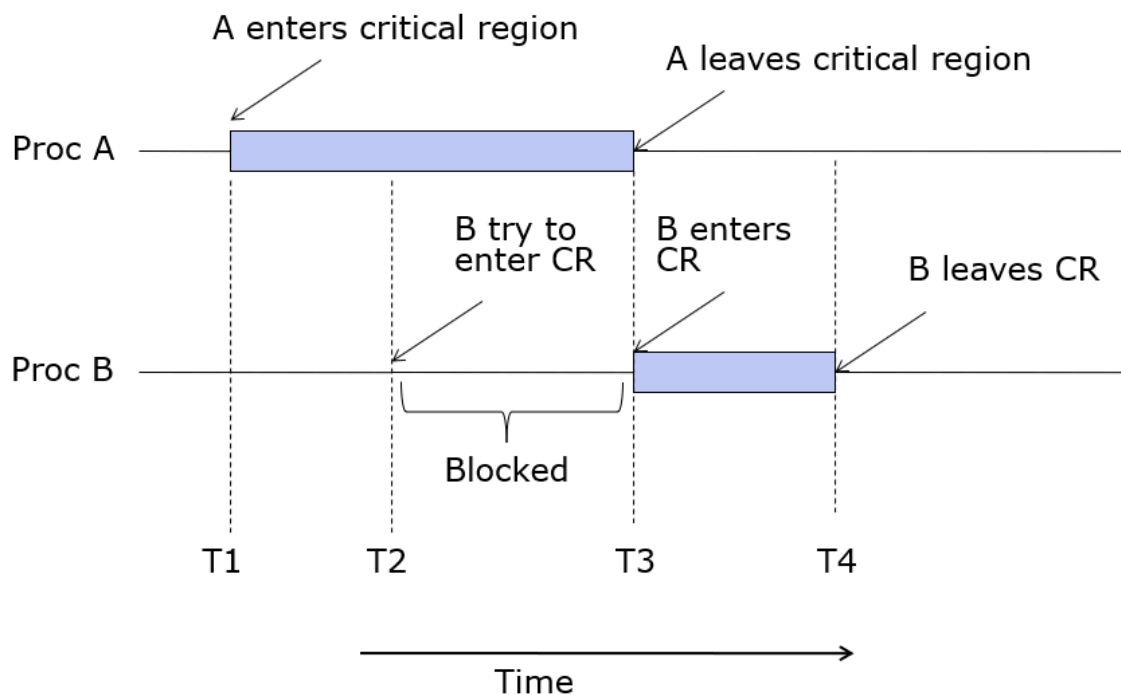
而且进程A在等待的时候 实际上处于忙等待的状态

采用临界区进行处理

entry section
critical section
exit section
remainder section

- 临界区
 - 进程中访问临界资源的一段需要互斥执行的代码
- 进入区
 - 检查可否进入临界区的一段代码
 - 如可进入，设置相应“正在访问临界区”标志
- 退出区
 - 清除“正在访问临界区”标志
- 剩余区
 - 代码中的其余部分

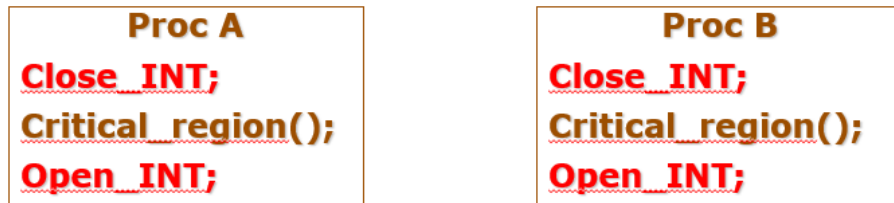
具体流程：



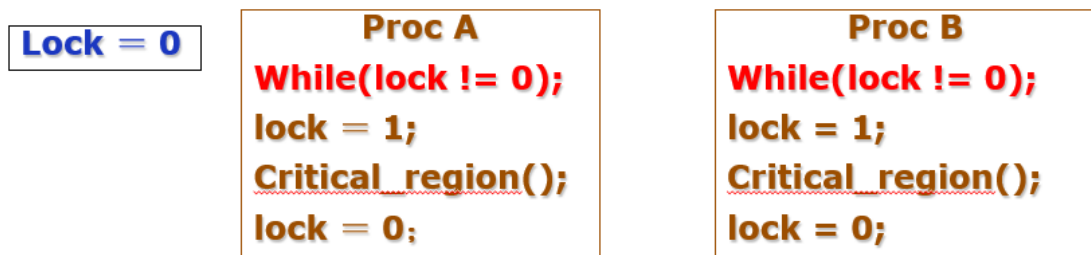
临界区的访问规则：

- 空闲就进入
 - 没有进程在临界区时，任何进程可进入
- 忙的话就等待
 - 有进程在临界区时，其他进程均不能进入临界区
- 有限的等待
 - 等待进入临界区的进程不能无限期等待
- 让权等待
 - 不能进入临界区的进程，应释放CPU（如转换到阻塞状态）

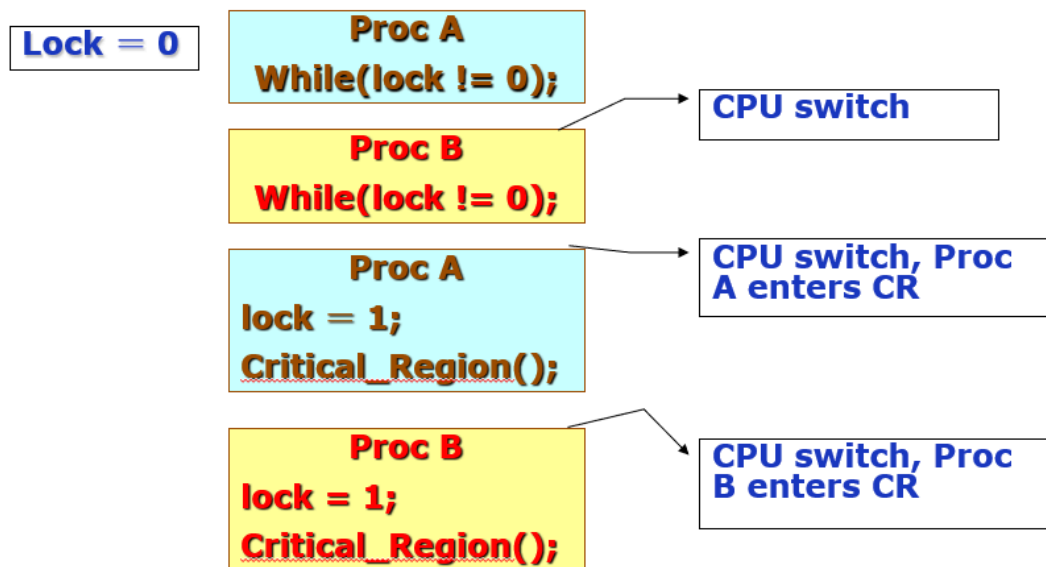
我们可以关闭中断，来防止我们在做一些重要操作的时候发生中断和切换



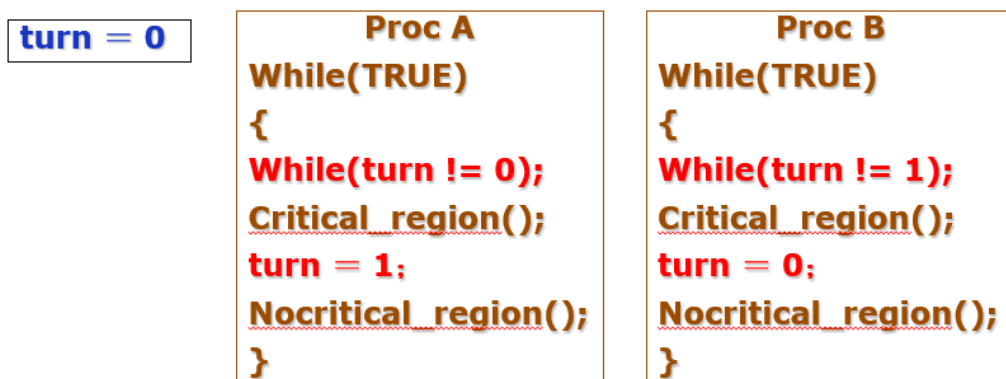
采用锁机制——也不能完全解决问题



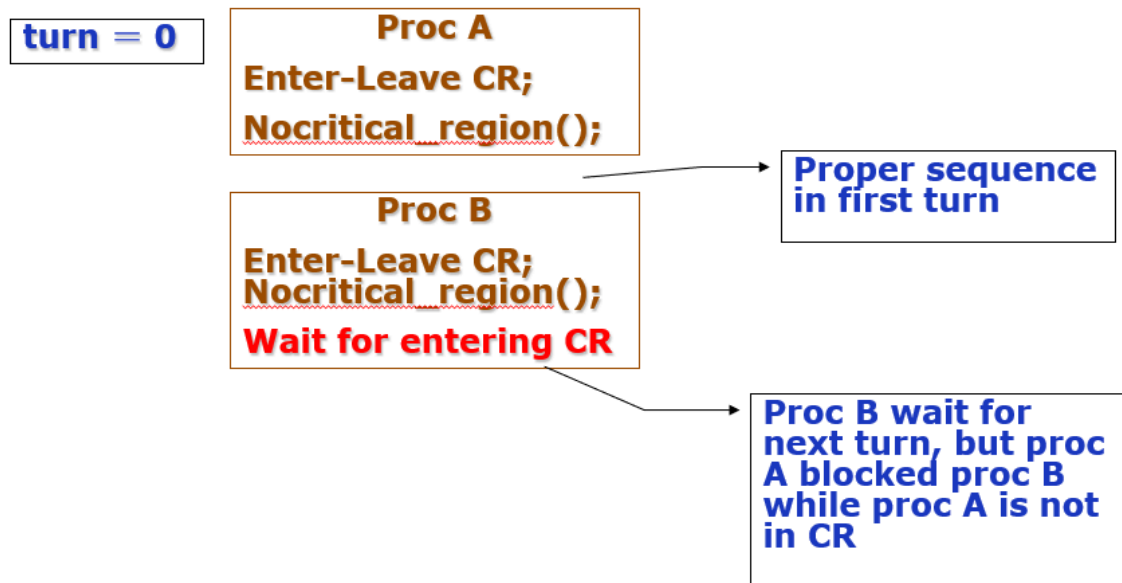
出现以下的问题：



采用while语句等待



这样也会出问题



成功的算法:

Peterson算法

■ 共享变量

```
int turn; //表示该谁进入临界区  
boolean flag[]; //表示进程是否准备好进入临界区
```

■ 进入区代码

```
flag[i] = true;  
turn = j;  
while (flag[j] && turn == j)
```

■ 退出区代码

```
flag[i] = false;
```

线程Ti 的代码

```
do {  
    flag[i] = true;  
    turn = j;  
    while ( flag[j] && turn == j);  
  
    CRITICAL SECTION  
  
    flag[i] = false;  
  
    REMAINDER SECTION  
  
} while (true);
```

Busy-Waiting Correct Solution(Peterson's)

```
boolean flag [2];
int turn;
void P0()
{
    while (true)
    {
        flag [0] = true;
        turn = 1;
        while (flag [1] && turn == 1)
            /* do nothing */;
        /* critical section */;
        flag [0] = false;
        /* remainder */;
    }
}

void P1()
{
    while (true)
    {
        flag [1] = true;
        turn = 0;
        while (flag [0] && turn == 0)
            /* do nothing */;
        /* critical section */;
        flag [1] = false;
        /* remainder */;
    }
}

void main()
{
    flag [0] = false;
    flag [1] = false;
    parbegin (P0, P1);
}
```

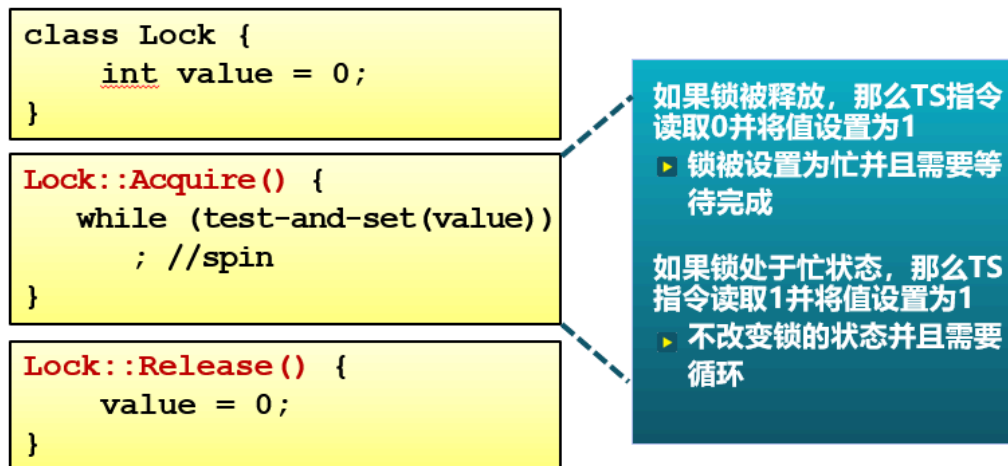
Dekkers算法

线程Ti 的代码

```
flag[0] := false; flag[1] := false; turn := 0; //or 1
do {
    flag[i] = true;
    while flag[j] == true {
        if turn ≠ i {
            flag[i] := false
            while turn ≠ i { }
            flag[i] := true
        }
        CRITICAL SECTION
        turn := j
        flag[i] = false;
        EMAINDER SECTION
    } while (true);
```

Test and Set Lock (TSL)solution

这是硬件上的实现，完成了值的存储和赋值两步操作



■ 线程在等待的时候消耗CPU时间

锁——Lock

通过加锁和解锁来完成对进程的控制

- 加锁: Lock::Acquire()
- 解锁: Lock::Release()

使用锁来控制临界区访问

```
lock_next_pid->Acquire();
new_pid = next_pid++;
lock_next_pid->Release();
```

当一个进程处于等待的状态的时候，他不一定一直在那边等，还可以将自己的进程让出去，让别人去调度

两种等待情况：无忙等待就是将自己的进程让出去，然后等现在在运行的那个线程去唤醒原来那个让出去的进程回来执行

忙等待

```
Lock::Acquire() {
    while (test-and-set(value))
        ; //spin
}

Lock::Release() {
    value = 0;
}
```

无忙等待

```
class Lock {
    int value = 0;
    WaitQueue q;
}

Lock::Acquire() {
    while (test-and-set(value)) {
        add this TCB to wait queue q;
        schedule();
    }
}

Lock::Release() {
    value = 0;
    remove one thread t from q;
    wakeup(t);
}
```

生产消费问题的解决方案：

```
#define N 100
int lock = 0
int count = 0
```

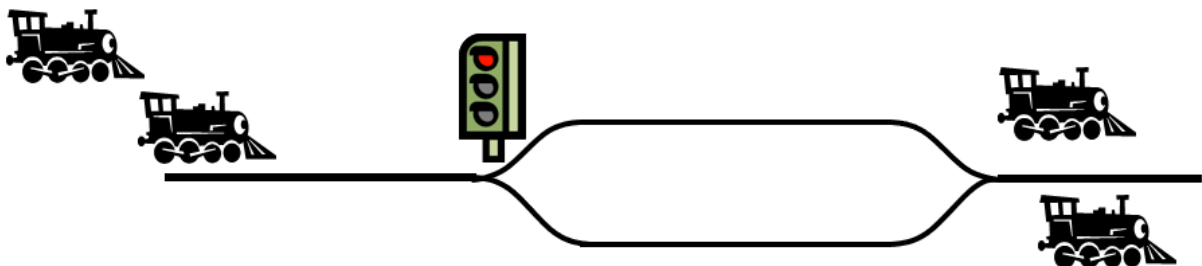
```
Producer
While(TRUE)
{
Produce-Item();
if(count == N)
    sleep();
Enter-item();
count = count + 1
if(count == 1)
wakeup(consumer);
}
```

```
Consumer
While(TRUE)
{
if(count == 0)
    sleep();
Remove-Item();
count = count - 1
if(count == N-1)
wakeup(producer);
Consume-item();
}
```

信号量

- 信号是一种抽象数据类型
 - ▣ 由一个整形 (**sem**)变量和两个原子操作组成
 - ▣ **P()** (Prolaag (荷兰语尝试减少))
 - ▣ sem减1
 - ▣ 如sem<0, 进入等待, 否则继续
 - ▣ **V()** (Verhoog (荷兰语增加))
 - ▣ sem加1
 - ▣ 如sem≤0,唤醒一个等待进程

原理：



首先，一共四辆车，能容纳两辆车

首先进去两辆车，然后第三辆进去的时候被告知满了，进入睡眠状态

必须等到右边的车出去了，就会给前面的等待的车发信号，通过他可以进来了

注意：与TSL指令的设计思路类似，信号量把状态检测、改变进程状态封装到了一个不可打断的操作中。由操作系统负责通过关中断等高权限操作以禁用调度程序员通过系统调用请求操作

具体的实现：

```
class Semaphore {  
    int sem;  
    WaitQueue q;  
}
```

```
Semaphore::P() {  
    sem--;  
    if (sem < 0) {  
        Add this thread t to q;  
        block(p);  
    }  
}
```

```
Semaphore::V() {  
    sem++;  
    if (sem <= 0) {  
        Remove a thread t from q;  
        wakeup(t);  
    }  
}
```

每个临界区设置一个信号量，其初值为1

```
mutex = new Semaphore(1);
```

```
mutex->P();  
Critical Section;  
mutex->V();
```

- 必须**成对使用P()操作和V()操作**
 - ▣ P()操作保证互斥访问临界资源
 - ▣ V()操作在使用后释放临界资源
 - ▣ PV操作**不能次序错误、重复或遗漏**

或者我们使用信号量来完成同步：

每个条件同步设置一个信号量，其初值为0

```
condition = new Semaphore(0);
```

线程A

```
... M ...  
condition->P();  
... N ...
```

线程B

```
... X ...  
condition->V();  
... Y ...
```



当线程A遇到N前，把自己锁住了，必须等线程B做V释放操作，才能执行N

简单的IPC模型

生产消费者模型



- 有界缓冲区的生产者-消费者问题描述
 - ▣ 一个或多个**生产者**在生成数据后放在一个缓冲区里
 - ▣ 单个**消费者**从缓冲区取出数据处理
 - ▣ 任何时刻**只能有一个**生产者或消费者可访问缓冲区
- 问题分析
 - ▣ 任何时刻只能有一个线程操作缓冲区 (**互斥访问**)
 - ▣ 缓冲区空时，消费者必须等待生产者 (**条件同步**)
 - ▣ 缓冲区满时，生产者必须等待消费者 (**条件同步**)
- 用信号量描述每个约束
 - ▣ 二进制信号量mutex
 - ▣ 资源信号量fullBuffers
 - ▣ 资源信号量emptyBuffers

解决方案:

```
Class BoundedBuffer {  
    mutex = new Semaphore(1);  
    fullBuffers = new Semaphore(0);  
    emptyBuffers = new Semaphore(n);  
}
```

```
BoundedBuffer::Deposit(c) {  
    emptyBuffers->P();  
    mutex->P();  
    Add c to the buffer;  
    mutex->V();  
    fullBuffers->V();  
}
```

```
BoundedBuffer::Remove(c) {  
    fullBuffers->P();  
    mutex->P();  
    Remove c from buffer;  
    mutex->V();  
    emptyBuffers->V();  
}
```

如果交换顺序，会发生什么？——死锁

```

Class BoundedBuffer {
    mutex = new Semaphore(1);
    fullBuffers = new Semaphore(0);
    emptyBuffers = new Semaphore(n);
}

```

```

BoundedBuffer::Deposit(c) {
    mutex->P();
    emptyBuffers->P();
    Add c to the buffer;
    fullBuffers->V();
    mutex->V();
}

```

```

BoundedBuffer::Remove(c) {
    mutex->P();
    fullBuffers->P();
    Remove c from buffer;
    emptyBuffers->V();
    mutex->V();
}

```

这段代码会发生死锁的故障。当我们的仓库都是空的情况下，一个消费者来取食物。调用mutex的p来锁住，然后发现没有食物，然后消费者休眠了。但是其他人进不来，不能调用mutex的p函数

我们的p和v函数，需要经过封装成类，然后我们对其进行调用即可，这样就不会出现一些风险

管程：管理了一个锁的队列

- Wait()操作：
 - 将自己阻塞在等待队列中
 - 唤醒一个等待者或释放管程的互斥访问
- Signal()操作：
 - 将等待队列中的一个线程唤醒
 - 如果等待队列为空，则等同空操作

条件变量的实现：

```

Class Condition {
    int numWaiting = 0;
    WaitQueue q;
}

```

```

Condition::Wait(lock) {
    numWaiting++;
    Add this thread t to q;
    release(lock);
    schedule(); //need mutex
    require(lock);
}

```

```

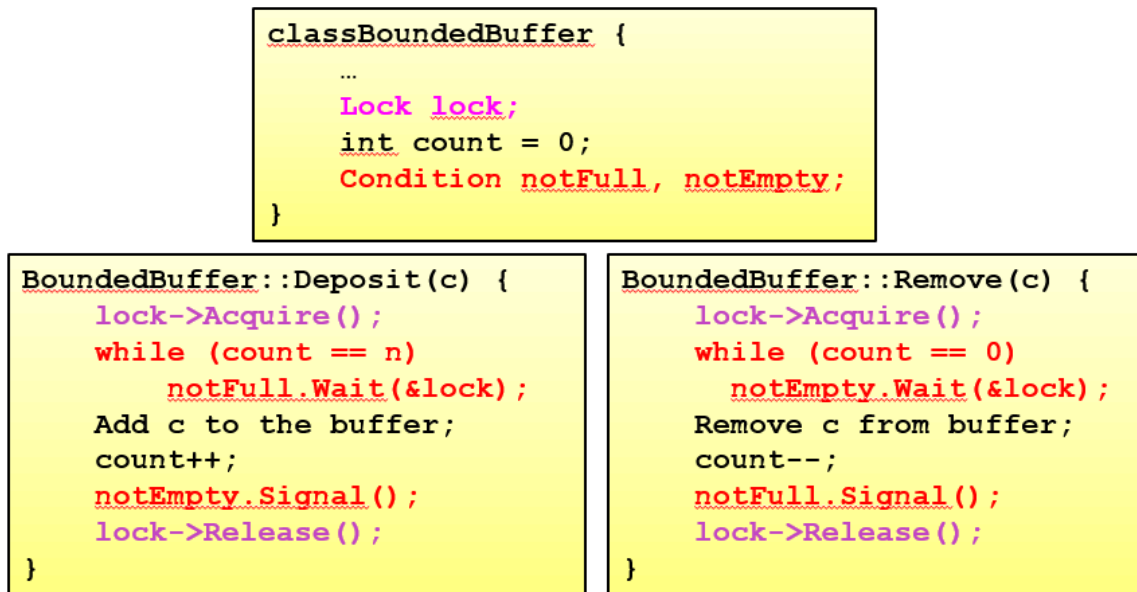
Condition::Signal() {
    if (numWaiting > 0) {
        Remove a thread t from q;
        wakeup(t); //need mutex
        numWaiting--;
    }
}

```

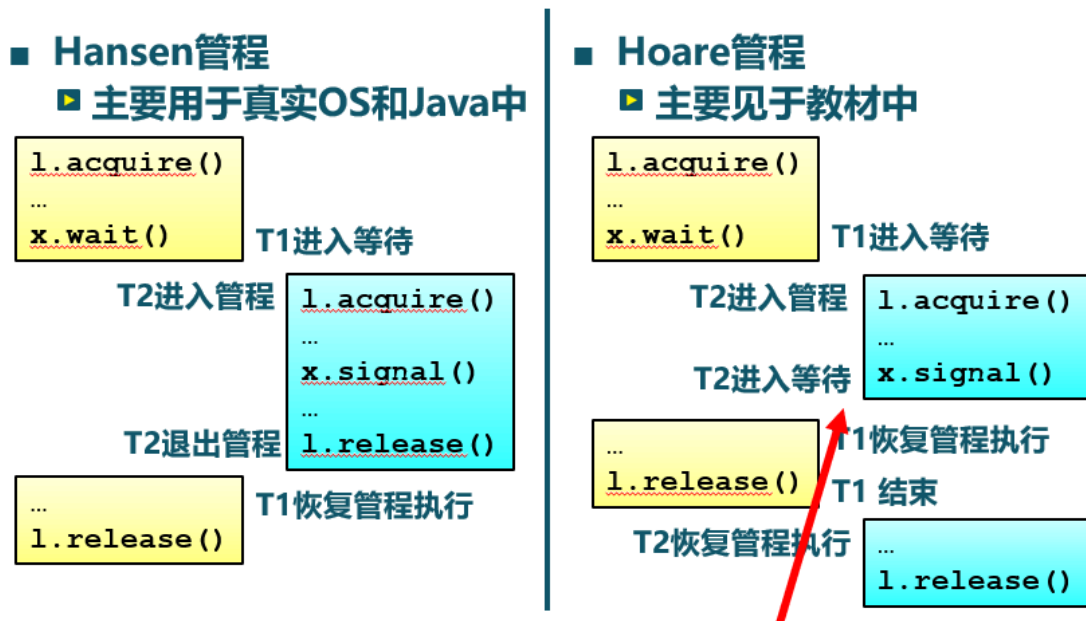
解释：我们将我们的锁的让出机制进行了封装，封装成了wait函数，首先将wait的数量加一，然后直接将lock给释放了，将自己的进程让出去给别人使用。然后，再去请求这把锁，也就是知道我们请求到这把锁，我们这个函数才会返回

同样的，Signal函数就是释放对应的进程，在释放的同时，将下一个对应的进程唤醒

解决生产者消费者问题：



管程的实现方式：



这里锁的细节需要思考如何实现

哲学家吃饭的问题：

```

#define N 5
semaphore fork[5];
void philosopher(int i)
    while(TRUE)
    {
        think( );
        if (i%2 == 0) {
            P(fork[i]);
            P(fork[(i + 1) % N]);
        } else {
            P(fork[(i + 1) % N]);
            P(fork[i]);
        }
        eat( );
        V(fork[i]);
        V(fork[(i + 1) % N]);
    }

```

// 哲学家个数
 // 信号量初值为1
 // 哲学家编号: 0 - 4
 // 哲学家在思考
 // 去拿左边的叉子
 // 去拿右边的叉子
 // 去拿右边的叉子
 // 去拿左边的叉子
 // 吃面条中....
 // 放下左边的叉子
 // 放下右边的叉子

```

typedef int semph
#define N 5
#define LEFT(i)
    (i + N - 1) % N
#define RIGHT(i)
    (i + 1) % N
#define THINKING
0
#define HUNGRY 1
#define EATING 2
int state[N];
semph mutex = 1;
semph s[N];

```

```

Philosopher
void philosopher(int i)
{
    While(TRUE){
        think( );
        take_Ch(i);
        eat();
        put_Ch(i);
    }
}

```

```

take_chs
void take_chs (int i)
{
    P(mutex);
    state[i] = HUNGRY;
    test(i);
    V(mutex);
    P(s[i]); // 得不到叉子
    阻塞, 否则继续执行
}

```

```

Put_chs
void put_forks (int i)
{
    P(mutex);
    state[i] = THINKING;
    test(LEFT(i)); // 唤醒旁
    边等叉子的人
    test(RIGHT(i));
    V(mutex);
}

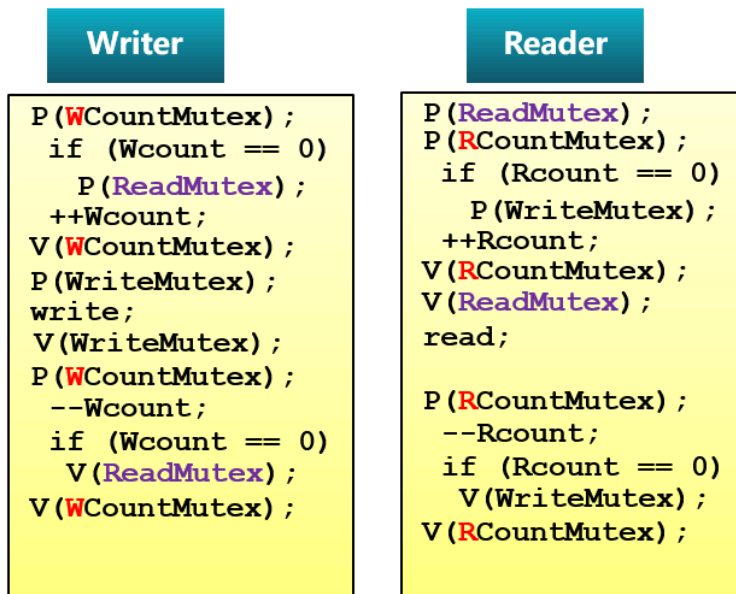
```

```

Test
void test (int i)
{
    if((state[i] == HUNGRY)
        && (state[LEFT(i)] != EATING)
        && (state[RIGHT(i)] != EATING))
    {
        state[i] = EATING;
        V(s[i]);
    }
}

```

最后的实现方案：

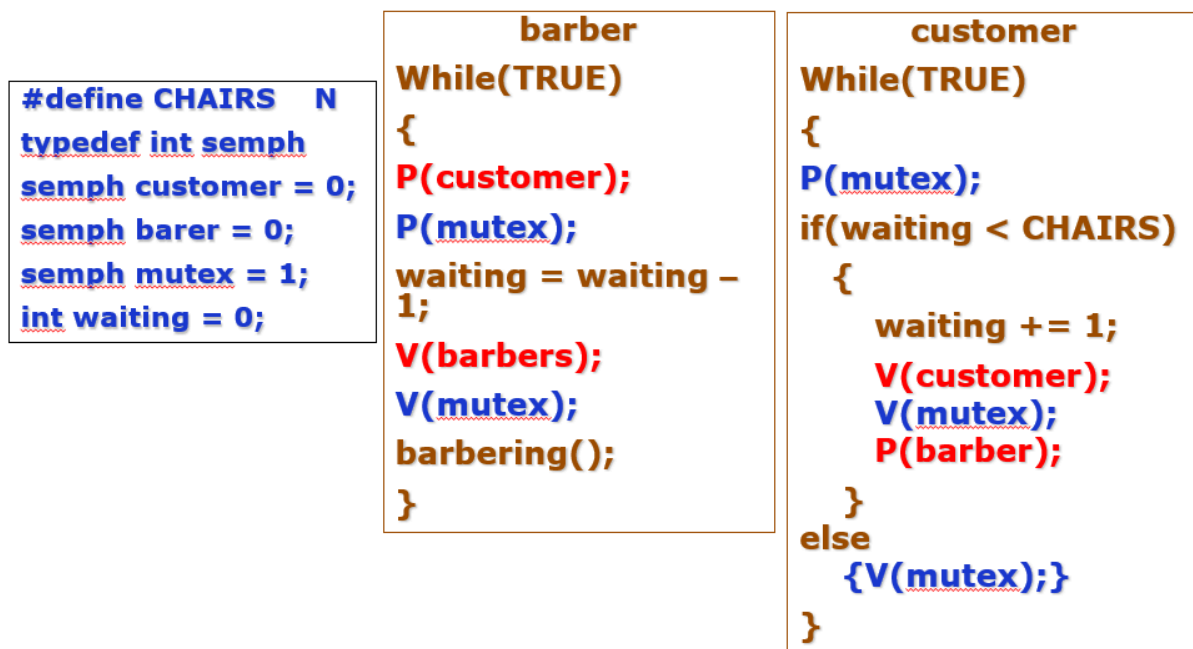


思考：

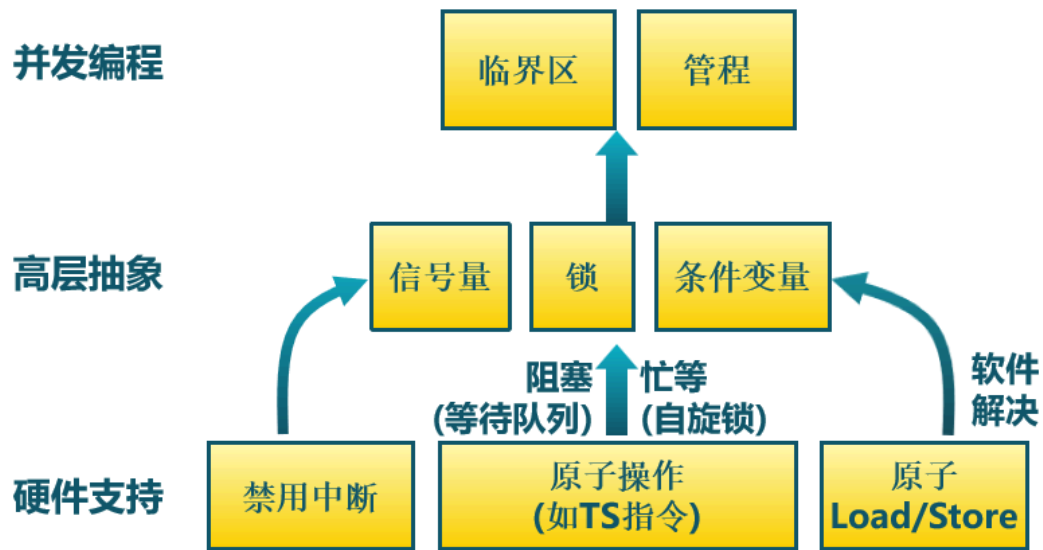
- 1.读者连续到来，则后续读者不必重新申请锁WriteMutex，就能连续读取
- 2.同理，写者连续到来时，写者也不必申请ReadMutex，就能连续写入
- 3.如果读者连绵不断而写者到来，写者能否打断读者并夺回控制权？

对于读者，我们只要在第一个人进去读的时候给写者加锁即可，当读者数量为0的时候，为写者解锁。对于写者，我们持有read的锁，然后可以去调用write来完成信息的写入，这样就能夺回控制权

Sleeping Barber——沉睡的理发师 跳过



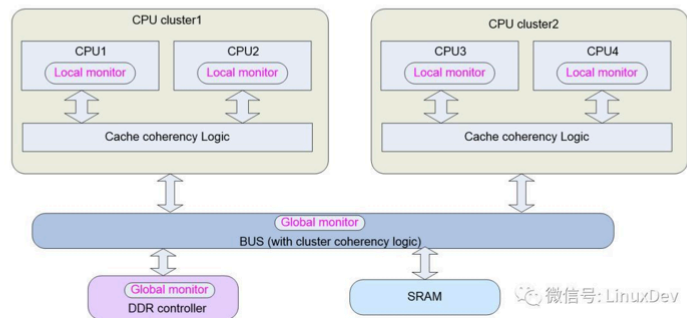
总结：



关于锁的机制：

关于锁机制的实现

- Test-and-Set是两个内存操作
- 如何在RISC指令中实现？
- Exclusive Monitor
- Load-Exclusive && Store-Conditional



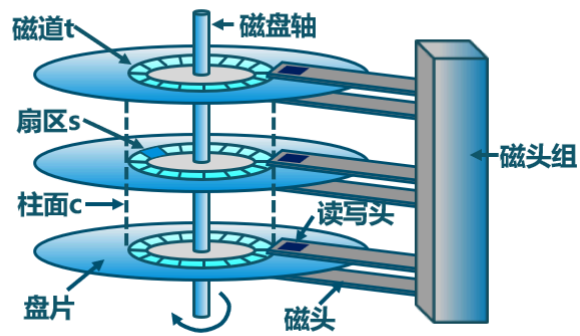
test and set需要对内存进行两次操作，所以需要两个指令

load和store两个标志位用于保护我们的操作

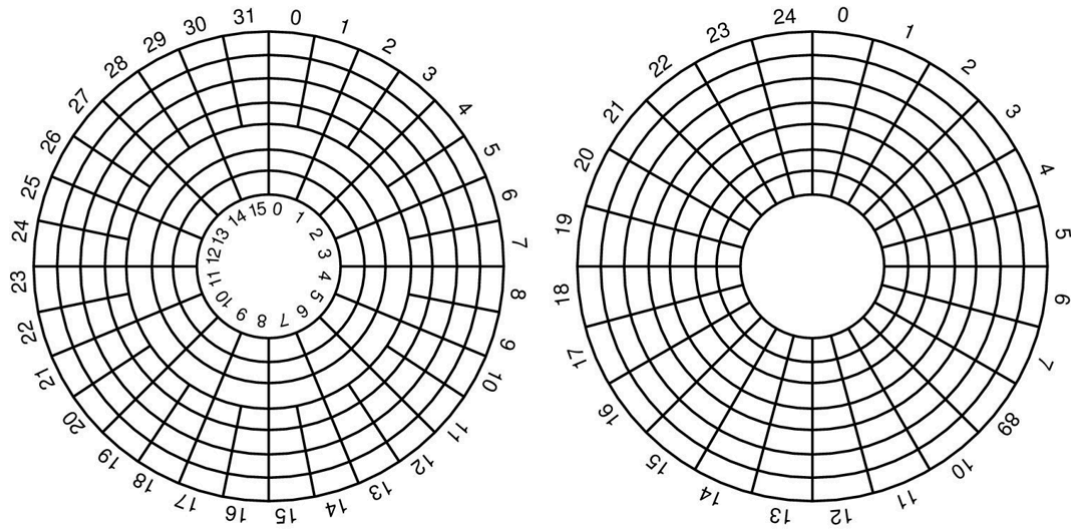
多核 可能会考？？ ——10月29日1:33:14

第五章 文件管理

磁盘：



磁道切开来，就是很多的扇区



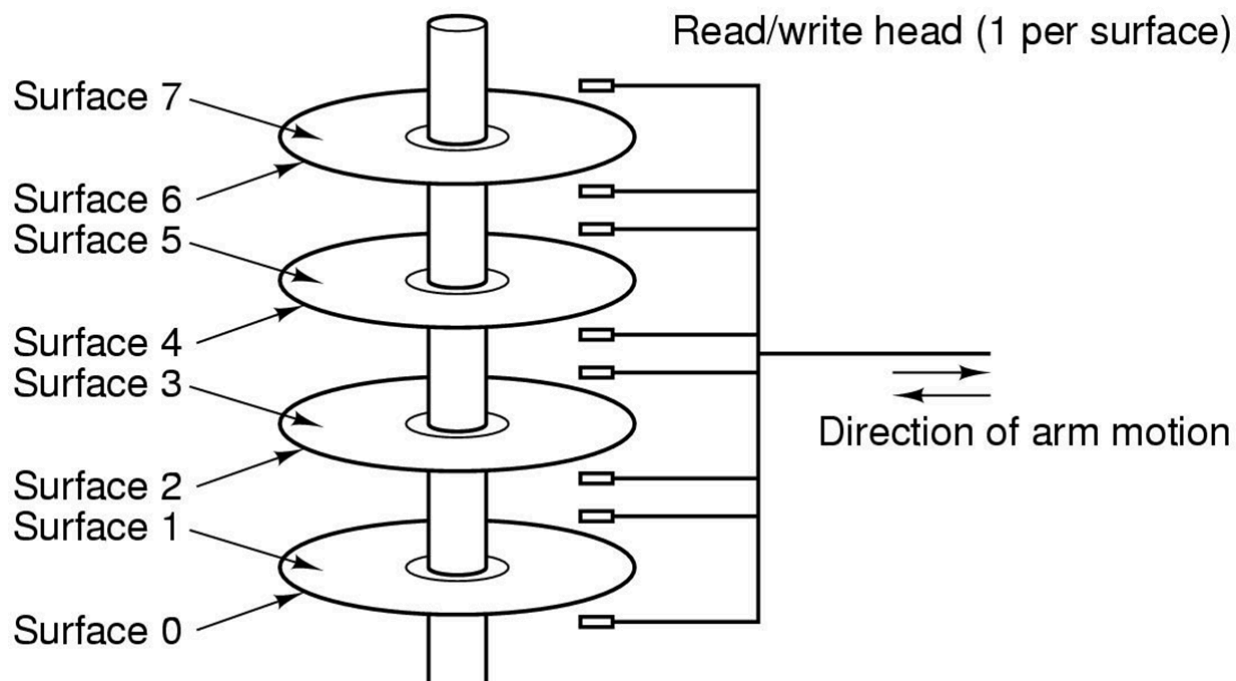
右边那个更好一些，匀速的滑动，在读取文件时不需要改变其转速

磁盘中的数据都是连续的，文件对应于磁带中的一段，需要记录它的长度和上次读取的位置，以可连续的读取

文件系统的作用，是把用户的文件（一段长度可以变化的连续数据）放到磁盘（磁头、磁道、柱面等单位管理的慢速旋转设备）上，并需要综合考虑空间利用率、读写性能、用户之间的隔离等问题

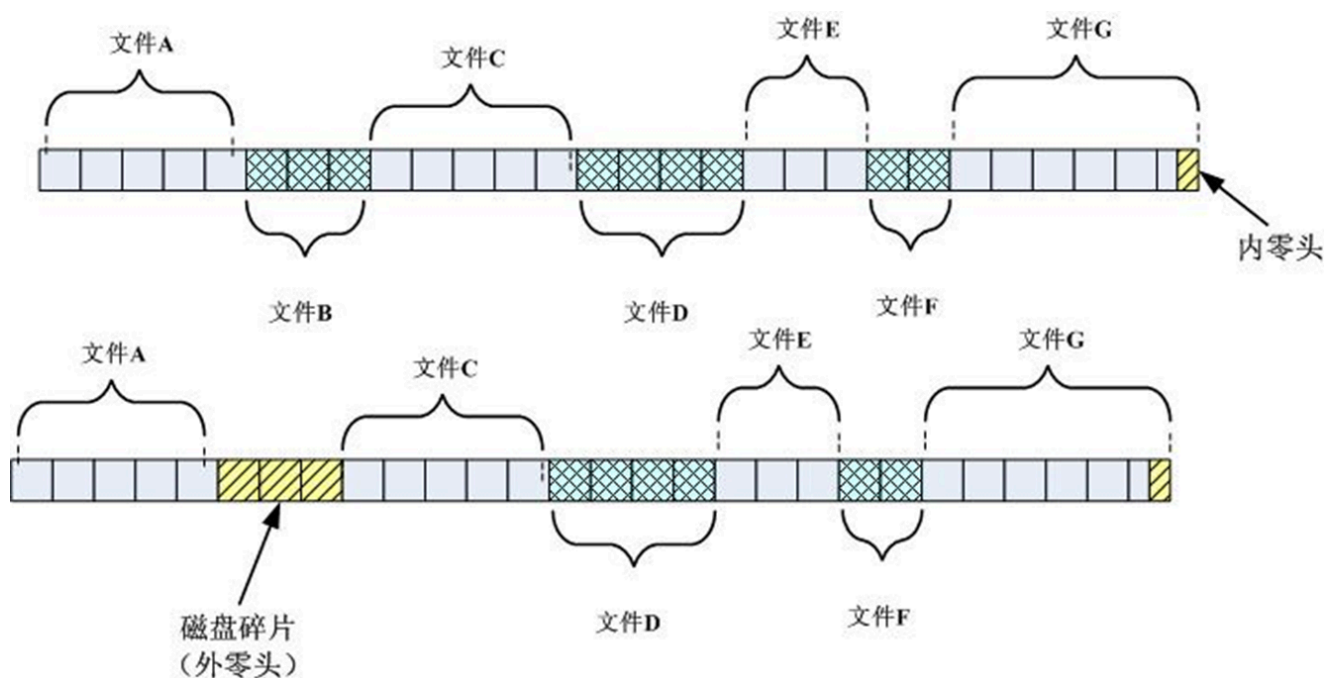
我们需要实现的是：实现文件的分块存储，也就是内外存的存储块相配合

一般的磁盘的读写方式：



我们在磁盘中，定位扇区需要靠旋转，所以我们的文件的存储连续性实际上十分影响我们的读写性能。

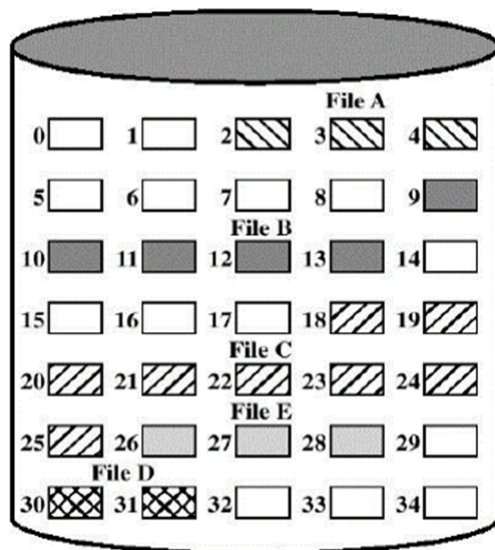
其中一种存储方式——直接按照顺序进行存储



但是这样的缺点就是，容易出现一堆磁盘的碎片，还有一些内零头，得不到我们的利用，就浪费了

具体的分配方式：

磁盘顺序的分配方式：



File Allocation Table		
File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

每个文件都分配一块连续的块，文件分配表中，每个文件都只占有一项，而且会出现外碎片

链式分配方式：

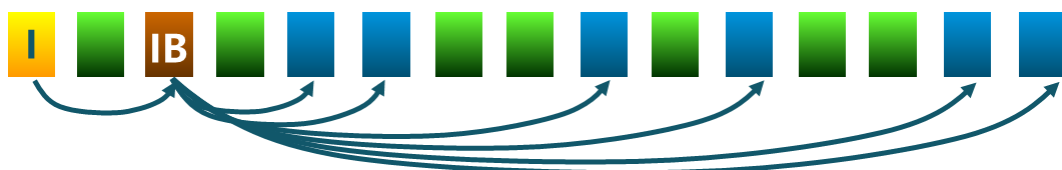
文件以数据块链表方式进行存储，文件头包含了到第一块和最后一块的指针



实现比较容易，不会出现碎片，但是无法实现真正的随机访问；可靠性比较差，破坏一个链，后面的数据就丢了

索引分配方式：

为每一个文件都创建一个索引数据块，指向文件数据块的指针列表



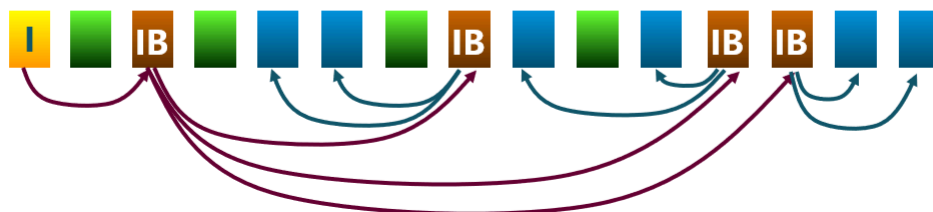
创建、增大和缩小十分容易，没有碎片，支持直接去访问文件

对于大文件来说，我们的索引分配有两个方法：

■ 链式索引块 (IB+IB+...)



■ 多级索引块 (IB*IB*...)



文件物理结构的总结：

		Continuous	Link table	Index table
media	tape	Supported	Unsupported	unsupported
	disk	Supported	supported	supported
Access mode		Sequential & random	sequential	Sequential & random
Efficiency		Low	Middle	High
Application		To simple to be used	Not popular	Widely used

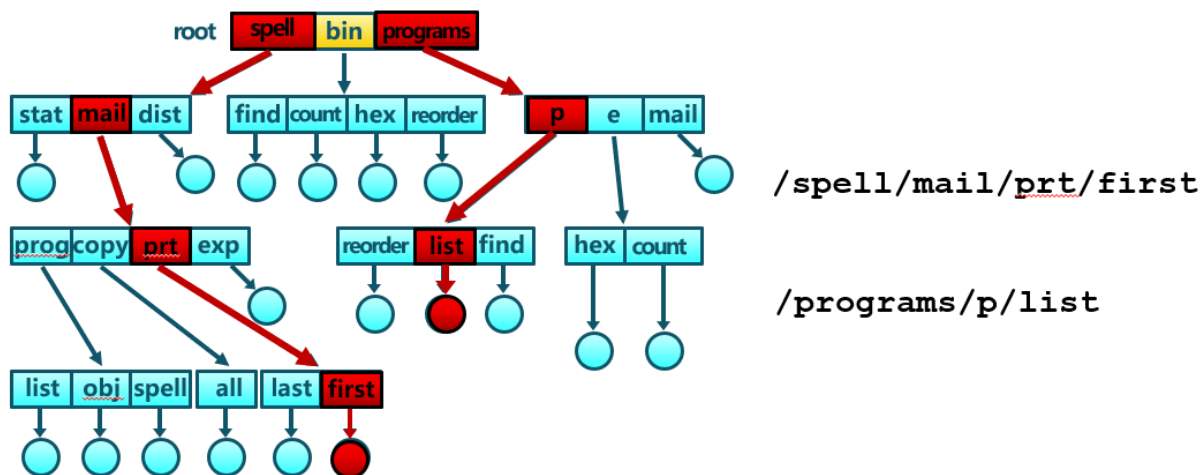
我们不需要去在内存中去删除前面的文件，如果需要替换的话，只需要在后面直接写就可以了

- 一块至少为64M，连续使用
- 数据总是追加，不必删除
- 当一个数据块追加超过限制的时候，把它复制到新的空闲位置上，并在后面预留一个新块

分层文件系统

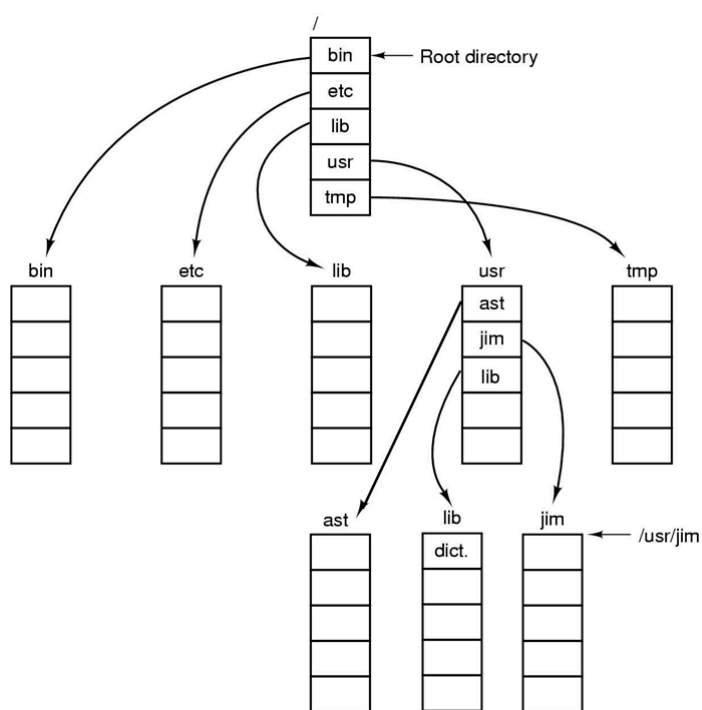
我们的文件是按照目录的方式组织起来的

目录的内容是文件索引表<文件名, 指向文件的指针>



我们的文件的表示形式：root节点是spell，指向mail，指向prt，指向first，所以这就是我们的文件名

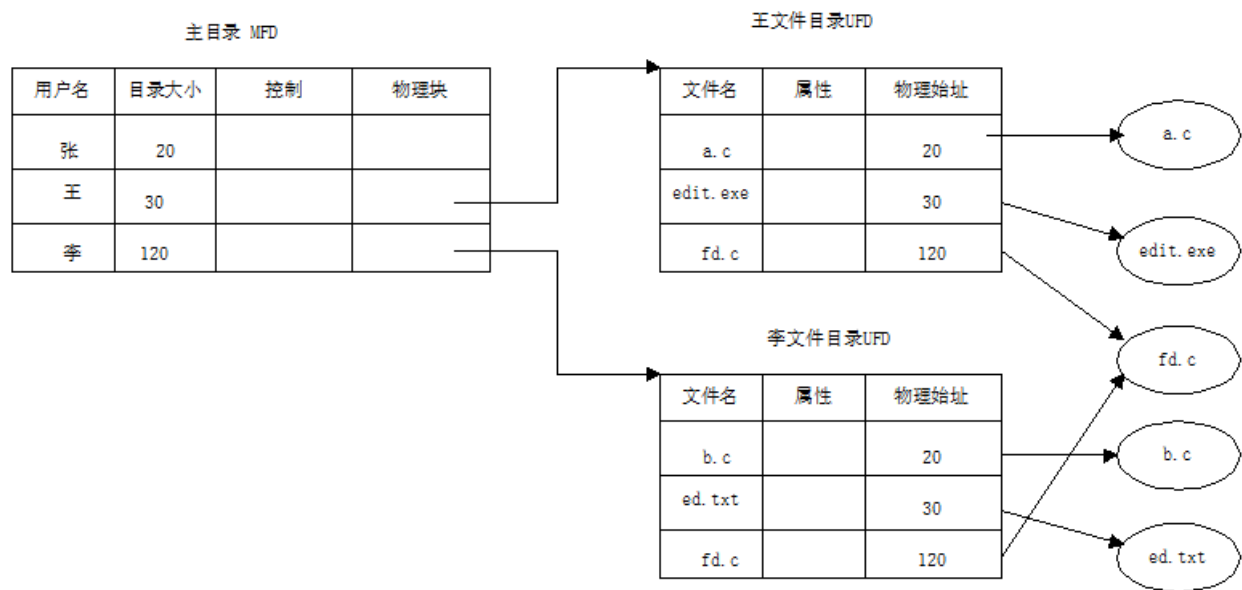
UNIX中的目录的结构：



我们通过路径名可清楚了解文件所处位置

文件夹：

- 目录(文件夹)是文件系统中一种特殊的文件
- 文件夹中存储了（放在该文件夹中的）文件名，以及文件在磁盘当中的起始地址。文件夹也是一种文件，其起始地址始放在上一级文件夹中。



属性中，记录我们的访问权限。

我们的文件名不存储在我们的存储数据的链表上，我们将其与对应文件内容的物理地址的开始位置存储在一起，这样我们读取到一个文件名的时候，就能得到我们该文件所在的链表头的位置

文件夹的名字在上一层的文件夹中存储，文件夹本身并不知道自己的名字

总结：

- 文件名和目录分层，是为了方便人类记忆的组织方
- 文件是扇区组织成的链表，或者是树
- 文件夹中记录着文件的名称，以及跟名字相对应的链表表头（或者树根）
- 文件夹也是一种文件，它也可能被存储为链表或者树
- 文件夹的名称，放在上一级文件夹当中，同时存储的是这个文件的表头或者树根

多级目录的结构：

- 目录名：指明目录结构中一个结点位置。
- 目录树：有根结点和中间结点（目录 / 子目录），叶子结点（文件）。
- 目录有上下级关系：当前目录 (current directory, working directory)、父目录 (parent directory)、子目录 (subdirectory)、根目录 (root directory) 等；
- 路径 (path)：每个目录或文件，可以由根目录开始依次经由的各级目录名，加上最终的目录名或文件名来表示（相对 / 绝对）。

对目录的操作：

- create：创建目录；
- delete：删除目录；
- opendir：打开目录使内容可读；
- closedir：关闭目录，释放各种表空间；
- readdir：返回打开目录的下一个目录项；
- rename：目录重命名；

- link: 对某一文件在不同目录中建立连接;
- Unlink: 删除目录中的连接文件。

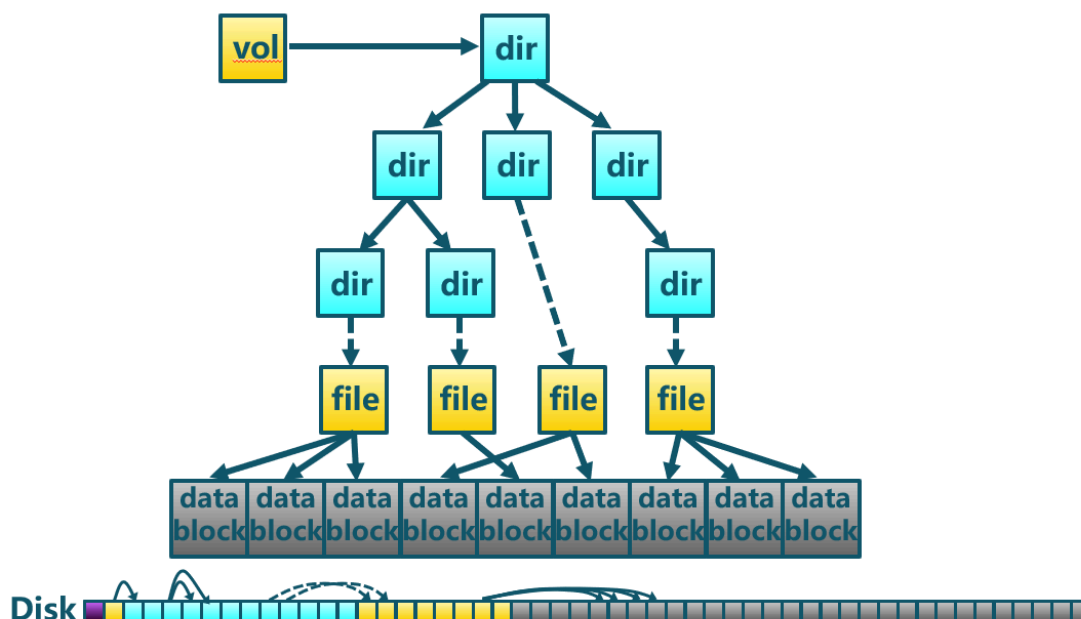
我们用于实现目录的数据结构——哈希表

其减少了目录的搜索时间，保证了两个文件的不重名，固定大小

在目录中处理长文件名的两种方法:

- 按字对齐处理: 文件名在文件项的尾部，并做字对齐处理。
- 在堆中处理: 所有文件名都放在一个文件名堆中处理。

文件系统的组织视图:



我们存储一个vol，也就是最先的那个黄色的块，通过它来指向我们的第一个根目录dir

最前面的紫色的块，就是写在我们的磁盘的第零个扇区的第零个位置，不能做修改，通过紫色的扇区指向第一个黄色的vol

总结 存储结构:

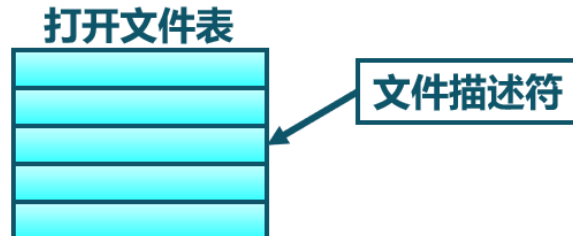
- **文件系统数据结构**
 - **卷控制块 (每个文件系统一个)**
 - **文件控制块 (每个文件一个)**
 - **目录节点(每个目录项一个)**
- **持久存储在外存中**
 - **存储设备的数据块中**
- **当需要时加载进内存**
 - **卷控制模块：当文件系统挂载时进入内存**
 - **文件控制块: 当文件被访问时进入内存**
 - **目录节点: 在遍历一个文件路径时进入内存**

打开文件和文件描述符：

- “打开”操作，是用户通知操作系统，为这个文件的读写做好准备，即找到文件名与文件位置的对应关系等
- 因为文件操作很慢，所以需要一些准备工作以“预热”，例如找到文件开始的磁道扇区，文件的长度等
- 因为文件很多，所以不能为每一个文件都“预热”

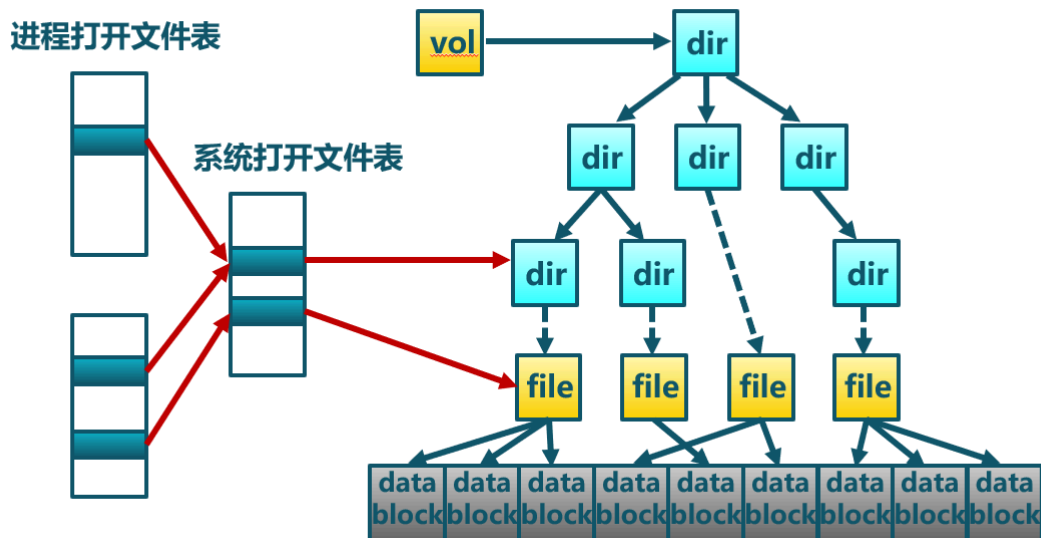
内核跟踪进程打开的所有文件

- 操作系统为每个进程维护一个打开文件表
- 文件描述符是打开文件的标识



操作系统在打开文件表中维护的打开文件状态和信息

- 文件指针
 - 最近一次读写位置
 - 每个进程为每个文件分别维护一个自己的打开文件指针
 - 文件指针仅对已打开的文件维护，是因为文件的数量可能非常庞大
 - OS可能设置一个进程的打开文件上限，也是为了节省文件指针的内存消耗
- 文件打开计数
 - 当前打开文件的次数
 - 最后一个进程关闭文件时，将其从打开文件表中移除
- 文件的磁盘位置
 - 缓存数据访问信息
- 访问权限
 - 每个进程的文件访问模式信息



通过系统打开文件表，来访问我们的文件file，是通过路径去找到对应的文件的，file再指向data block，也就是我们的数据块

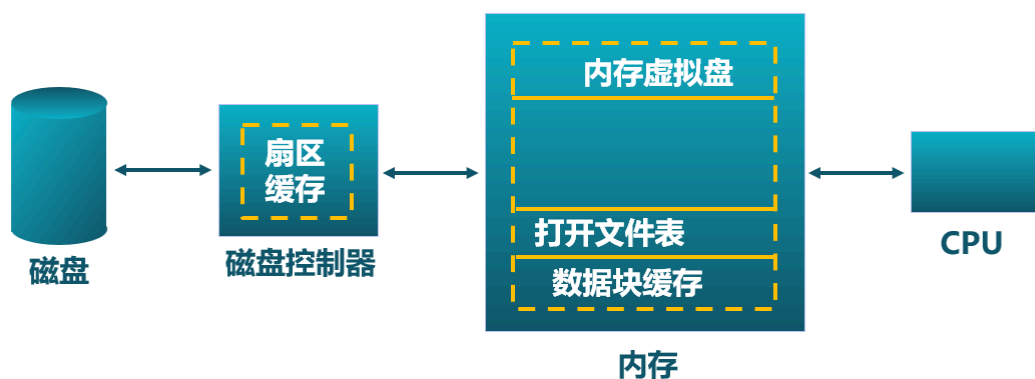
打开文件锁：

- 强制 – 根据锁保持情况和访问需求确定是否拒绝访问
- 劝告 – 进程可以查找锁的状态来决定怎么做

文件被打开后，多个进程可以对同一个文件读写，多个线程也可以因为读取而并发修改内存中的文件指针

但是，因为考虑到文件操作过慢，OS在大多数情况下并没有刻意保障其操作的原子性，而是将这些任务交给了程序员来完成

为了提高文件读取的性能，我们设置了对应的缓存区



磁盘将自己的数据块缓存到了内存中，然后CPU就可以直接调用这一块数据了，不需要再去文件中进行读取了

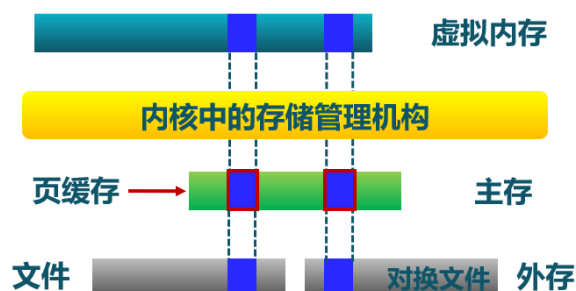
数据块的缓存：

如果数据块被使用了，那么他就会被缓存下来，假设我们的数据块在后续还会被用到

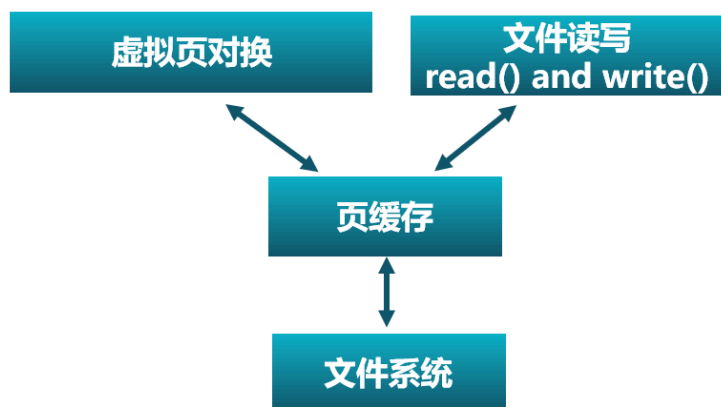
数据块缓存可以只缓存数据块，也可以进行页缓存——统一缓存数据块和内存页

页缓存：

- 虚拟页式存储
 - ▣ 在虚拟地址空间中虚拟页面可映射到本地外存文件中
- 文件数据块的页缓存
 - ▣ 在虚拟内存中文件数据块被映射成页
 - ▣ 文件的读/写操作被转换成对内存的访问
 - ▣ 可能导致缺页和/或设置为脏页
 - ▣ 问题: 页置换算法需要协调虚拟存储和页缓存间的页面数



页缓存的机制：



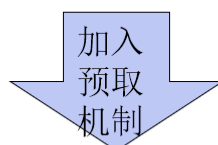
细节：

1. read/write并没有固定的大小，合并之后，统一采用4K对齐
2. 虚拟页没有清晰的读写标记而read/write可以清楚的知道是否命中

我们采用4K作为我们的页缓存的大小

我们还可以使用预先读取页的缓存机制，来帮我们提升数据处理的效率

数据读取：	1===	2====	3===	4====	5=====
数据处理：	1-----	2--	3---	4----	5-----
运行时间：	----->				



数据读取：	1===2====3===4====5=====
数据处理：	1-----2--3---4----5-----
运行时间：	----->

我们在开始的时候直接把1-5的数据全部读进来，然后在我们处理后面的数据的时候，就不用去读取对应的数据了，可以直接处理难分的额数据，这样可以大大缩短我们的运行时间

当前的linux预测方案：

现行的Linux Readahead方案

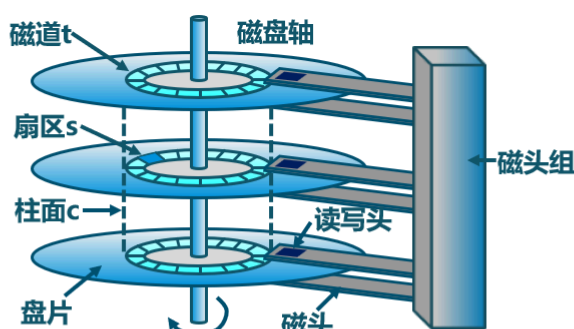
- 同步预取获得用户请求的内容，异步预取获取未来的信息
- 预取来的数据存在内核空间中，在用户请求时直接使用，并且可以检测预测是否准确
- 准确预测会得到奖励，使得下次预测更激进，窗口变大一倍
- 预测错误会使得预取窗口复位



详见
<http://blog.chinaunix.net/uid-667478-id-2384354.html>

就是说，我们先帮他提前取4个页面，如果用户下次访问，访问到了其中的一个页，那么说明我们预测对了！这激励了我们的预测器，预测器会在下一次预测中去预先读取8个页面（也就是翻倍），直到有一次没有读取到对应的页面的时候，我们就会将预先读取的页面降为4，然后重新开始我们的翻倍的过程

磁盘的工作机制和性能参数：



- 读取或写入时，磁头必须被定位在**期望的磁道**，并从**所期望的柱面和扇区**的开始
 - 寻道时间
 - ▣ 定位到期望的磁道所花费的时间
 - 旋转延迟
 - ▣ 从零扇区开始处到达目的地花费的时间
- 平均旋转延迟时间=磁盘旋转一周时间的一半**



具体的计算公式：

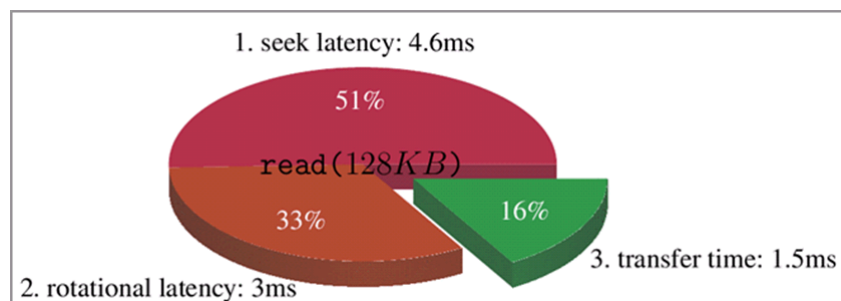
$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

T_a 是我们的总时间，也就是我们的访问时间

T_s 是我们的寻道时间，也就是我们定位到所期望的磁道所花费的时间

$\frac{1}{2r}$ 是我们的旋转延时的时间，我们一般只需要旋转半周就可以了

$\frac{b}{rN}$ 是我们的传输时间，其中 b 是传输的比特数； r 是我们的磁盘的转数； N 是我们磁道上的比特数



实际上，我们不能去优化我们的传输时间和旋转延时的时间，所以我们只能从我们的寻道时间上进行优化。

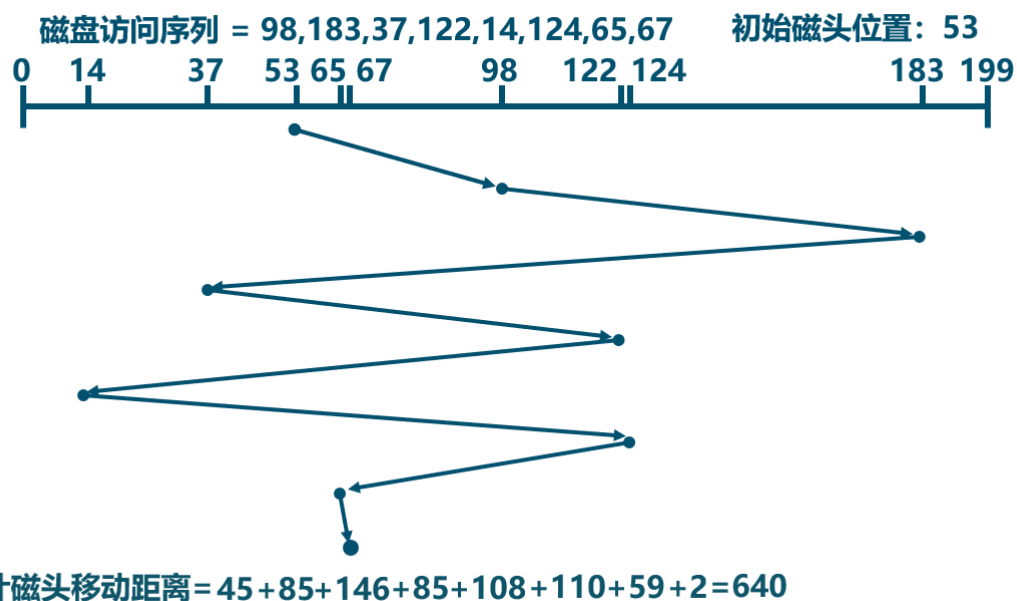
我们有很多的磁盘调度的算法，通过优化磁盘访问请求顺序来提高磁盘访问性能。

- 寻道时间是磁盘访问最耗时的部分
- 同时会有多个在同一磁盘上的I/O请求
- 随机处理磁盘访问请求的性能表现很差

磁盘调度算法：

1.先进先出(FIFO)算法

按照对应的顺序来处理请求，公平的对待所有的进程

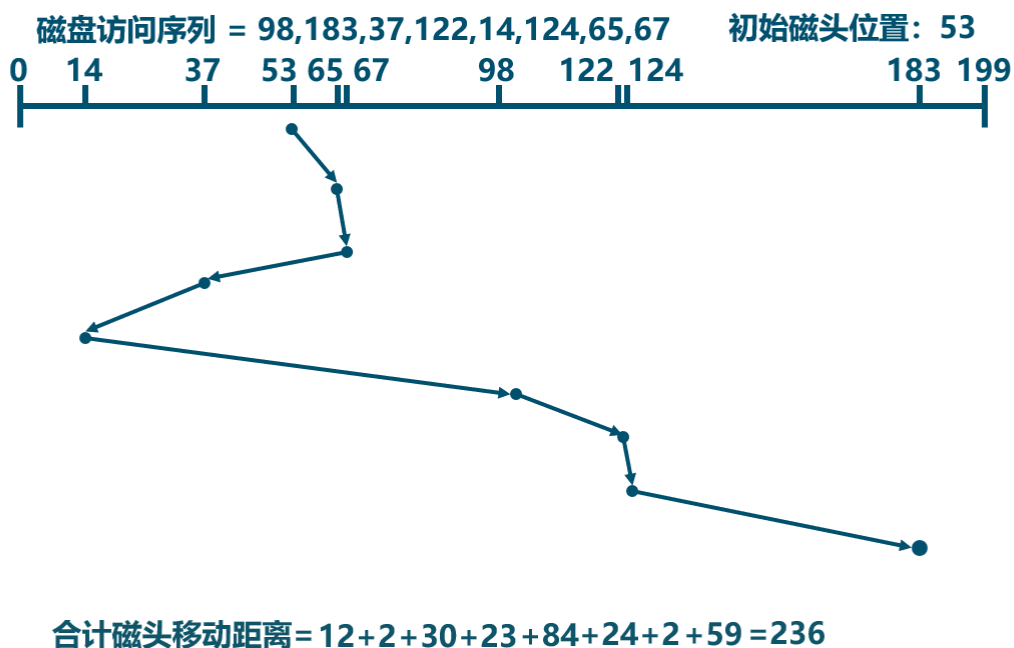


比如上面的图。我们直接按照磁盘访问序列的顺序来完成我们磁盘的移动，这样的话，接近我们的随机访问，而且合计的移动距离是很大的

2.最短服务时间优先(SSTF)

简单来说就是，选择从磁臂当前位置需要移动最少的I/O请求，总是选择最短寻道时间。

举个例子：

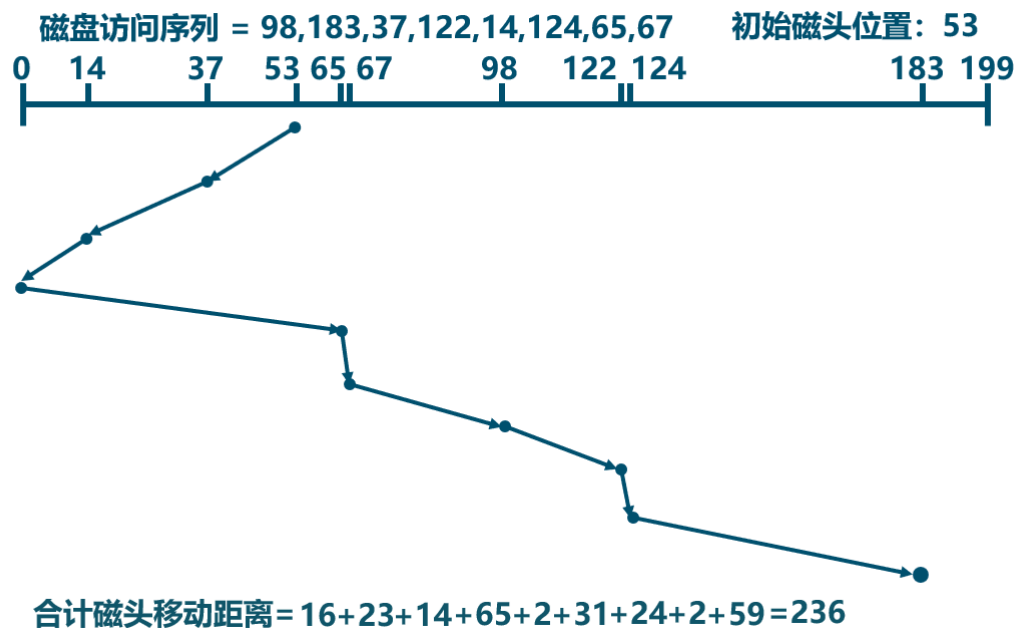


比如上图，从53开始，先找最近的，也就是65，然后再找67，这样一直下去，每一次都找最近的那一个磁盘的请求，这样得到的合计距离，理论上是最短的

3.扫描算法(SCAN)——电梯算法

磁臂在一个方向上移动，访问所有未完成的请求，直到磁臂到达该方向上最后的磁道

然后访问完一个方向的之后，再调换方向，反方向将我们的另一边的请求都访问完



4. 循环扫描算法(C-SCAN)

限制了仅在一个方向上扫描，当最后一个磁道也被访问过了后，磁臂返回到磁盘的另外一端再次进行

5. C-LOOK算法

磁臂先到达该方向上最后一个请求处，然后立即反转，而不是先到最后点路径上的所有请求

6. N步扫描(N-step-SCAN)算法

- 磁头粘着(Arm Stickiness)现象
 - ▣ SSTF、SCAN及CSCAN等算法中，可能出现磁头停留在某处不动的情况
 - ▣ 如：进程反复请求对某一磁道的I/O操作
- N步扫描算法
 - ▣ 将磁盘请求队列分成长度为N的子队列
 - ▣ 按FIFO算法依次处理所有子队列
 - ▣ 扫描算法处理每个队列

7. 双队列扫描(FSCAN)算法

- FSCAN算法是N步扫描算法的简化
 - ▣ FSCAN只将磁盘请求队列分成两个子队列
- FSCAN算法
 - ▣ 把磁盘I/O请求分成两个队列
 - ▣ 交替使用扫描算法处理一个队列
 - ▣ 新生成的磁盘I/O请求放入另一队列中
 - 所有的新请求都将被推迟到下一次扫描时处理

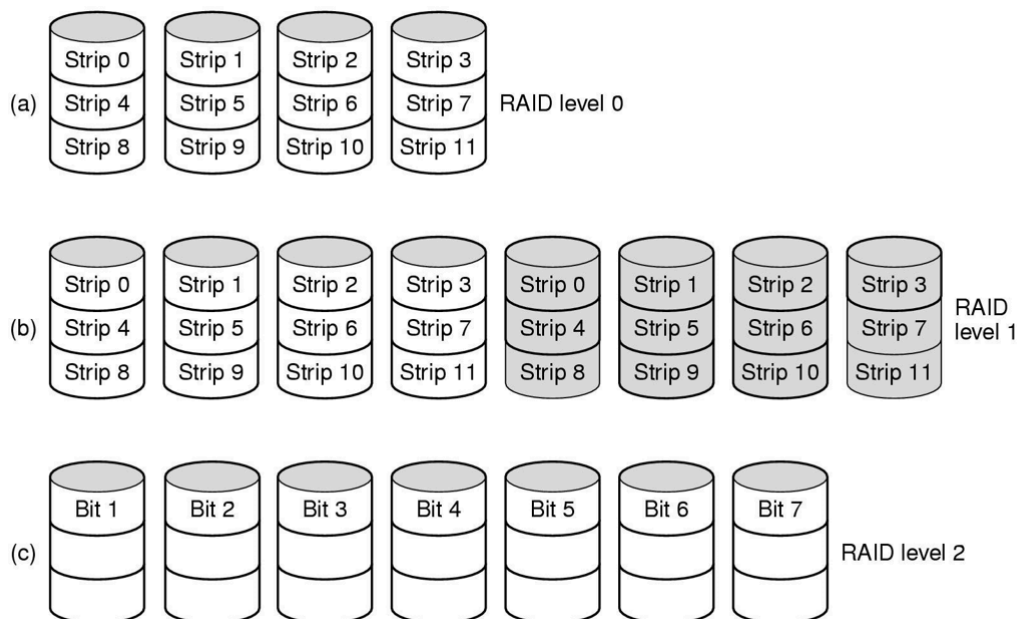
对于RAID(Redundant Array of Inexpensive Disks)的研究：

通过并行的方法来提升我们的磁盘访问性能，但是缺点就是对应的成本大大提升了！

我们只需要将磁盘的数量变多，然后将文件进行分块存储即可，这样我们在访问的时候，就可以大大提升我们的带宽

当然，我们还可以用**冗余的磁盘空间保存奇偶检验信息**，使数据具有可恢复性。

RAID 0-2层的结构：

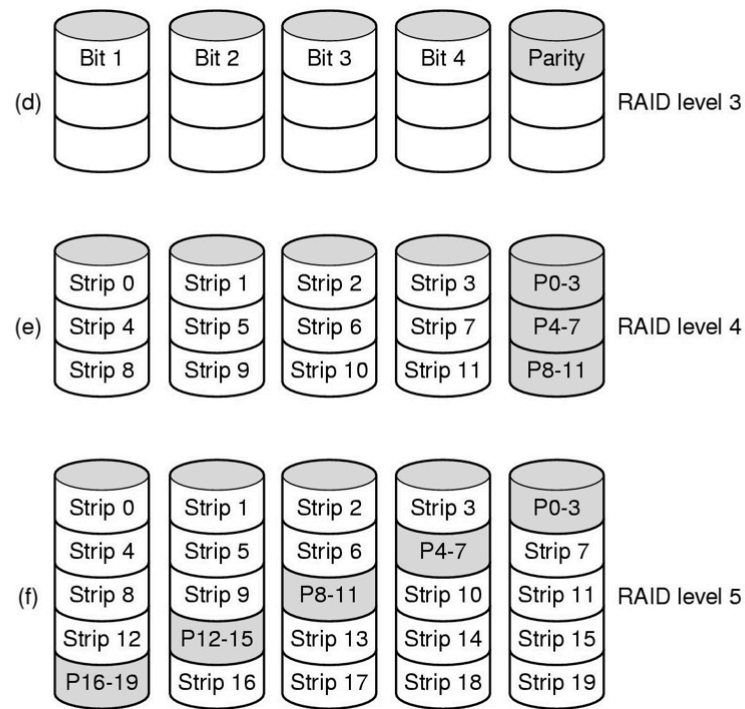


0层的结构就是相当于，直接将磁盘的数量增大了，这样我们可以分块的去访问对应的信息

1层的结构就是在0层的基础上又翻倍了，这样在我们请求0和4的时候，不必都在第一个048上等，可以用第一个和第五个。这样也可以提升一些性能，就是磁盘数量增大之后会很贵。

2层的结构就更离谱了，直接将文件中的比特位拆出来，分别用磁盘去存储，这样也是不太恰当的。因为我们将数据拆成比特位的时候，就会出现一些错误

RAID 3-5层的结构：



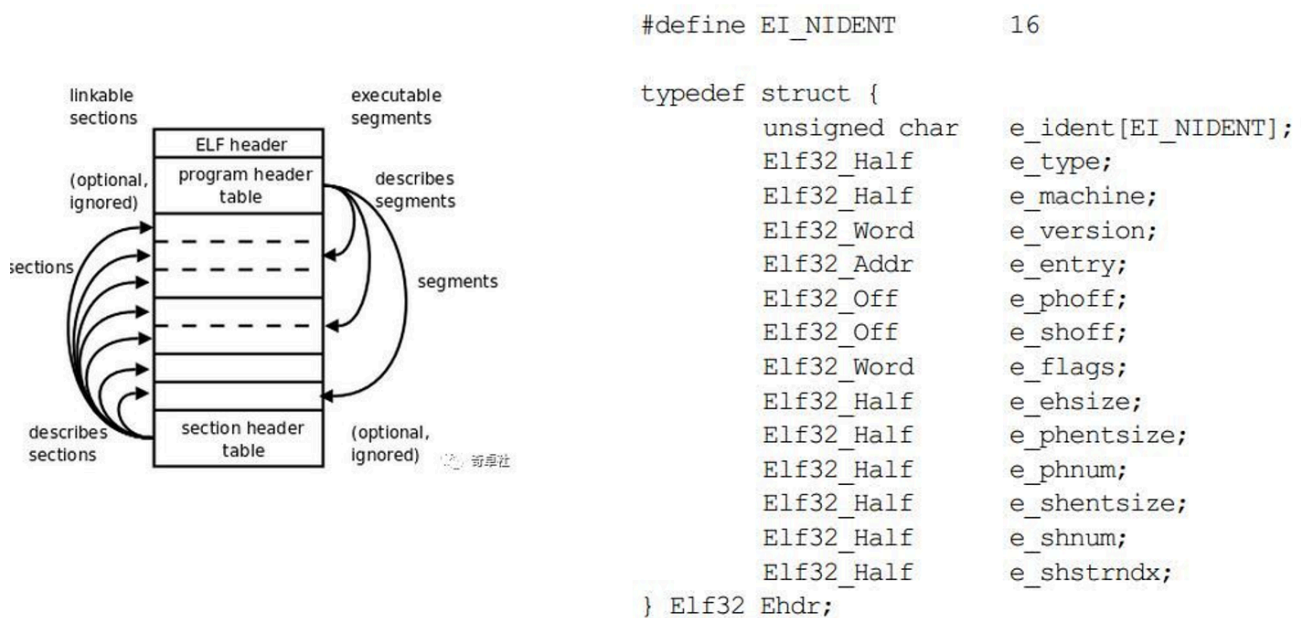
3层的结构就是，我们在前面的基础上添加了一个冗余的检验位，来保证我们前面的数据都是正确的，这样也是可以的，但是就是多使用了一个磁盘来保证我们的正确性

4层的结构就是把3层的比特位改成了前面的文件分块，同时增加了冗余的磁盘来确保我们的数据的正确性，但是有个缺点就是，检验的工作都需要让第五个磁盘来完成

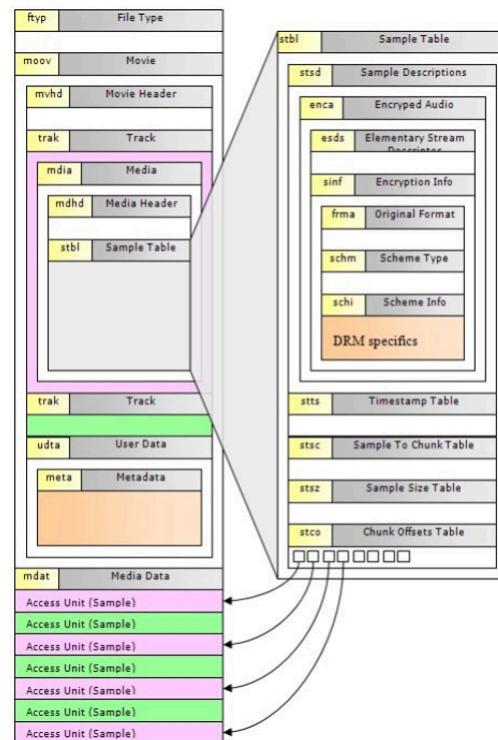
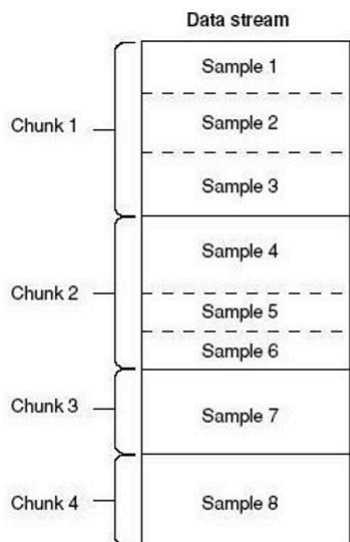
5层的结构相比于4层，有提升，将冗余位分开存放了，也就是五个磁盘各自都会承担一部分的检验工作。

用户读写磁盘的方式：通过执行程序，播放电影，读写数据库等等

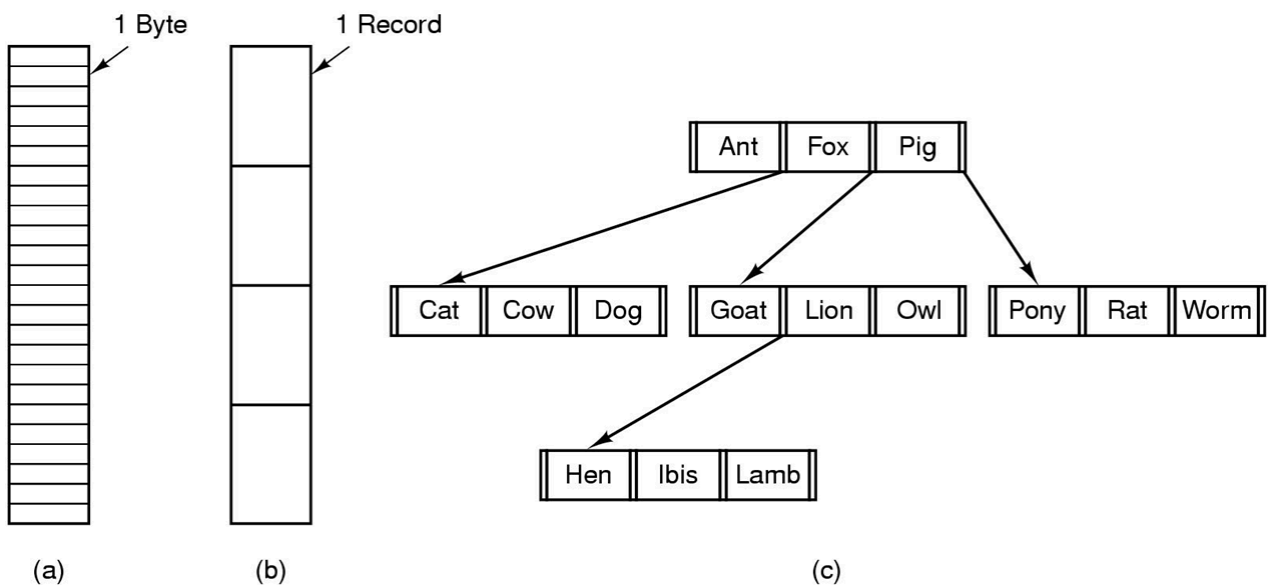
linux可执行文件的格式：



有关于媒体文件的格式：



文件的逻辑结构：



这棵树叫字典树，加速查找

在文件系统中，我们有一些特殊的问题：

关于磁盘的空间管理和容错；文件的去重；多文件的系统兼容

我们的磁盘分配策略：

- 文件系统的数据块越大，单次传输时的磁盘控制动作就越简单，传输效率越高
- 但数据本身并不可能总是能够与数据块对齐，因此存储效率就越低

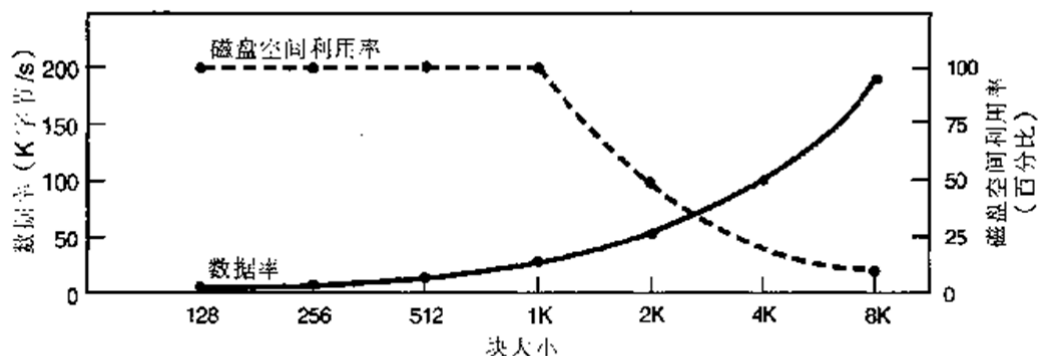
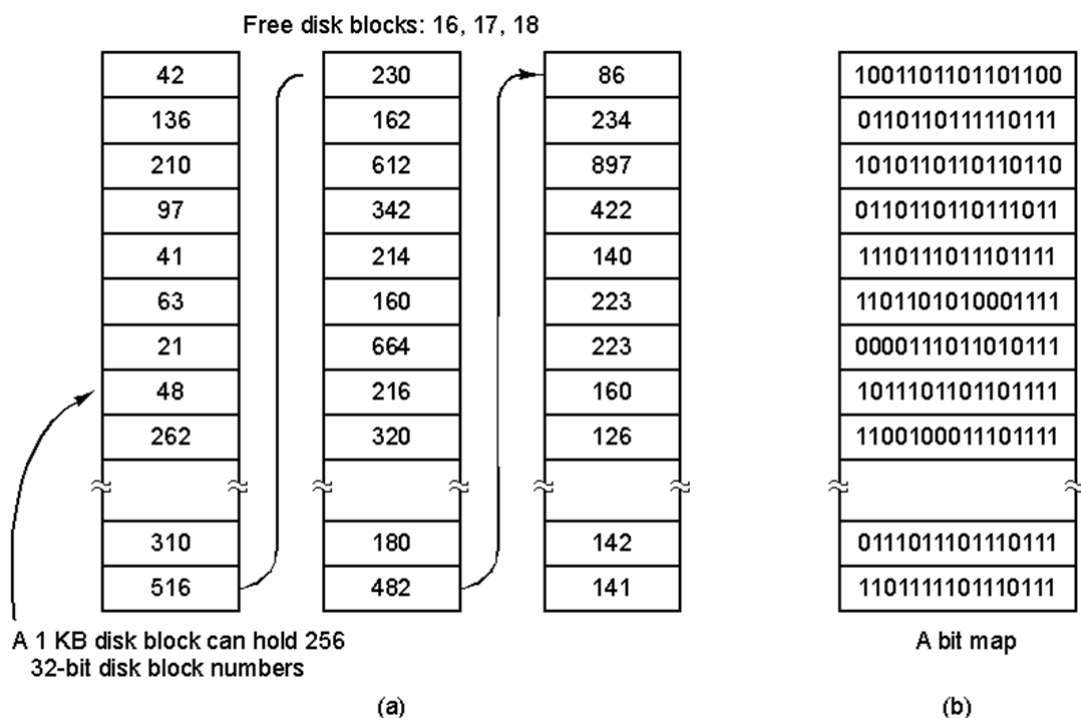
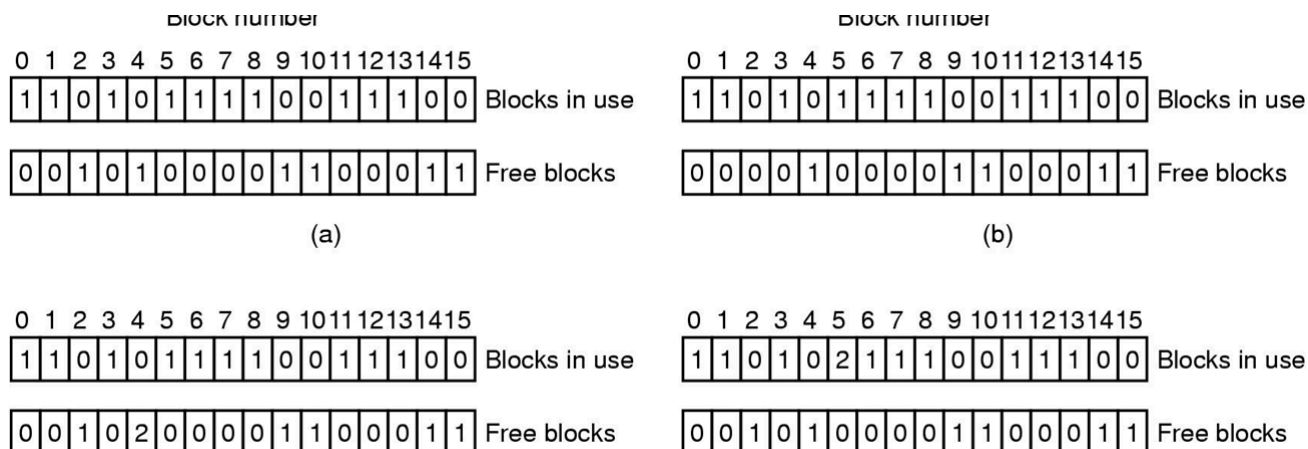


图 5-15 实线(左边标度)给出磁盘数据率,虚线(右边标度)给出磁盘空间效率,所有文件均为 1K

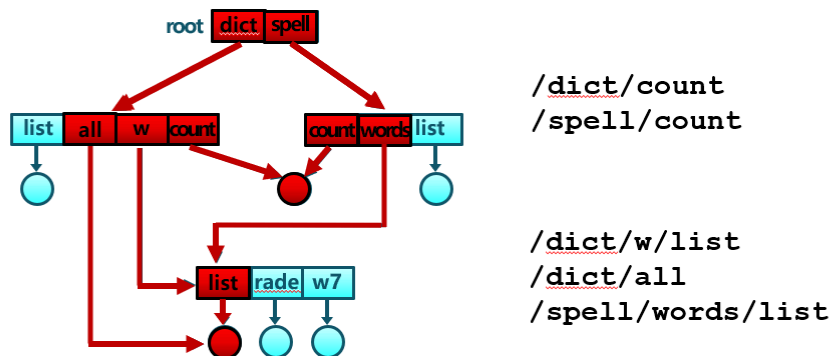
我们怎么管理空闲的数据块?——和内存管理是类似的,我们也可以使用位图的方法来存储我们的空闲数据块,当然也可以使用链表来存储我们的空闲数据块



对于文件系统的一致性检查——我们可以通过检查磁盘块中的0和1, 来判断我们的磁盘有没有损坏。



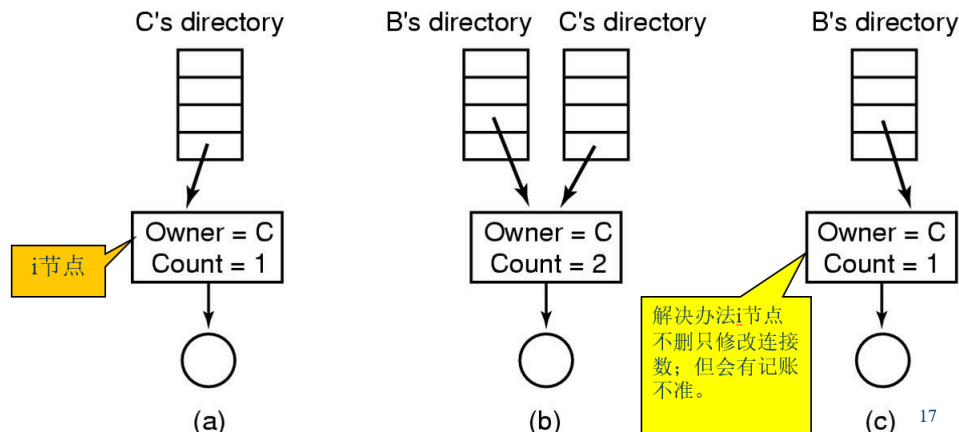
当文件系统中用链接实现共享时，系统目录树结构会发生变化：



- **硬链接:** 多个文件夹中存储指向同一个文件的“链表头”（树根）
- **软链接:** 以“快捷方式”指向其他文件，快捷方式也是一个文件——通过存储真实文件的逻辑名称来实现

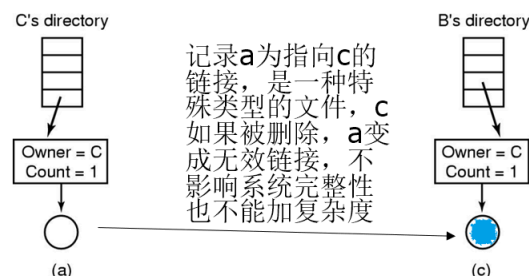
用链接实现文件共享控制 (i节点方式)

- 每个文件有指向自己i节点的指针;
- 创建共享时多指针指向i节点并增加引用数;
- 当属主文件被删除并连带i节点删除时会出错。

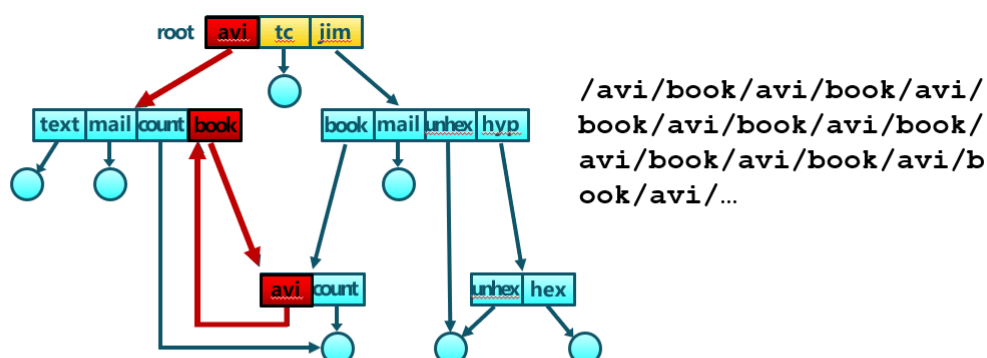


采用符号链接可较好解决问题

- 只有文件属主有指向i节点的指针
- 建立一个新文件类型link, 用做目录管理
- 有共享需求的用户只了解找到共享文件的目录, 而无指向i节点的指针
- 删除符号链接不会影响原文件
- 若属主删除被链接的文件, 只能带来链接文件找不到正确目录文件, 无大错



在我们的文件系统中, 会出现循环的问题:



- 如何保证没有循环?
 - 只允许到文件的链接, 不允许在子目录的链接
 - 增加链接时, 用循环检测算法确定是否合理
- 更多实践
 - 限制路径可遍历文件目录的数量

文件共享还需要访问类型和权限的支持

文件访问类型：

- 读read：可读出文件内容；
- 写write（修改update或添加append）：可把数据写入文件；
- 执行execute：可由系统读出文件内容，作为代码执行；
- 删除delete：可删除文件；
- 修改访问权限change protection：修改文件属主或访问权限

我们还可以建立权限矩阵——类似数据库

管理用户类型范围：

- 指定用户
- 用户组
- 任意用户

访问类型和用户范围建立有机组合：

- 建立访问矩阵：矩阵中一维是包含的目录和文件，另一维是用户，每个元素是允许的访问方式

	wang	liu	zhang	zhao
a.c	RWE	RE	E	R
b.c	RE	E	RE	RWE
d.c	R	R	E	E

有共享访问时查询该矩阵

虚拟文件系统——为了解决多种文件系统识别问题

目的：对所有不同的文件系统的抽象

功能：

- 提供相同的文件和文件系统接口
- 管理所有文件和文件系统关联的数据结构
- 高效查询例程, 遍历文件系统
- 与特定文件系统模块的交互

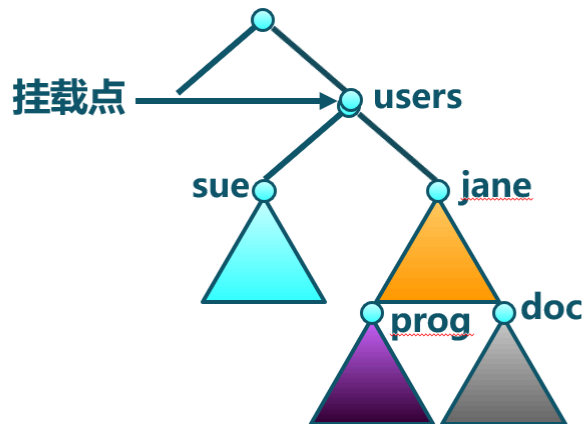
文件系统的基本数据结构：

- 文件卷控制块 (Unix: “superblock”)
 - 每个文件系统一个
 - 文件系统详细信息
 - 块、块大小、空余块、计数/指针等
- 文件控制块(Unix: “vnode” or “inode”)
 - 每个文件一个
 - 文件详细信息
 - 访问权限、拥有者、大小、数据块位置等

- 目录项 (Linux: “dentry”)
 - 每个目录项一个(目录和文件)
 - 将目录项数据结构及树型布局编码成树型数据结构
 - 指向文件控制块、父目录、子目录等

文件系统挂载：

- 文件系统需要先挂载才能被访问
- 未挂载的文件系统被挂载在挂载点上

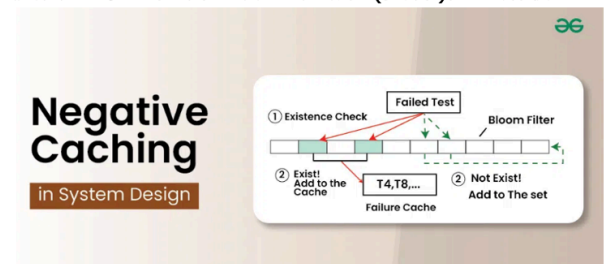


未挂载的文件系统被挂载到了挂载点上

VFS功能举例：名字解析（路径遍历）

- 名字解析: 把逻辑名字转换成物理资源（如文件）
 - 依据路径名，在文件系统中找到实际文件位置
 - 遍历文件目录直到找到目标文件
- 举例: 解析 “/bin/ls”
 - 读取根目录的文件头 (在磁盘固定位置)
 - 读取根目录的数据块，搜索 “bin” 项
 - 读取bin的文件头
 - 读取bin的数据块; 搜索 “ls” 项
 - 读取ls的文件头
- 当前工作目录 (PWD)
 - 每个进程都会指向一个文件目录用于解析文件名
 - 允许用户指定相对路径来代替绝对路径
如，用 PWD= “/bin” 能够解析 “ls”

反向思考：如何证明一个路径(文件)无法解析？



设计细节：

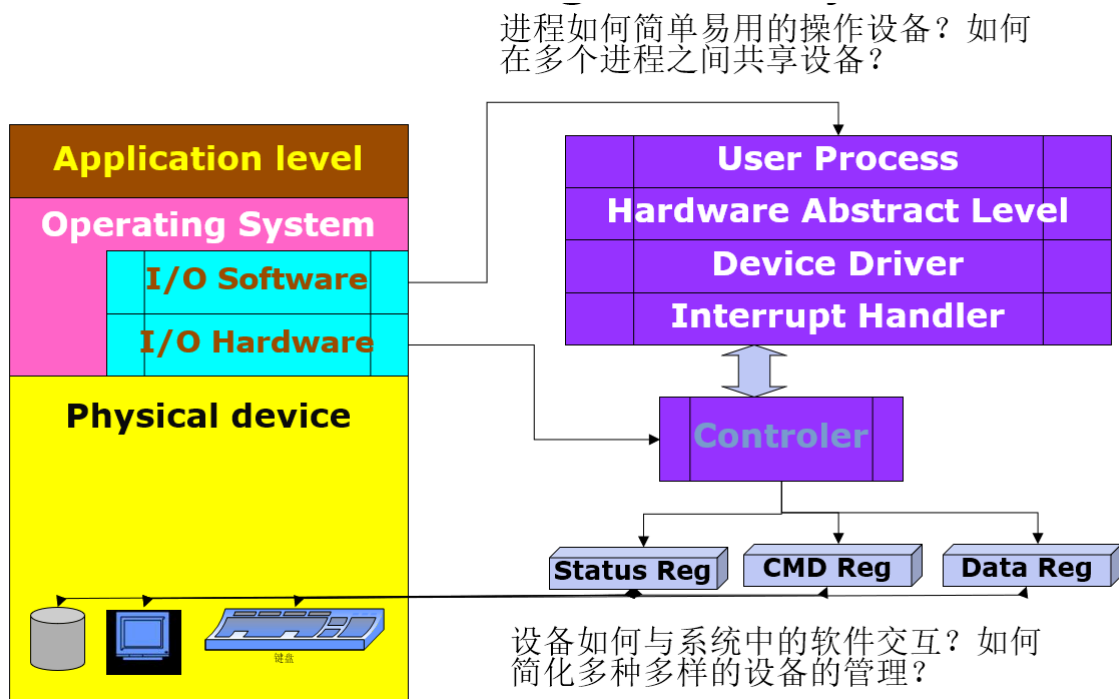
1. 负缓存，要存多少项呢？
2. 负缓存怎么跟文件操作同步？
3. 负缓存什么时候失效？

证明一个文件不能被解析，我们需要去找到所有的文件都不符合他，才能证明，所以这个很难实现。

于是我们出现了负缓存，也就是将那些确认不能解析的文件名直接加入到缓存中，这样下一次请求时就可以直接说找不到了

第六章 设备管理

设备和前面的一些内存和文件的管理方式不相同，设备的种类太多了，导致我们不可能为了每一种设备去设计对应的处理方式，不能共享

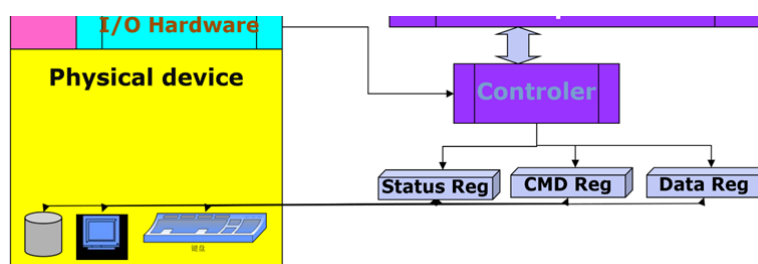


为了分别处理不同类型的设备，我们把一些设备进行了分类：

- 按照功能来分，分为存储型的设备和输入输出的设备
- 按照数据管理方法来分，分为字节流和数据块流
- 按照设备完成的任务来分，分为独占的设备——低速的IO设备，共享设备——高速的IO设备，虚拟设备——用软件模拟硬件
- 按照工作模式来分，分为逻辑设备和物理设备

对于设备的管理，我们应该尽量减少为不同的硬件做出的修改，还要具有一定的灵活性，能够支持新的一些设备

- 我们的设备需要完成的就是，将我们的外部的机械变化转化为电信号，然后再转化为计算机可以识别的数字信号。
- 设备完成上述过程后，将所得到的数据，放于自自的寄存器、数据buffer等结构中，供CPU读取，并准备好控制接口，供CPU发布控制命令



我们的CPU怎么读写设备提供的接口呢？

我们使用IO端口，也就是输入输出端口来进行读取即可

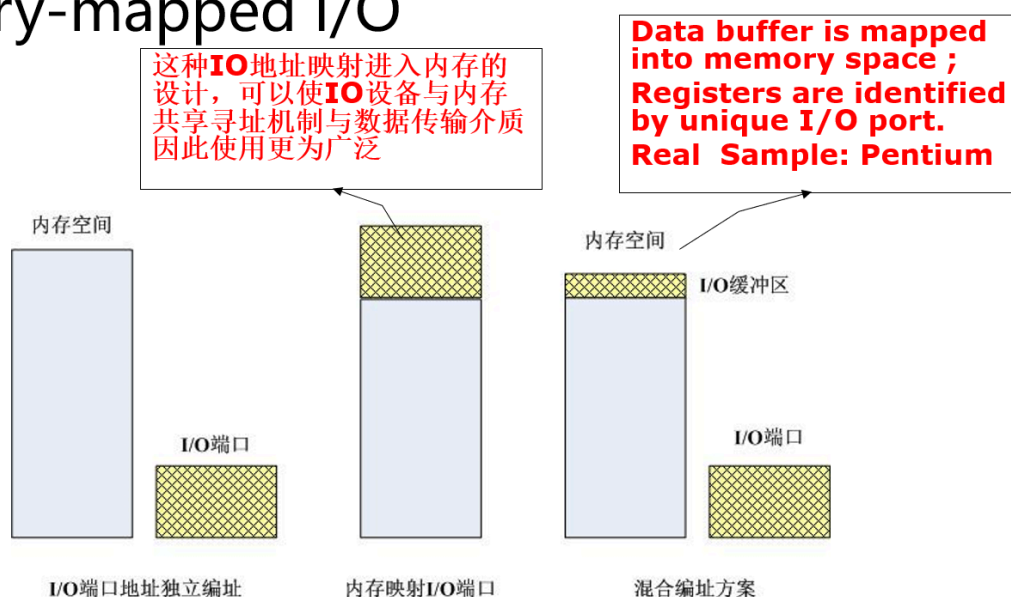
我们有一张对应的表：

I/O控制器	I/O地址	中断向量
时钟	040-043	8
键盘	060-063	9
辅助RS-232	2F8-2FF	11
硬盘	320-32F	13
打印机	378-37F	15
单色显示	380-3BF	-
彩色显示	3D0-3DF	-
软盘	3F0-3F7	14
主RS-232	3F8-3FF	12

PC上的I/O控制器及其对应的I/O地址和中断向量

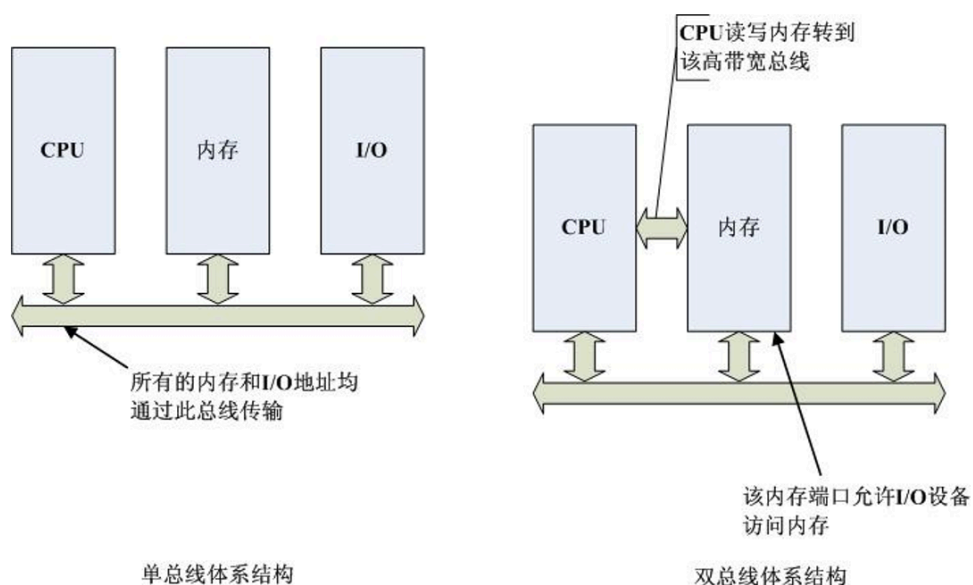
还有IO内存映射的方法

Memory-mapped I/O



总线的共享：

共享的总线方便了设计但也限制速度

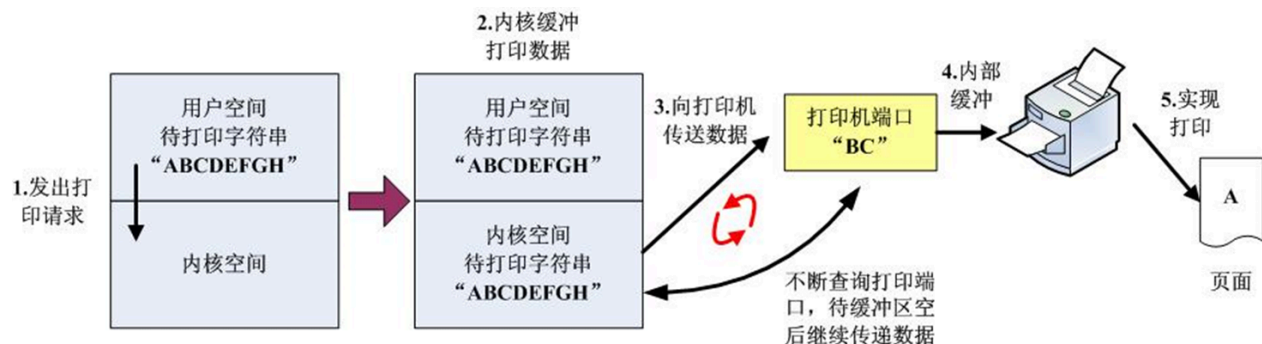


CPU怎么及时发现要传输的数据？什么时候取数据？从CPU单字节传输到DMA多字节传输(以块为单位传输)

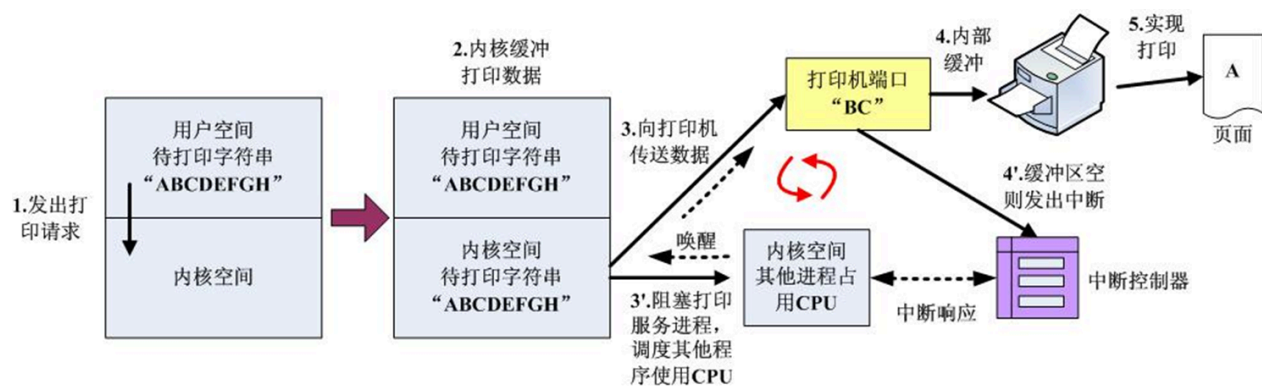
我们使用DMA来完成我们的传输，本来是一个一个传输的，但是这样太浪费CPU的时间了，所以DMA直接帮忙传输了，然后传输完了之后再跟CPU汇报即可

设备的工作模式：

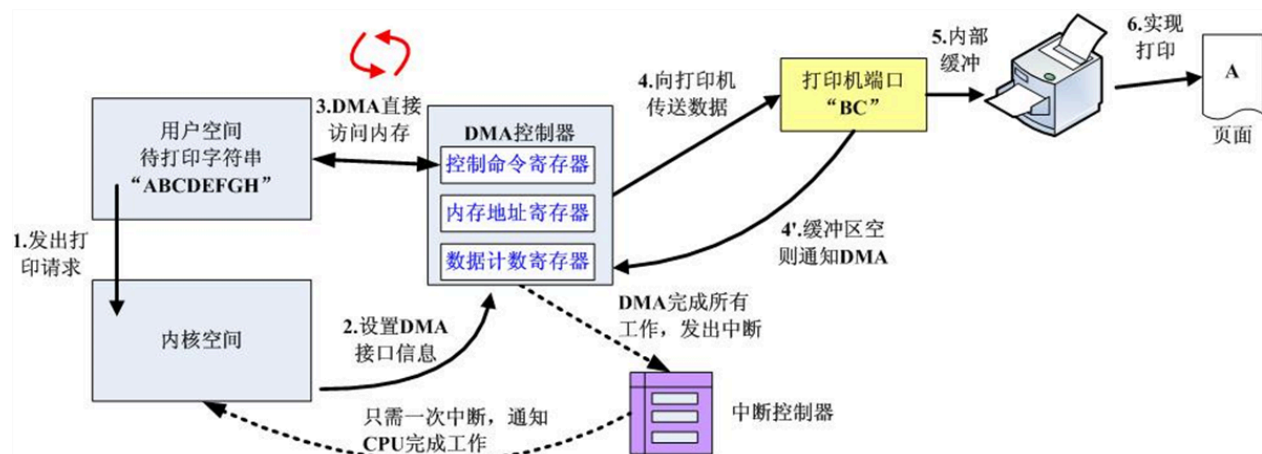
- 轮询：就是向打印机传输数据，然后一直轮询，直到打完一个字，再去放下一个字



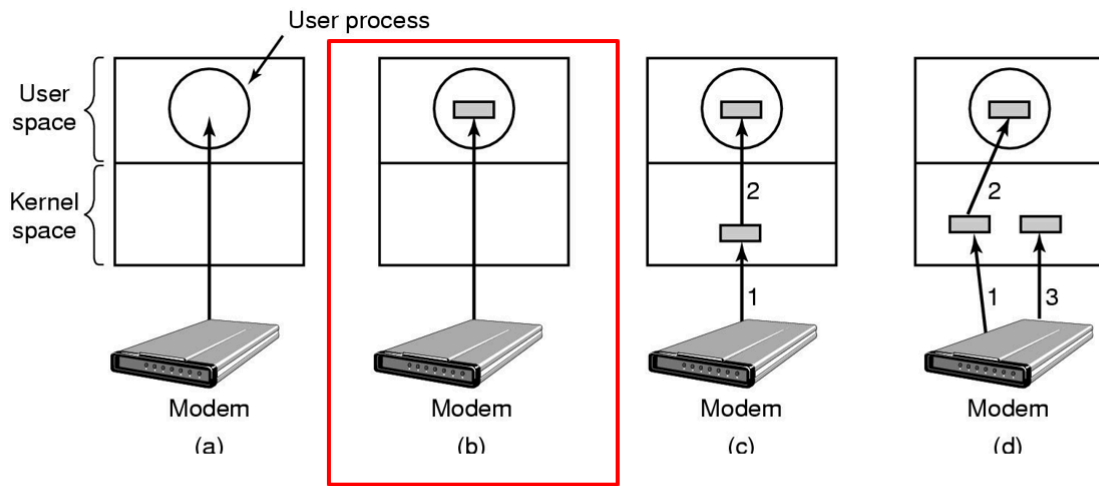
- 中断：就是直到将CPU唤醒了，CPU才会去处理对应的数据



- DMA：做一个电路，可以产生中断，可以读写数据，直到所有的数据都完成之后，DMA才会通知CPU



DMA在读内存的时候，CPU不能去访问内存，所以CPU会去休息，只能做一些与内存无关的事情



- (a) 无缓冲输入
- (b) 缓冲区放在用户空间
- (c) 缓冲区在核心空间, 然后被复制到用户空间
- (d) 在内核空间有双缓冲区

- 我们一般DMA工作完之后, 直接把数据传给用户空间就行
- 但是比较难实现, 所以我们一般使用的就是C
- DMA把数据填到内核空间内的一个位置, 然后再还给用户
- D就是, 数据太多了, 所以构建了双缓冲区

第七章 死锁